Informazione e Codici

Note

Giorgio Gambosi

UNIVERSITÀ DI ROMA "TOR VERGATA"

a.a. 2025-2026

Indice

1 Introduzione 3
- 1.1 Bit di informazione . . . 3
- 1.2 Modelli di sistemi di elaborazione dell'informazione . . . 5
- 1.3 Bit e algebra booleana . . . 6
- 1.4 Bit e circuiti . . . 8
- 1.5 Codifica . . . 9
- 1.6 Esempi di codifica . . . 10

2 Un po' di matematica dell'informazione 17
- 2.1 Misure di informazione . . . 18
- 2.2 L'entropia . . . 20
- 2.3 Perché proprio questa definizione di entropia? . . . 23
- 2.4 Esempi . . . 24
- 2.5 Entropia congiunta, condizionata e informazione mutua . . . 27
- 2.6 Altri esempi: ricerca, giochi e linguaggi . . . 30
- 2.7 Entropia del linguaggio naturale . . . 33

3 Compressione e correzione di errori 37
- 3.1 Codici su simboli, canali e teoremi fondamentali di Shannon . . . 37
- 3.2 Codici binari . . . 44
- 3.3 Codici di Huffman . . . 52
- 3.4 Codici aritmetici . . . 56
- 3.5 Codifica di Lempel–Ziv . . . 63
- 3.6 Errori e loro correzione . . . 66

Capitolo 1

Introduzione

Nei corsi di fisica si introduce molto presto una grandezza fondamentale: l'energia. Essa può assumere forme molto diverse — cinetica, potenziale, termica, elettrica, chimica — ma la sua quantità totale in un sistema isolato rimane invariata: questa è l'essenza del Primo Principio della Termodinamica. L'energia è inoltre localizzata nello spazio, può essere trasferita da un sistema a un altro, accumulata e successivamente rilasciata, e soprattutto convertita da una forma a un'altra. Gran parte dello sviluppo tecnologico dell'umanità è legato proprio al controllo di tali trasformazioni: la rivoluzione industriale, per esempio, è stata resa possibile dalla capacità di convertire l'energia chimica dei combustibili in lavoro meccanico utile. La validità del principio di conservazione si è rivelata talmente profonda che, ogni volta che sembrava essere violata, la spiegazione non è mai stata l'abbandono del principio, ma l'identificazione di una nuova forma di energia precedentemente non riconosciuta — come la relazione di Einstein $E = mc^2$, che ha rivelato come la massa stessa possa essere interpretata come energia.

È naturale chiedersi se l'informazione condivida alcune di queste proprietà. In effetti, anche l'informazione può essere localizzata su un supporto fisico, trasmessa da un punto a un altro, immagazzinata e recuperata. Tuttavia esiste una differenza cruciale: a differenza dell'energia, l'informazione non è una quantità conservata. Il Secondo Principio della Termodinamica introduce l'entropia, grandezza strettamente collegata al concetto di informazione, che nei processi spontanei tende ad aumentare: questo aumento può essere interpretato come una perdita di informazione sullo stato microscopico del sistema, ovvero come un aumento dell'incertezza. Informazione ed energia sono dunque concetti intrecciati ma non sovrapponibili: l'una obbedisce a un principio di conservazione rigoroso, l'altra è soggetta a vincoli più sottili che riflettono la direzione privilegiata dei processi naturali e il legame tra dinamica fisica, irreversibilità e conoscenza.

In questo capitolo studieremo come l'informazione possa essere quantificata, rappresentata, trasmessa, compressa e protetta: vedremo che a ognuna di queste operazioni corrisponde una teoria matematica precisa, con risultati sorprendenti e conseguenze pratiche profonde.

1.1 Bit di informazione

Obiettivo. In questa sezione introduciamo l'unità di misura dell'informazione, il bit, mostrando come essa emerga naturalmente dal legame tra informazione e incertezza.

Partiamo da un celebre rompicapo. Abbiamo 12 palline apparentemente identiche, tranne una che ha peso diverso dalle altre; non sappiamo però se sia più pesante o più leggera. Disponiamo di una bilancia a due piatti, che confronta il peso di due sottoinsiemi e può restituire esattamente tre esiti: i due piatti sono in equilibrio, il piatto sinistro è più pesante, oppure il piatto destro è più pesante. "Qual è il

numero minimo di pesate necessario per identificare la pallina anomala e stabilire se è più pesante o più leggera?"

All'inizio ci troviamo in una situazione di massima incertezza: qualunque delle 12 palline potrebbe essere quella diversa, e per ciascuna esistono due possibilità (più pesante o più leggera), per un totale di 24 stati possibili compatibili con le informazioni iniziali. L'intera sequenza di pesate è un processo di riduzione progressiva dell'incertezza: a ogni pesata alcune possibilità vengono escluse e il numero di stati compatibili diminuisce, finché non ne rimane uno solo. Al termine, abbiamo identificato la pallina corretta e sappiamo in che modo differisce dalle altre. Il rompicapo è quindi, nella sua essenza, un problema di "acquisizione di informazione".

Questa osservazione solleva domande fondamentali. Come misurare quantitativamente l'informazione acquisita? Quanta informazione è necessaria per passare da 24 possibilità a una sola? Come progettare una strategia che massimizzi, a ogni pesata, l'informazione ottenuta? Per rispondervi occorre un'unità di misura.

Definizione 1.1 (Bit). Un bit è la quantità di informazione necessaria per eliminare l'incertezza tra due alternative equiprobabili. È l'unità fondamentale di misura dell'informazione.

È importante sottolineare fin da subito che conoscere la quantità di informazione espressa in bit non equivale a conoscerne il contenuto: il bit misura "quanto" sappiamo, non "che cosa" sappiamo. Così come la lunghezza si misura in metri senza che il metro dica nulla sulla forma dell'oggetto misurato, il bit quantifica l'informazione senza specificarne il significato.

L'idea alla base della quantificazione è il legame tra informazione e incertezza: l'informazione acquisita tramite un'osservazione è tanto maggiore quanto più grande era l'incertezza prima dell'evento. Se osserviamo qualcosa di altamente prevedibile, impariamo poco; se invece osserviamo qualcosa di raro o inaspettato, acquisiamo molta informazione. Questa intuizione sarà formalizzata in modo rigoroso nel capitolo sull'entropia; per ora ci limitiamo al caso più semplice, quello degli esiti equiprobabili.

Consideriamo una situazione in cui un evento può avere diversi esiti possibili: il lancio di una moneta o di un dado, l'andamento di un titolo in borsa, il risultato di una partita, quale studente verrà interrogato. Alice osserva l'esito e vuole comunicarlo a Bruno nel modo più conciso possibile. Nel caso più semplice — lancio di una moneta equilibrata — esistono due esiti equiprobabili: testa o croce. Alice può comunicare l'esito usando un solo simbolo binario, per esempio 0 per testa e 1 per croce. In questo caso diciamo che l'informazione comunicata è pari a 1 bit.

Se invece Alice osserva il lancio di due monete, gli esiti possibili diventano quattro: (testa, testa), (testa, croce), (croce, testa), (croce, croce). Per distinguerli servono due simboli binari: 00, 01, 10, 11. L'informazione è quindi 2 bit. È interessante notare che ciascun bit corrisponde concettualmente all'esito di una singola moneta: il primo bit descrive il risultato della prima moneta, il secondo quello della seconda. In generale, se Alice osserva il lancio di n monete, esistono 2^n esiti equiprobabili e servono n bit. Ogni bit aggiuntivo "raddoppia" il numero di messaggi distinguibili.

Generalizzando: se un evento $\mathscr{E}$ può avere N esiti equiprobabili, la quantità di informazione associata all'osservazione di un esito è

$$\log_2 N \text{ bit}. \tag{1.1}$$

Il logaritmo in base 2 non è una scelta arbitraria: da un lato riflette la natura binaria dell'unità di informazione; dall'altro garantisce che l'informazione di due eventi indipendenti con N_1 e N_2 esiti sia la somma delle informazioni dei singoli eventi, poiché $\log_2(N_1 N_2) = \log_2 N_1 + \log_2 N_2$. Questa proprietà di additività è fondamentale: significa che l'informazione si accumula linearmente, esattamente come ci aspettiamo da una grandezza fisica ben definita.

Conclusione. Il bit misura la riduzione di incertezza prodotta da un'osservazione. Più alternative si escludono, più informazione si acquisisce. La relazione $\log_2 N$ traduce questo principio in una formula precisa e additiva, che costituisce il punto di partenza di tutta la teoria dell'informazione.

1.2 Modelli di sistemi di elaborazione dell'informazione

La trasmissione dell'informazione da Alice a Bruno avviene in due fasi distinte. Prima i due si accordano su come rappresentare i possibili esiti: a ogni esito deve corrispondere un messaggio ben definito, e questo insieme di regole condivise prende il nome di codice. Poi avviene la comunicazione vera e propria: Alice, osservato un certo esito, lo trasforma nel messaggio corrispondente, lo invia attraverso un canale, e Bruno, ricevuto il messaggio, ne ricostruisce l'esito originale.

Questo processo si articola in tre stadi ben distinti, mostrati in Figura 1.1:

1. **Sorgente.** L'entità che genera l'informazione: un testo, un segnale audio, un'immagine, l'esito di un esperimento, la sequenza di istruzioni di un programma.

2. **Codificatore e canale.** Il codificatore trasforma il messaggio prodotto dalla sorgente in una rappresentazione adatta al canale fisico disponibile; il canale lo trasmette o lo memorizza, eventualmente introducendo rumore o errori.

3. **Decodificatore e destinatario.** Il decodificatore inverte la trasformazione del codificatore e consegna il messaggio al destinatario, idealmente senza alcuna perdita di informazione.

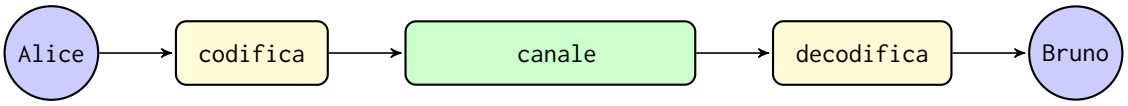

Figura 1.1: Schema della comunicazione da Alice a Bruno

Questo schema astratto è universale: si applica identicamente a un testo digitale trasmesso su Internet, a un segnale audio compresso in formato MP3, a un'immagine memorizzata su disco, a una sequenza genomica conservata in una banca dati biologica, a un messaggio radio inviato da una sonda spaziale. In tutti questi casi compaiono le stesse tre componenti — sorgente, canale, destinatario — e gli stessi due problemi fondamentali: come rappresentare l'informazione in modo efficiente, e come proteggerla dagli errori introdotti dal canale.

Dal punto di vista di Bruno, prima di ricevere il messaggio esiste un'incertezza sul valore osservato da Alice: questa incertezza è misurabile in bit. Quando il messaggio viene ricevuto e decodificato correttamente, l'incertezza si azzera e l'informazione viene acquisita. Nel processo compaiono quindi due quantità distinte, entrambe misurate in bit: l'incertezza iniziale (e quindi l'informazione potenzialmente acquisibile) e la lunghezza del messaggio effettivamente trasmesso. Il rapporto tra queste due quantità è al centro di tutta la teoria della codifica.

Le operazioni svolte dal codificatore possono essere ricondotte a tre categorie concettuali. La codifica di sorgente riduce la dimensione del messaggio eliminando ridondanze presenti nell'informazione originaria: è il problema della compressione, affrontato da formati come ZIP, MP3 o JPEG. La codifica di canale introduce deliberatamente ridondanza controllata per consentire il rilevamento e la correzione degli errori introdotti dal canale: è il problema della protezione dall'errore, alla base dei codici usati nelle

trasmissioni satellitari, nei dischi rigidi o nelle reti mobili. La modulazione, infine, converte le sequenze digitali in segnali fisici compatibili con il canale: variazioni di tensione in un cavo, impulsi luminosi in una fibra ottica, onde elettromagnetiche nell'aria.

Conclusione. Il modello sorgente–canale–decodificatore è il riferimento concettuale di tutta la teoria dell'informazione. Comprenderlo significa comprendere perché compressione e protezione dagli errori siano due facce dello stesso problema, e perché le stesse idee matematiche si applichino a sistemi fisici così diversi come reti di telecomunicazione, dispositivi di memoria e sequenze biologiche.

1.3 Bit e algebra booleana

L'informazione può essere rappresentata in modo estremamente generale come sequenza di simboli binari, cioè di valori nell'insieme $\{0, 1\}$. Questa scelta, apparentemente restrittiva, è in realtà straordinariamente potente: con soli due simboli è possibile rappresentare, elaborare e trasmettere dati di qualsiasi natura, indipendentemente dal loro significato originario. Una stessa sequenza di bit può rappresentare, a seconda del contesto e della codifica adottata, un carattere testuale, un numero intero, un campione audio, un pixel di un'immagine o un'istruzione di un programma. Il livello binario è un linguaggio universale per il trattamento dell'informazione, che separa nettamente la rappresentazione dal significato.

La matematica che governa la manipolazione di valori binari è l'algebra booleana, introdotta da George Boole nel XIX secolo per formalizzare il ragionamento logico. Nell'algebra booleana le variabili possono assumere soltanto due valori, che interpretiamo come 0 e 1 e che corrispondono anche ai valori di verità "Falso" e "Vero".

Operatori fondamentali

Tre operatori sono sufficienti a generare tutte le funzioni booleane: la negazione NOT ($\overline{x}$), il prodotto logico AND ($x \cdot y$) e la somma logica OR ($x + y$). Le loro tavole di verità sono:

x	$\overline{x}$
0	1
1	0

x	y	$x \cdot y$	$x + y$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Da questi tre operatori si derivano tutti gli altri. La funzione XOR ($x \oplus y$), che vale 1 se e solo se i due ingressi sono diversi, si scrive come $(\overline{x} \cdot y) + (x \cdot \overline{y})$: è l'addizione modulo 2, usata pervasivamente nei circuiti aritmetici e nella crittografia. Le funzioni NAND e NOR, usate frequentemente nella progettazione di circuiti perché ciascuna è realizzabile con un solo transistor e da sola è sufficiente a generare tutte le altre funzioni, sono rispettivamente $\overline{x \cdot y}$ e $\overline{x + y}$.

Quante funzioni booleane esistono?

Poiché il dominio $\{0, 1\}^k$ contiene 2^k elementi, una funzione booleana di k variabili è completamente determinata specificando il suo valore su ciascuno di essi: il numero totale di tali funzioni è 2^{2^k}. Per $k = 1$ otteniamo $2^2 = 4$ funzioni (identità, NOT, costante 0, costante 1). Per $k = 2$ otteniamo $2^4 = 16$ funzioni, elencate nella tabella seguente:

x_0	x_1	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Tra le 16 funzioni spiccano per importanza: f_1 (AND), f_7 (OR), f_6 (XOR), f_{14} (NAND), f_8 (NOR), f_0 (costante 0) e f_{15} (costante 1). La crescita rapida del numero di funzioni con k — 2^{2^k} cresce come una torre di esponenziali — riflette la straordinaria ricchezza espressiva dell'algebra booleana anche per valori piccoli di k.

Leggi dell'algebra booleana

Le proprietà più importanti, utili per semplificare espressioni e progettare circuiti efficienti, sono le seguenti:

- **Elementi neutri e nulli:** $x \cdot 1 = x$, $x + 0 = x$, $x \cdot 0 = 0$, $x + 1 = 1$.
- **Complementazione:** $x \cdot \overline{x} = 0$, $x + \overline{x} = 1$.
- **Idempotenza:** $x \cdot x = x$, $x + x = x$. (A differenza dell'algebra ordinaria, dove $x \cdot x = x^2 \neq x$ in generale.)
- **Commutatività e associatività** di AND e OR, analoghe all'algebra ordinaria.
- **Distributività reciproca:** $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$ e, simmetricamente, $x + (y \cdot z) = (x + y) \cdot (x + z)$. La seconda relazione non ha analogo nell'algebra ordinaria.
- **Assorbimento:** $x \cdot (x + y) = x$ e $x + (x \cdot y) = x$.
- **Leggi di De Morgan:** $\overline{x \cdot y} = \overline{x} + \overline{y}$ e $\overline{x + y} = \overline{x} \cdot \overline{y}$.

Le leggi di De Morgan meritano un'attenzione particolare. Esse stabiliscono una simmetria profonda tra AND e OR: la negazione di un AND è un OR di negazioni, e viceversa. In pratica, permettono di trasformare qualsiasi espressione booleana in una forma equivalente che usa solo NAND (o solo NOR), una proprietà essenziale per la progettazione di circuiti integrati in cui si vuole minimizzare il numero di tipi di porte usati.

Oltre all'interpretazione algebrica, queste operazioni hanno un'interpretazione logica diretta. La negazione $\overline{x}$ corrisponde alla negazione proposizionale; il prodotto $x \cdot y$ alla congiunzione ("x e y"); la somma $x + y$ alla disgiunzione ("x o y"). Questa corrispondenza costituisce il ponte concettuale tra ragionamento formale e realizzazione fisica dei sistemi digitali.

Conclusione. L'algebra booleana fornisce il linguaggio matematico universale per descrivere qualsiasi elaborazione di informazione binaria. Tre operatori (NOT, AND, OR) bastano a esprimere qualsiasi funzione; le leggi di De Morgan e la distributività sono gli strumenti principali per semplificare le espressioni e progettare circuiti efficienti.

1.4 Bit e circuiti

Le espressioni booleane non rimangono entità astratte: possono essere implementate fisicamente mediante reti logiche combinatorie, strutture in cui ogni operatore booleano è realizzato da un elemento fisico elementare chiamato porta logica. Una porta logica riceve in ingresso uno o due bit e produce in uscita un singolo bit, secondo la regola dell'operatore che implementa. Collegando opportunamente le porte tra loro si costruiscono circuiti capaci di realizzare qualunque funzione booleana. In Figura 1.4 sono riportate le rappresentazioni grafiche standardizzate delle porte fondamentali.

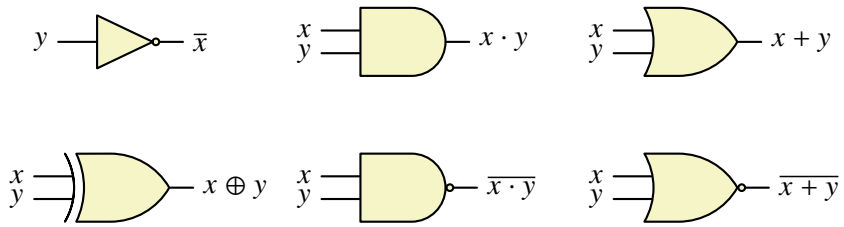

Figura 1.2: Porte logiche di base

Il collegamento tra porte avviene connettendo l'uscita di una porta all'ingresso di una o più porte successive. L'uscita di una porta può essere usata come ingresso per più componenti contemporaneamente, realizzando una duplicazione dell'informazione binaria. In Figura 1.4 sono mostrati due esempi di reti logiche che implementano espressioni booleane composte: leggendo il circuito da sinistra a destra, si segue il flusso dell'informazione dagli ingressi verso le uscite e si ricostruisce passo per passo la funzione realizzata.

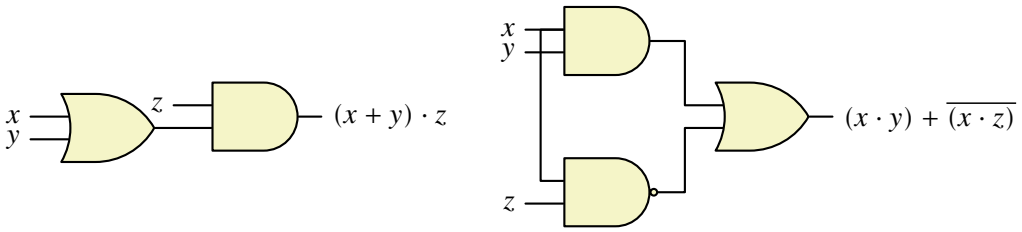

Figura 1.3: Esempi di reti logiche

Reti combinatorie e reti sequenziali

Le reti finora considerate sono prive di "retroazioni": nel grafo dei collegamenti, orientato nel verso che va dalle uscite agli ingressi, non esistono cicli. Questo significa che il valore di ogni uscita dipende unicamente dai valori correnti degli ingressi esterni, ed è calcolabile direttamente con l'algebra booleana. Reti di questo tipo prendono il nome di reti combinatorie e il loro comportamento è completamente descritto dalle funzioni booleane che implementano.

La situazione cambia radicalmente se la rete contiene cicli, cioè se l'uscita di una porta può, direttamente o indirettamente, influenzare il proprio ingresso. La Figura 1.4 mostra due esempi istruttivi. A sinistra, una singola porta NOT è collegata su se stessa: non esiste alcuna assegnazione coerente dei valori (se l'ingresso è 0, l'uscita è 1, ma questa uscita è anche l'ingresso, che dovrebbe quindi essere

1: contraddizione; e lo stesso accade assumendo ingresso uguale a 1). Il circuito non ammette stati. A destra, due porte NOT in cascata con retroazione ammettono invece esattamente due stati stabili: (0, 1) e (1, 0), dove i due numeri indicano rispettivamente l'uscita della prima e della seconda porta. In questi casi l'algebra booleana non è più sufficiente a descrivere il comportamento del circuito: è necessario introdurre un formalismo più ricco, capace di tener conto dell'evoluzione temporale e della memoria.

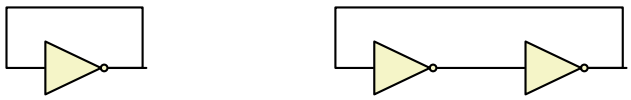

Figura 1.4: Esempi di reti logiche sequenziali

Reti logiche che contengono cicli sono dette reti sequenziali. In esse l'uscita non dipende solo dagli ingressi istantanei, ma anche dallo "stato interno" del circuito, che riflette la storia passata del sistema. I due stati stabili della rete a destra in Figura 1.4 rappresentano i due possibili valori memorizzati: si tratta del circuito più elementare con memoria, il cosiddetto "latch". Su questo principio si basano registri, contatori e, più in generale, tutti i dispositivi digitali capaci di elaborare informazione nel tempo.

Conclusione. Le reti combinatorie implementano funzioni booleane senza memoria e il loro comportamento è completamente descritto dall'algebra booleana. Le reti sequenziali aggiungono la memoria, rendendo possibile l'elaborazione di sequenze di informazione nel tempo. Questa distinzione è fondamentale in tutta l'architettura dei sistemi digitali, dalla singola cella di memoria al processore più complesso.

1.5 Codifica

Assumendo di lavorare con un alfabeto binario $\Sigma = \{0, 1\}$, affrontiamo il problema fondamentale della codifica: come rappresentare mediante sequenze di bit un insieme $\mathscr{D}$ di valori da comunicare, memorizzare o elaborare. Gli elementi di $\mathscr{D}$ possono essere di natura molto diversa: caratteri di un testo, numeri interi, istruzioni di un programma, stati di un sistema fisico. Dal punto di vista della codifica, però, ciò che conta non è il significato dei simboli, ma la "cardinalità" di $\mathscr{D}$: quante alternative distinte dobbiamo essere in grado di distinguere.

La relazione tra cardinalità e numero di bit è diretta e fondamentale. Una sequenza di k bit può assumere esattamente 2^k valori distinti, e ogni bit aggiuntivo raddoppia il numero di messaggi distinguibili. Affinché esista una corrispondenza uno-a-uno tra simboli di $\mathscr{D}$ e codifiche binarie, deve valere $2^k \geq |\mathscr{D}|$, da cui $k \geq \log_2 |\mathscr{D}|$. Poiché k deve essere un intero, la lunghezza minima richiesta è

$$k = \lceil \log_2 |\mathscr{D}| \rceil, \tag{1.2}$$

dove $\lceil x \rceil$ indica il più piccolo intero maggiore o uguale a x.

La tabella seguente riassume i casi principali che si presentano in pratica:

$\lvert\mathscr{D}\rvert$	Numero minimo di bit
1	0
2	1
$2^k,\ k > 1$	k
finito, non potenza di 2	$\lceil \log_2 \lvert\mathscr{D}\rvert \rceil$
infinito numerabile	intero arbitrariamente grande
infinito non numerabile	approssimazione mediante interi

Se $|\mathscr{D}| = 1$ non c'è incertezza da eliminare e servono 0 bit: il simbolo è noto a priori. Se $|\mathscr{D}| = 2^k$ le sequenze di lunghezza k bastano esattamente, senza sprechi né ridondanza. Se $|\mathscr{D}|$ non è una potenza di 2, alcune sequenze di bit rimarranno inutilizzate: si introduce una "ridondanza strutturale", cioè il sistema offre più codifiche di quelle strettamente necessarie. Per insiemi infiniti numerabili (come gli interi) ogni simbolo può essere codificato con una stringa finita, ma la lunghezza cresce senza limite. Per insiemi non numerabili (come i reali) occorre invece approssimare, selezionando un sottoinsieme discreto sufficientemente denso: è il principio della rappresentazione in virgola mobile, dove ogni valore memorizzabile è in realtà un'approssimazione di un numero reale "ideale".

Perché lunghezze inferiori non bastano: il principio della piccionaia

Vale la pena giustificare rigorosamente perché $k = \lceil \log_2 |\mathscr{D}| \rceil$ sia il minimo assoluto, e non semplicemente una scelta conveniente. Supponiamo, per assurdo, di voler codificare n simboli con sequenze di lunghezza al più $\lceil \log_2 n \rceil - 1$. Il numero massimo di sequenze distinte disponibili è $2^{\lceil \log_2 n \rceil - 1}$. Si può mostrare che questa quantità è strettamente minore di n in tutti i casi (sia che n sia o meno una potenza di 2). Abbiamo quindi più simboli da codificare che codici disponibili: per il principio della piccionaia (se n oggetti devono essere collocati in meno di n contenitori, almeno due oggetti finiranno nello stesso contenitore), almeno due simboli dovrebbero condividere lo stesso codice. Ma una codifica che assegna lo stesso codice a simboli distinti non è decodificabile: il destinatario non potrebbe risalire in modo univoco al simbolo originario. La contraddizione dimostra che non esiste alcuna codifica valida con meno di $\lceil \log_2 |\mathscr{D}| \rceil$ bit.

Conclusione. La relazione $k = \lceil \log_2 |\mathscr{D}| \rceil$ non è una convenzione, ma una necessità logica: nessuna codifica binaria può fare di meglio senza perdere la capacità di distinguere tutti i simboli. Ogni bit aggiuntivo raddoppia lo spazio dei messaggi distinguibili; ogni bit in meno lo dimezza, e sotto la soglia critica la decodifica univoca diventa impossibile.

1.6 Esempi di codifica

Codifica di caratteri testuali

Uno dei primi e più importanti esempi di codifica riguarda la rappresentazione dei caratteri testuali. La necessità di associare simboli grafici (lettere, cifre, punteggiatura) a sequenze di bit nasce con i primi sistemi di comunicazione e di elaborazione automatica dei testi, e ha portato allo sviluppo di standard condivisi, indispensabili affinché dispositivi diversi potessero interpretare correttamente gli stessi dati. La tabella seguente riassume l'evoluzione dei principali standard:

Standard	Bit	Simboli	Note
ASCII (1963)	7	$2^7 = 128$	95 stampabili, 33 di controllo
ASCII esteso	8	$2^8 = 256$	128 aggiuntivi, non standardizzati
Unicode (base)	16	$2^{16} = 65536$	multilingue, universale

L'ASCII (Figura 1.5) è il caso ideale dal punto di vista della codifica: $|\mathscr{D}| = 128 = 2^7$, e tutte le 2^7 sequenze di 7 bit sono utilizzate o riservate a uno scopo ben definito. I 95 codici "stampabili" coprono le lettere maiuscole e minuscole dell'alfabeto inglese, le cifre decimali e i principali simboli di punteggiatura. I 33 codici di controllo, non associati a caratteri visibili, gestiscono il flusso della comunicazione: indicano la fine di una riga, un ritorno a capo, l'inizio o la fine di una trasmissione.

L'estensione di ASCII a 8 bit ha raddoppiato lo spazio disponibile, ma ha introdotto un problema serio: il significato dei 128 codici aggiuntivi non è mai stato standardizzato in modo univoco. La stessa sequenza

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

Figura 1.5: Codice ASCII

di bit può quindi rappresentare caratteri diversi su sistemi diversi, rendendo ambigua l'interpretazione di testi scambiati tra piattaforme differenti. Questo è un esempio pratico di come la ridondanza strutturale (i bit aggiuntivi rispetto al minimo necessario) possa creare problemi di compatibilità se non gestita con cura.

Unicode risolve il problema alla radice fornendo una codifica universale per tutti i caratteri utilizzati nei sistemi di scrittura umani: non solo gli alfabeti europei, ma anche gli ideogrammi cinesi e giapponesi, l'alfabeto arabo, il devanagari, e molti altri. Con 16 bit si raggiungono $2^{16} = 65536$ simboli distinti, un numero sufficiente a coprire la grande maggioranza dei sistemi di scrittura in uso. Versioni estese di Unicode (UTF-32) usano 32 bit, portando la capacità a oltre quattro miliardi di simboli.

Conclusione. L'evoluzione da ASCII a Unicode riflette la crescita della complessità dell'informazione da rappresentare e la necessità di codifiche universali e prive di ambiguità. Il passaggio da 7 a 16 bit non è arbitrario: è dettato dalla cardinalità dell'insieme di simboli che si vuole codificare, esattamente come prevede la relazione $k = \lceil \log_2 |\mathscr{D}| \rceil$.

Codifica di interi

La codifica degli interi illustra in modo vivido come la scelta della rappresentazione non sia unica, ma dipenda dagli obiettivi applicativi. Poiché nessuna rappresentazione di lunghezza fissa può coprire tutti gli interi, è inevitabile che computazioni su interi producano talvolta valori non rappresentabili con il numero di bit a disposizione: questo fenomeno è noto come overflow ed è una delle principali sorgenti di errori nei sistemi digitali. La tabella seguente confronta i principali schemi:

Schema	Interi con k bit	Unicità dello 0	Note
BCD	cifre 0–9 (4 bit)	sì	6 codici inutilizzati
Binario unsigned	$0, \dots, 2^k - 1$	sì	semplice, solo non negativi
Codice Gray	$0, \dots, 2^k - 1$	sì	successivi differiscono di 1 bit
Segno e modulo	$-(2^{k-1} - 1), \dots, 2^{k-1} - 1$	no	due zeri
Complemento a 1	$-(2^{k-1} - 1), \dots, 2^{k-1} - 1$	no	due zeri
Complemento a 2	$-2^{k-1}, \dots, 2^{k-1} - 1$	sì	somma e diff. uniformi

Il Binary Coded Decimal (BCD) codifica separatamente ogni cifra decimale con 4 bit, permettendo di rappresentare le cifre 0–9. Con 4 bit si hanno $2^4 = 16$ sequenze disponibili, ma solo 10 vengono usate: le 6 rimanenti (da 1010 a 1111) costituiscono ridondanza strutturale. Possono essere trattate come errori, oppure mappate su valori esistenti in modo conveniente. BCD è ancora usato in applicazioni dove la conversione decimale-binaria deve essere evitata, come nei display numerici e in alcuni sistemi di calcolo finanziario.

La rappresentazione **unsigned** in base 2 è la più naturale per interi non negativi: usa tutte le 2^k sequenze senza sprechi, ed è quella impiegata tipicamente per indirizzi di memoria e contatori. Il codice Gray ha invece la proprietà che due interi consecutivi differiscono per un solo bit, come si vede nell'esempio a 4 bit:

0	1	2	3	4	5	6	7
0000	0001	0011	0010	0110	0111	0101	0100
8	9	10	11	12	13	14	15
1100	1101	1111	1110	1010	1011	1001	1000

Questa proprietà è preziosa nelle applicazioni in cui un errore su un singolo bit (per esempio in un sensore di posizione) deve produrre un errore al più unitario nel valore decodificato. Nella codifica binaria standard, invece, il passaggio da 7 (0111) a 8 (1000) comporta il cambiamento simultaneo di tutti e 4 i bit: un errore durante la transizione potrebbe produrre un valore arbitrario.

Nella rappresentazione **segno e modulo** il primo bit indica il segno (0 per positivi, 1 per negativi) e i rimanenti $k - 1$ bit codificano il valore assoluto. Con $k = 4$ si rappresentano i valori da -7 a $+7$, ma lo zero ha due rappresentazioni (0000 e 1000), il che complica i confronti e le operazioni aritmetiche.

Il complemento a 2 è lo schema dominante nei processori moderni. I numeri positivi si codificano anteponendo uno 0 alla rappresentazione binaria con $k-1$ bit; per i negativi si prende la rappresentazione del valore assoluto con $k - 1$ bit, si mantengono da destra tutti i bit fino al primo 1 incluso e si invertono i restanti, anteponendo infine un 1. Per $k = 4$:

-8	-7	-6	-5	-4	-3	-2	-1
1000	1001	1010	1011	1100	1101	1110	1111
0	1	2	3	4	5	6	7
0000	0001	0010	0011	0100	0101	0110	0111

Il vantaggio decisivo del complemento a 2 è che addizione e sottrazione si eseguono con lo stesso circuito, senza necessità di esaminare il segno: $3 - 5$ si calcola come $3 + (-5)$, dove -5 in complemento a 2 a 4 bit è 1011. La somma $0011 + 1011 = 1110$ produce 1110, che in complemento a 2 vale -2: il risultato è corretto. Questo è il motivo per cui il complemento a 2 è universalmente adottato nei processori reali.

Il complemento a 1 inverte semplicemente tutti i bit del valore assoluto per i negativi. È più semplice da costruire del complemento a 2, ma reintroduce la doppia rappresentazione dello zero (0000 e 1111 per $k = 4$) e rende le operazioni aritmetiche leggermente più complesse.

Conclusione. La scelta della rappresentazione degli interi dipende dall'applicazione: il codice Gray minimizza gli errori nelle transizioni, il complemento a 2 massimizza la semplicità dei circuiti aritmetici. In tutti i casi, il numero di bit determina l'intervallo rappresentabile, e l'overflow è il prezzo inevitabile di una rappresentazione a lunghezza fissa.

Codice genetico

La biologia offre un esempio di codifica dell'informazione che precede di miliardi di anni qualsiasi tecnologia umana. L'intera informazione genetica di un organismo è immagazzinata nel DNA come sequenza di quattro nucleotidi: adenina (A), guanina (G), timina (T) e citosina (C). Le due catene del DNA sono complementari secondo regole precise (A si lega con T, G con C), garantendo sia la stabilità della molecola sia la sua fedele replicazione.

L'informazione del DNA non agisce direttamente sulla cellula, ma viene tradotta in proteine attraverso due passi distinti. Il primo è la **trascrizione**: una porzione di DNA viene copiata in una molecola di RNA messaggero (m-RNA), in cui la timina è sostituita dall'uracile (U), con le regole di accoppiamento A→U, T→A, G→C, C→G. Il secondo è la **traduzione**: il ribosoma legge l'm-RNA a gruppi di tre nucleotidi, detti codoni, e assembla la catena di amminoacidi corrispondente, con l'intermediazione delle molecole di RNA di trasporto (t-RNA).

Il problema di codifica che emerge è il seguente: 20 amminoacidi distinti devono essere rappresentati da sequenze di nucleotidi tratte da un alfabeto di 4 simboli. Un singolo nucleotide distingue solo 4 possibilità; una coppia distingue $4^2 = 16$, ancora insufficiente. Tre nucleotidi distinguono $4^3 = 64$ combinazioni: più che sufficienti per 20 amminoacidi. Il numero minimo di nucleotidi per codone è quindi $\lceil \log_4 20 \rceil = 3$, esattamente come osservato in natura. Il codice genetico assegna a ogni codone un amminoacido (o un segnale di inizio/fine), con una ridondanza di $64/20 \approx 3.2$: in media, ogni amminoacido è codificato da circa tre codoni diversi.

Questa ridondanza non è uno spreco: è un meccanismo di protezione dagli errori selezionato dall'evoluzione. Per molti amminoacidi il terzo nucleotide del codone è irrilevante (per esempio, la prolina è codificata da CC* indipendentemente dal terzo nucleotide, e la glicina da GG*): una mutazione puntuale in quella posizione non altera la proteina prodotta. Tre codoni (UAA, UAG, UGA) segnalano la fine della sintesi proteica; il codone AUG segnala l'inizio ed è associato alla metionina, che compare quindi all'inizio di ogni proteina.

Simbolo	Amminoacido	Codone
Ala	Alanina	GC*
Arg	Arginina	CG*, AGA, AGG
Asn	Asparagina	AAU, AAC
Asp	Acido aspartico	GAU, GAC
Cys	Cisteina	UGU, UGC
Gln	Glutammina	CAA, CAG
Glu	Acido glutammico	GAA, GAG
Gly	Glicina	GG*
His	Istidina	CAU, CAC
Ile	Isoleucina	AUU, AUC, AUA
Leu	Leucina	UUA, UUG, CU*
Lys	Lisina	AAA, AAG
Met	Metionina	AUG
Phe	Fenilalanina	UUU, UUC
Pro	Prolina	CC*
Ser	Serina	UC*, AGU, AGC
Thr	Treonina	AC*
Trp	Triptofano	UGG
Tyr	Tirosina	UAU, UAC
Val	Valina	GU*
start	Metionina	AUG
stop		UAA, UAG, UGA

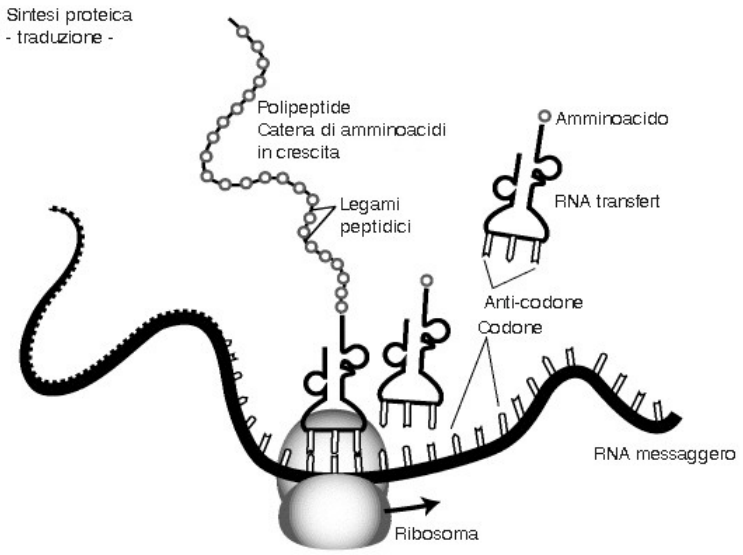

Figura 1.6: Sintesi proteica.

Conclusione. Il codice genetico è un esempio di codifica ridondante ottimizzata per la robustezza agli errori: usa tre nucleotidi da un alfabeto di 4 (equivalenti a $\log_2 64 = 6$ bit) per rappresentare 20

amminoacidi (che richiederebbero al minimo $\lceil \log_2 20 \rceil = 5$ bit). La ridondanza risultante non è casuale, ma strutturata in modo da minimizzare le conseguenze delle mutazioni puntali più comuni.

Codifiche di messaggi

Nella trasmissione e memorizzazione di messaggi su canali fisici emergono simultaneamente due problemi fondamentali, che riflettono i limiti e le imperfezioni intrinseche dei sistemi reali. Il primo è l'**efficienza**: il canale è una risorsa limitata, e si vuole trasmettere il messaggio nel modo più compatto possibile, eliminando ridondanze senza perdere informazione. Il secondo è l'**affidabilità**: il canale introduce rumore ed errori, e si vuole garantire che il messaggio ricevuto coincida con quello trasmesso, anche in presenza di disturbi. Questi due obiettivi sono in parte in competizione: aumentare la ridondanza per proteggersi dagli errori va nella direzione opposta alla compressione. Trovare il bilanciamento ottimale tra i due è il nucleo della teoria della codifica dell'informazione.

Un esempio storico orientato all'efficienza è il codice Morse, introdotto nel 1836 da Samuel Morse per la trasmissione di messaggi tramite telegrafo. I caratteri sono rappresentati come sequenze di impulsi brevi e lunghi (punti e linee) di lunghezza variabile, separati da pause. In Figura 1.7 sono riportate le codifiche dei caratteri alfabetici e delle cifre decimali.

A	·–	J	·–––	S	···	1	·–––
B	–···	K	·–·	T	–	2	··–––
C	–·–·	L	·–··	U	··–	3	···––
D	–··	M	––	V	···–	4	····–
E	·	N	–·	W	·––	5	·····
F	··–·	O	–––	X	–··–	6	–····
G	––·	P	·––·	Y	–·––	7	––···
H	····	Q	––·–	Z	––··	8	–––··
I	··	R	·–·	0	–––––	9	––––·

Figura 1.7: Codice Morse

La scelta di lunghezze variabili non è arbitraria, ma sfrutta la distribuzione non uniforme delle frequenze dei caratteri (Figura 1.8). In inglese, la lettera E compare in circa il 13% dei caratteri di un testo tipico e riceve il codice più corto (un solo punto); lettere rare come Q, X o Z, che compaiono meno dell'1% delle volte, ricevono codifiche di quattro simboli. Il principio è chiaro: i simboli più frequenti devono costare meno, in termini di simboli trasmessi, perché appaiono più spesso nel messaggio.

Il codice Morse tuttavia non soddisfa la proprietà di "prefisso": alcune parole di codice sono prefissi di altre. Per esempio, E è codificata come un singolo punto, e questa codifica è il prefisso della codifica di A (punto-linea), di I (punto-punto) e di molte altre. Questo significa che la decodifica non è determinata dalla sola sequenza di punti e linee: senza le pause che separano lettere e parole, il flusso sarebbe ambiguo. Il codice Morse sfrutta dunque le caratteristiche fisiche del canale (la temporizzazione degli impulsi) per risolvere un'ambiguità che il codice stesso introduce.

Questa limitazione permette un confronto diretto con i codici di Huffman, che incontreremo più avanti. Anche Huffman assegna parole di codice più corte ai simboli più frequenti e più lunghe a quelli

E	132	S	61	U	24
T	104	H	53	G,P,Y	20
A	82	D	38	W	19
O	80	L	34	B	14
N	71	F	29	V	9
R	68	C	27	K	4
I	63	M	25	X,J,Q,Z	1

Figura 1.8: Frequenza relativa dei caratteri in inglese (su 1000)

rari, perseguendo lo stesso obiettivo di minimizzazione della lunghezza media. La differenza fondamentale è che i codici di Huffman sono costruiti in modo da soddisfare rigorosamente la proprietà di prefisso: nessuna parola di codice è il prefisso di un'altra. Ciò garantisce una decodifica univoca e immediata, senza necessità di separatori o informazioni temporali aggiuntive. Si può quindi interpretare il codice Morse come un precursore empirico e intuitivo dei codici ottimali a lunghezza variabile: esso implementa in modo naturale il principio di adattamento alle statistiche della sorgente, ma senza una formalizzazione matematica né una garanzia teorica di ottimalità. I codici di Huffman, come vedremo, trasformano questa intuizione in una procedura costruttiva con garanzie precise.

Conclusione. Efficienza e affidabilità sono i due obiettivi fondamentali della codifica, e sono in tensione tra loro. Il codice Morse mostra come il primo possa essere perseguito adattando empiricamente la lunghezza delle codifiche alle frequenze dei simboli. La teoria di Shannon, che svilupperemo nei capitoli successivi, trasformerà questa intuizione in risultati rigorosi: una formula per il limite teorico della compressione (l'entropia), e una procedura costruttiva per raggiungerlo (i codici di Huffman e, più in generale, la codifica aritmetica).

Capitolo 2

Un po' di matematica dell'informazione

Informazione e incertezza: un'introduzione intuitiva

Supponiamo di osservare, uno dopo l'altro, i simboli prodotti da una sorgente di informazione. Il nostro obiettivo è quantificare quanta informazione possediamo, in un dato istante, a proposito del prossimo simbolo che comparirà. È importante sottolineare fin da subito una distinzione fondamentale: una volta che il simbolo è stato osservato, non esiste più alcuna incertezza sul suo valore. Prima dell'osservazione, invece, ci troviamo in una condizione di incertezza che può assumere livelli molto diversi a seconda del contesto e della sorgente che stiamo guardando.

Questa osservazione suggerisce un collegamento profondo tra i concetti di informazione e di incertezza. L'informazione acquisita attraverso l'osservazione di un simbolo può essere naturalmente identificata con l'incertezza che avevamo su quel simbolo "prima" dell'osservazione. In questa prospettiva, l'informazione non è una proprietà intrinseca del simbolo in sé, ma è piuttosto una misura della riduzione di incertezza prodotta dall'atto osservativo. Maggiore è l'incertezza iniziale, maggiore sarà l'informazione potenzialmente acquisibile. Una sorgente che produce sempre lo stesso simbolo non ci dice nulla di nuovo: l'osservazione non aggiunge nulla alla nostra conoscenza del mondo.

Prima che il simbolo venga osservato, siamo in una condizione di incertezza che dipende fortemente da ciò che sappiamo sulla sorgente. Se i simboli prodotti sono bit e dall'osservazione delle sequenze passate risulta che il valore 1 compare nel 99% dei casi, allora la nostra incertezza sul valore del prossimo bit è relativamente bassa: ci aspettiamo quasi certamente un 1. Al contrario, se i due valori 0 e 1 compaiono con la stessa frequenza del 50%, l'incertezza è massima e non possiamo fare previsioni migliori di un semplice lancio di moneta. Intuitivamente, queste due situazioni non dovrebbero essere trattate allo stesso modo dal punto di vista informativo, e una buona teoria dell'informazione deve essere in grado di catturare questa differenza in modo preciso e numerico.

Diventa quindi necessario individuare un criterio quantitativo in grado di esprimere questa differenza di incertezza. Un primo approccio naturale consiste nel considerare il numero minimo di bit necessari per comunicare il valore del simbolo osservato. Se i valori possibili sono due, come nel lancio di una moneta, un singolo bit è sufficiente. In generale, se i valori possibili sono 2^k, allora k bit risultano sufficienti per rappresentarli tutti. Seguendo questa linea di pensiero, si potrebbe tentare di quantificare l'informazione acquisita osservando un simbolo come il numero minimo di bit necessari per comunicarne il valore.

Tuttavia, questo approccio presenta alcune difficoltà. In primo luogo, il numero di bit deve essere necessariamente intero. Se il numero di valori possibili non è una potenza di 2, questo comporta inevitabilmente un utilizzo inefficiente dei bit disponibili. Ad esempio, se i valori possibili sono 5, è necessario utilizzare $\lceil \log_2 5 \rceil = 3$ bit per rappresentare ciascun simbolo, ma con 3 bit si potrebbero rappresen-

tare fino a 8 valori distinti. Tale inefficienza può essere ridotta considerando non singoli simboli, ma sequenze. Una coppia di simboli con 5 valori ciascuno genera $5^2 = 25$ combinazioni, rappresentabili con $\lceil \log_2 25 \rceil = 5$ bit, ovvero 2.5 bit per simbolo. All'aumentare della lunghezza delle sequenze considerate, il numero medio di bit per simbolo decresce e tende asintoticamente al valore $\log_2 5 \approx 2.32$. Questo suggerisce che $\log_2 5$ sia la quantità "naturale" di informazione associata a una sorgente con 5 esiti equiprobabili.

In secondo luogo, e questo è forse il punto più importante, i diversi valori che un simbolo può assumere non sono necessariamente equiprobabili. Se uno dei valori possibili ha probabilità 1 e tutti gli altri probabilità 0, allora l'osservazione del simbolo non fornisce alcuna informazione aggiuntiva, non esisteva alcuna incertezza precedente: in questo caso, non è necessario comunicare nulla, e il numero di bit richiesto è pari a 0. Ne segue che una definizione sensata di informazione deve dipendere in modo esplicito dalla probabilità degli eventi: eventi più probabili devono avere associata una misura di informazione minore rispetto a eventi rari.

Un'ulteriore proprietà desiderabile per una misura dell'informazione è l'"additività". Se due eventi x e y sono indipendenti (cioè il verificarsi di uno non influenza le probabilità dell'altro) allora l'informazione acquisita osservando entrambi dovrebbe essere pari alla somma delle informazioni acquisite osservandoli separatamente. Questa proprietà è essenziale per costruire una teoria coerente: se lanciamo due monete indipendenti, l'informazione totale dovrebbe essere il doppio di quella fornita da un singolo lancio.

Infine, prima dell'osservazione non sappiamo quale valore si realizzerà. Una misura naturale dell'informazione attesa è quindi la media dell'informazione associata ai diversi esiti, pesata dalle rispettive probabilità: questa quantità prende il nome di "entropia" e sarà al centro della trattazione che segue.

2.1 Misure di informazione

Resta ora da stabilire come la riduzione di incertezza prodotta dall'osservazione possa essere quantificata in modo coerente, confrontabile e matematicamente ben definito. Prima di arrivare alla formula, è utile elencare esplicitamente le proprietà che vogliamo che una misura dell'informazione soddisfi:

1. **Dipendenza dalla probabilità.** L'informazione fornita da un esito deve dipendere solo dalla sua probabilità, non dal suo valore specifico.

2. **Zero per eventi certi.** Se un esito ha probabilità 1, la sua osservazione non fornisce alcuna informazione: non c'era nulla di incerto da risolvere.

3. **Additività per eventi indipendenti.** Se due eventi sono indipendenti, l'informazione totale è la somma delle informazioni dei singoli eventi.

L'idea fondamentale, introdotta da Claude Shannon negli anni Quaranta del Novecento, è sorprendentemente semplice nella sua formulazione ma straordinariamente potente nelle sue conseguenze: l'informazione associata a un singolo esito non dipende dal valore del simbolo in sé, ma da quanto tale valore fosse improbabile prima dell'osservazione. Se un esito era praticamente certo, la sua osservazione non aggiunge nulla di nuovo: sapevamo già cosa aspettarci. Se invece era molto improbabile, la sua comparsa elimina una grande quantità di incertezza e risulta quindi altamente informativa.

Seguendo questa intuizione, Shannon propose di misurare l'informazione associata a un singolo esito A_i mediante la quantità

$$I(A_i) = \log_2 \frac{1}{p(A_i)} = -\log_2 p(A_i), \tag{2.1}$$

che viene detta informazione di Shannon. L'unità di misura è il "bit" quando si usa il logaritmo in base 2, il "nat" con il logaritmo naturale, e il "trit" con il logaritmo in base 3. Nella maggior parte dei contesti pratici si usa la base 2 e i bit, poiché è naturale pensare all'informazione come al numero di domande binarie (sì/no) necessarie per determinare l'esito.

È immediato verificare che questa definizione soddisfa tutte e tre le proprietà elencate sopra. L'informazione è sempre non negativa, poiché $0 < p(A_i) \leq 1$ implica $-\log_2 p(A_i) \geq 0$. Se $p(A_i) = 1$, l'informazione è nulla. Al diminuire della probabilità l'informazione cresce: un evento con probabilità $\frac{1}{2}$ fornisce 1 bit; uno con probabilità $\frac{1}{4}$ ne fornisce 2; uno con probabilità $\frac{1}{1000}$ ne fornisce circa 10. In questo senso, l'informazione di Shannon può essere vista come una misura quantitativa della "sorpresa" associata all'osservazione di un esito: più raro è un evento, più siamo sorpresi quando si verifica, e più informazione acquisiamo.

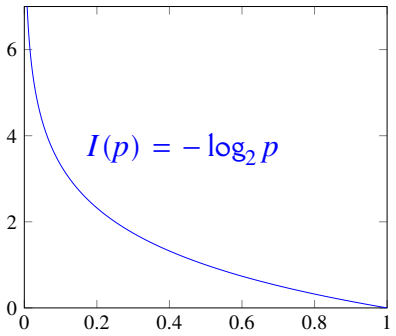

Figura 2.1: Informazione di Shannon in funzione della probabilità dell'esito. Al diminuire della probabilità, l'informazione cresce senza limite.

In Figura 2.1 è mostrato l'andamento dell'informazione di Shannon in funzione della probabilità dell'esito. La forma della curva rende evidente l'interpretazione intuitiva della misura: eventi rari corrispondono a un'elevata informazione, mentre eventi frequenti sono poco informativi. Vale la pena soffermarsi su un altro aspetto: la curva non è lineare. Questo significa che raddoppiare l'improbabilità di un evento non raddoppia semplicemente l'informazione, ma la aumenta in modo logaritmico. La scelta del logaritmo non è arbitraria: è l'unica funzione in grado di garantire l'additività dell'informazione per eventi indipendenti.

Esempio 2.1.1

Consideriamo una classe composta da 32 studenti, di cui 2 ragazze e 30 ragazzi. Se viene scelto casualmente, con stessa probabilità, uno studente, l'incertezza iniziale sulla sua identità è pari a 5 bit, poiché con 5 bit possiamo distinguere tutti i $32 = 2^5$ possibili esiti: ogni studente ha probabilità $\frac{1}{32}$ di essere selezionato.
All'evento relativo alla selezione è associato anche un evento secondario, ossia il sesso dello studente estratto. In questo caso, gli esiti possibili sono due, ma con probabilità molto diverse: $p(M) = \frac{30}{32}$ e $p(F) = \frac{2}{32}$. Se veniamo a sapere che lo studente estratto è una ragazza, allora sappiamo anche che si tratta di una delle sole 2 ragazze della classe: è sufficiente un solo bit per distinguere quale delle due sia stata selezionata. L'incertezza residua è quindi pari a 1 bit, e l'informazione acquisita è

$$5 - 1 = 4 \text{ bit.} \tag{2.2}$$

Se invece lo studente estratto è maschio, l'incertezza residua è pari a $\log_2 30 \approx 4.91$ bit, e l'informazione acquisita risulta essere

$$5 - \log_2 30 \approx 0.09 \text{ bit}. \tag{2.3}$$

Questo esempio mostra chiaramente qualcosa che forse sembra controintuitivo a prima vista: sapere che lo studente è una ragazza ci dice "molto di più" che sapere che è maschio. La ragione è ovvia una volta che la si vede: le ragazze sono pochissime, quindi il solo fatto di sapere che lo studente estratto è una ragazza riduce drasticamente le possibilità. Sapere che è maschio, al contrario, è quasi scontato e riduce di poco il numero di candidati possibili. In termini informativi, gli eventi rari producono una riduzione significativa dell'incertezza, mentre gli eventi molto probabili aggiungono poca informazione nuova.

L'additività dell'informazione di Shannon. Se due eventi A e B sono indipendenti, la probabilità congiunta è $p(A, B) = p(A) \cdot p(B)$. La proprietà di additività si verifica naturalmente grazie alla proprietà dei logaritmi:

$$I(A, B) = \log_2 \frac{1}{p(A, B)} = \log_2 \frac{1}{p(A)p(B)} = \log_2 \frac{1}{p(A)} + \log_2 \frac{1}{p(B)} = I(A) + I(B). \tag{2.4}$$

Le informazioni si sommano quando le osservazioni sono indipendenti, esattamente come ci aspettiamo.

Conclusione. L'informazione di Shannon è la misura della sorpresa prodotta dall'osservazione di un esito: quanto più era improbabile, tanto più è informativo. Tre proprietà naturali — dipendenza dalla probabilità, zero per eventi certi, additività per eventi indipendenti — la determinano in modo quasi unico, come vedremo nella sezione dedicata agli assiomi di Khinchin.

2.2 L'entropia

Due livelli distinti. Finora abbiamo considerato l'informazione che acquisiamo "dopo" aver osservato un particolare valore: questa è l'informazione di Shannon di quel singolo esito. In molte situazioni, tuttavia, siamo interessati a caratterizzare la sorgente "prima" dell'osservazione, cioè a quantificare quanta informazione ci aspettiamo "mediamente" di ottenere osservando un suo simbolo. Questi due livelli devono essere tenuti distinti: l'informazione di Shannon misura la sorpresa per un esito specifico; l'entropia misura l'incertezza complessiva della sorgente, ovvero la media di quella sorpresa su tutti i possibili esiti.

Dati gli esiti possibili $A_1, \dots, A_n$ con probabilità $p(A_1), \dots, p(A_n)$, l'entropia è definita come la media dell'informazione di Shannon pesata con le rispettive probabilità:

$$H(A_1, \dots, A_n) = \sum_{i=1}^{n} p(A_i) \log_2 \frac{1}{p(A_i)}. \tag{2.5}$$

In parole semplici: l'entropia è l'informazione media prodotta dalla sorgente. Una sorgente molto prevedibile ha entropia bassa; una sorgente imprevedibile ha entropia alta.

In Figura 2.2 è rappresentata l'entropia nel caso binario, in cui la sorgente può produrre due soli esiti, uno con probabilità p e l'altro con probabilità $1 - p$. L'entropia assume il valore massimo per $p = \frac{1}{2}$, cioè quando i due esiti sono equiprobabili e la nostra incertezza è massima. Al contrario, quando p tende a 0 o a 1, l'entropia tende a zero: la sorgente diventa prevedibile e l'osservazione fornisce poca

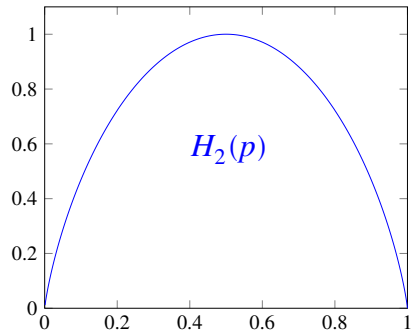

Figura 2.2: Entropia binaria $H_2(p)$ in funzione di p. Il massimo si raggiunge per $p = \frac{1}{2}$, quando i due esiti sono equiprobabili.

o nessuna informazione nuova. Questo comportamento non è specifico del caso binario: in generale, l'entropia è massima quando tutti gli esiti sono equiprobabili e diminuisce al crescere dello squilibrio nella distribuzione.

Additività per variabili indipendenti. L'entropia condivide con l'informazione di Shannon la proprietà di additività per variabili indipendenti. Se X e Y sono due variabili indipendenti, allora $H(X,Y) = H(X) + H(Y)$. La dimostrazione segue direttamente dalla definizione di entropia congiunta: poiché $p(x,y) = p(x)p(y)$, il logaritmo si fattorizza e i contributi di X e Y si separano,

$$\begin{aligned} H(X,Y) &= -\sum_x \sum_y p(x,y) \log_2 p(x,y) \\ &= -\sum_x \sum_y p(x)p(y)\left[\log_2 p(x) + \log_2 p(y)\right] \\ &= -\sum_x p(x) \log_2 p(x) \underbrace{\sum_y p(y)}_{=1} - \sum_y p(y) \log_2 p(y) \underbrace{\sum_x p(x)}_{=1} \\ &= H(X) + H(Y). \end{aligned} \tag{2.6}$$

Il risultato è intuitivo: la complessità combinata di due processi indipendenti è la somma delle loro complessità individuali.

Esempio numerico. Per rendere più concreto il significato delle definizioni introdotte, consideriamo una sorgente binaria con due simboli A_1 e A_2, con probabilità $p(A_1) = 0.9$ e $p(A_2) = 0.1$. Questa sorgente è fortemente sbilanciata: il simbolo A_1 compare quasi sempre, mentre A_2 è raro. L'informazione di Shannon associata ai due esiti è:

$$I(A_1) = -\log_2(0.9) \approx 0.15 \text{ bit}, \qquad I(A_2) = -\log_2(0.1) \approx 3.32 \text{ bit}. \tag{2.7}$$

Osservare il simbolo A_1 fornisce pochissima informazione, poiché era già altamente prevedibile; al contrario, osservare A_2 è sorprendente e riduce drasticamente l'incertezza. L'entropia della sorgente, ovvero l'informazione media attesa, è:

$$H(A_1, A_2) = 0.9 \times 0.15 + 0.1 \times 3.32 \approx 0.47 \text{ bit}. \tag{2.8}$$

L'entropia è relativamente bassa: in media, ogni simbolo prodotto dalla sorgente fornisce meno di mezzo bit di informazione, coerentemente con il fatto che la sorgente è altamente prevedibile. Per confronto,

una sorgente binaria equiprobabile con $p(A_1) = p(A_2) = 0.5$ ha entropia pari a 1 bit, il massimo possibile per una sorgente binaria.

Entropia e informazione parziale: l'esempio della classe. L'entropia non descrive soltanto l'incertezza iniziale, ma si presta anche a quantificare come tale incertezza cambi quando si acquisisce informazione parziale sul sistema. Riprendiamo l'esempio della classe di 32 studenti, introdotto nella sezione precedente, e rileggiamolo ora in termini di entropia.

Esempio 2.2.1

Consideriamo la classe precedente di 32 studenti, con 2 ragazze e 30 ragazzi, e supponiamo di essere interessati a chi sarà il primo studente (o studentessa) che entrerà in classe domattina. L'informazione che otterremo nel momento in cui osserveremo chi effettivamente sarà il primo entrato sarà, come visto, pari a $I = -\log_2 \frac{1}{32} = 5$ bit. L'incertezza media, vale a dire l'entropia, sarà evidentemente la somma di tutte queste informazioni ognuna moltiplicata per la rispettiva probabilità, che abbiamo assunto essere la stessa per tutti gli studenti:

$$H = \sum_{i=1}^{32} \frac{1}{32} \log_2 32 = 5. \tag{2.9}$$

Questo ci dice semplicemente che, se ogni esito dell'evento ha la stessa probabilità ($\frac{1}{32}$) e fornisce la stessa informazione (5 bit), allora anche la media, e quindi l'entropia, assume lo stesso valore.

Supponiamo ora di essere interessati al sesso del primo studente (o studentessa) che entrerà in classe. Dato che c'è una probabilità $p_M = \frac{30}{32} = 0.938$ che il primo studente ad arrivare sarà un maschio e solo $p_F = \frac{2}{32} = 0.062$ che sia femmina, l'informazione associata ad ognuno dei due eventi sarà

$$I_M = -\log_2 p_M \simeq 0.093$$
$$I_F = -\log_2 p_F = 4$$

e l'entropia risultante sarà

$$H = p_M \cdot I_M + p_F \cdot I_F \simeq 0.337. \tag{2.10}$$

Consideriamo ora il caso in cui ad entrare è stato un maschio: a questo punto, se torniamo a interessarci a chi è lo studente, l'entropia sarà

$$H_M = \sum_{i=1}^{30} \frac{1}{30} \log_2 30 \simeq 4.907, \tag{2.11}$$

e nel caso di una femmina sarà

$$H_F = \sum_{i=1}^{2} \frac{1}{2} \log_2 2 = 1. \tag{2.12}$$

Come si può notare, risulta che $H = I_M + H_M = I_F + H_F$. Questo ci dice che dell'incertezza iniziale rispetto allo studente che arriverà per primo (5 bit da acquisire), ne resta una gran parte (4.907 bit) nel caso in cui scopriamo che si tratta di un maschio, per cui sapere di chi effettivamente si tratta ci darà più informazione (esattamente quei 4.907 bit). Al contrario, se scopriamo che si tratta di una femmina, l'incertezza rimanente è molto minore (1 solo bit) da acquisire verificando di quale delle due ragazze si tratta. L'informazione acquisita nel corso di una sequenza di osservazioni si somma, passaggio dopo passaggio, fino a esaurire tutta l'incertezza iniziale.

Conclusione. L'entropia è l'informazione media prodotta dalla sorgente: misura non quanto siamo sorpresi da un esito specifico, ma quanto siamo incerti prima di osservarlo. Massima per distribuzioni uniformi, minima per distribuzioni concentrate su un singolo esito, l'entropia è la quantità fondamentale che lega incertezza, compressione e comunicazione.

Questo esempio prefigura già i concetti di entropia condizionata e di informazione mutua, che svilupperemo nella Sezione 2.5; prima di arrivarvi, vale la pena chiedersi se la formula dell'entropia qui introdotta sia l'unica scelta possibile, o se esistano misure di incertezza altrettanto legittime.

2.3 Perché proprio questa definizione di entropia?

Arrivati a questo punto, è naturale chiedersi se la definizione di entropia introdotta finora sia davvero inevitabile, oppure se rappresenti soltanto una scelta conveniente tra molte possibili. Una risposta profonda e definitiva a questa domanda è stata fornita nel 1956 dal matematico russo Aleksandr Khinchin con un approccio "assiomatico": invece di partire da una formula e verificarne le proprietà, si individuano alcune proprietà che qualunque misura ragionevole dell'incertezza dovrebbe soddisfare, e si studia quali funzioni siano compatibili con tali richieste. Il risultato è notevole: l'entropia di Shannon emerge come l'unica scelta possibile, a meno di una costante moltiplicativa.

I quattro assiomi. Consideriamo una funzione $H_r(p_1, \dots, p_r)$ definita su r probabilità con $\sum_{i=1}^{r} p_i = 1$, che vogliamo interpretare come misura dell'incertezza. Richiediamo che soddisfi le seguenti proprietà.

1. Continuità e simmetria. La funzione è continua rispetto alle probabilità e simmetrica rispetto ai suoi argomenti. La continuità richiede che piccole variazioni nelle probabilità producano piccoli cambiamenti nell'incertezza. La simmetria richiede che l'incertezza non dipenda dall'etichetta assegnata agli esiti: contano solo le probabilità, non i nomi dei simboli.

2. Invarianza rispetto agli eventi impossibili. Aggiungere un esito con probabilità nulla non modifica l'incertezza:

$$H_{r+1}(p_1, \dots, p_r, 0) = H_r(p_1, \dots, p_r). \tag{2.13}$$

Un evento impossibile non contribuisce all'incertezza. Questa è una semplice richiesta di coerenza.

3. Massimo per la distribuzione uniforme. Tra tutte le distribuzioni su r esiti, quella che massimizza l'incertezza è quella uniforme:

$$H_r(p_1, \dots, p_r) \leq H_r\left(\frac{1}{r}, \dots, \frac{1}{r}\right). \tag{2.14}$$

Quando non abbiamo ragione per preferire un esito agli altri, l'incertezza deve essere massima.

4. Decomposizione per osservazioni a più stadi. Supponiamo di avere $r = m \cdot k$ esiti suddivisi in m gruppi da k esiti ciascuno, con probabilità $q_j = \sum_i p_{j,i}$ per il gruppo j. L'incertezza totale deve decomporsi in modo coerente come somma dell'incertezza sulla scelta del gruppo e dell'incertezza residua all'interno di ciascun gruppo:

$$H_r(p_{1,1}, \dots, p_{m,k}) = H_m(q_1, \dots, q_m) + \sum_{j=1}^{m} q_j H_k\left(\frac{p_{j,1}}{q_j}, \dots, \frac{p_{j,k}}{q_j}\right). \tag{2.15}$$

Questa è la proprietà più "operativa" della lista: dice che possiamo calcolare l'incertezza totale decomponendola in stadi successivi. Il quarto assioma, insieme agli altri, esclude praticamente ogni alternativa alla formula di Shannon: la struttura che richiediamo è così stringente da lasciare pochissima libertà.

Il risultato di Khinchin. Imponendo tutte queste proprietà, le uniche funzioni H_r possibili hanno necessariamente la forma

$$H_r(p_1, \dots, p_r) = -c \sum_{i=1}^{r} p_i \log_2 p_i, \tag{2.16}$$

dove c è una costante positiva. Scegliendo $c = 1$ e il logaritmo in base 2, l'entropia viene misurata in bit. La costante c è irrilevante dal punto di vista concettuale: cambia solo l'unità di misura, così come scegliere tra chilometri e miglia cambia il numero ma non la distanza.

Conclusione. L'entropia di Shannon non è una scelta arbitraria tra molte possibili: è l'unica funzione compatibile con le proprietà che ci aspettiamo da qualunque misura ragionevole dell'informazione. In un certo senso, Shannon non ha inventato l'entropia, l'ha scoperta: essa era già nascosta nelle proprietà che volevamo dalla nostra misura di incertezza.

2.4 Esempi

I concetti introdotti fin qui trovano applicazione immediata in problemi classici di ricerca e decisione. Il filo comune è sempre lo stesso: l'entropia fornisce un limite inferiore al numero di operazioni necessarie, e la strategia ottimale è quella che massimizza la riduzione di entropia a ogni passo.

Le 12 palline

Il problema e il suo spazio dei casi. Una delle dodici palline ha peso diverso dalle altre, senza che si sappia se sia più pesante o più leggera. Abbiamo a disposizione una bilancia a due piatti. Ogni pallina può essere quella anomala, e per ciascuna pallina sono possibili due casi (più pesante o più leggera): in totale $12 \times 2 = 24$ esiti distinti, tutti equiprobabili.

Il limite informativo. Ogni pesata può produrre tre esiti distinti: piatto destro più pesante, piatto sinistro più pesante, equilibrio. Una singola pesata distingue al massimo 3 casi; k pesate al massimo 3^k. Poiché $3^3 = 27 > 24$, tre pesate sono teoricamente sufficienti, mentre due ($3^2 = 9 < 24$) non lo sono. Questo è un limite che discende direttamente dalla quantità di informazione disponibile, non da una proprietà specifica delle pesate.

Usando i trit (logaritmo in base 3, più naturale per uno strumento a tre esiti), l'entropia iniziale del problema è:

$$H_3 = \log_3 24 \approx 2.893 \text{ trit}. \tag{2.17}$$

Poiché ciascuna pesata può fornire al massimo 1 trit di informazione (quando i tre esiti sono equiprobabili), tre pesate sono non solo sufficienti ma anche necessarie.

La strategia ottimale. Il principio guida è immediato: ogni pesata dovrebbe distribuire il più uniformemente possibile i casi ancora compatibili tra i tre esiti, massimizzando la riduzione di entropia a ogni passo. Questa è la strategia greedy applicata all'albero decisionale.

Introduciamo la notazione i^+ per "la pallina i è più pesante" e i^- per "la pallina i è più leggera". Consideriamo prima una scelta ingenua: poniamo le palline $1, 2, 3, 4, 5, 6$ su un piatto e $7, 8, 9, 10, 11, 12$ sull'altro. In questo caso, l'esito "equilibrio" ha probabilità zero (la pallina anomala sta su uno dei due piatti), e i casi si distribuiscono come $p_D = p_S = \frac{1}{2}$, $p_E = 0$. L'entropia di questa pesata è:

$$H_p = \log_3 2 \approx 0.631 \text{ trit}, \tag{2.18}$$

molto inferiore al massimo di 1 trit. Dopo questa pesata, l'entropia residua sarebbe $2.893 - 0.631 = 2.262$ trit, che non può essere eliminata con sole due pesate da 1 trit ciascuna. La scelta è inefficiente perché non sfrutta il terzo esito della bilancia.

Una scelta migliore pone le palline $1, 2, 3, 4$ su un piatto e $5, 6, 7, 8$ sull'altro, lasciandone quattro da parte. In questo caso i 24 casi si distribuiscono equamente tra i tre esiti (8 casi ciascuno), ottenendo $p_D = p_S = p_E = \frac{1}{3}$ e l'entropia massima di 1 trit.

Continuando con la seconda pesata, nel caso in cui la prima abbia dato esito "destra più pesante", i casi possibili sono $1^+, 2^+, 3^+, 4^+, 5^-, 6^-, 7^-, 8^-$. Ponendo le palline $1, 2, 5$ su un piatto e $3, 4, 7$ sull'altro, i tre sottoinsiemi risultanti hanno cardinalità 3, 3 e 2, con entropia:

$$H_p = \frac{3}{8}\log_3\frac{8}{3} + \frac{3}{8}\log_3\frac{8}{3} + \frac{1}{4}\log_3 4 \approx 0.985 \text{ trit.} \tag{2.19}$$

Questo valore è molto vicino a 1 trit ma non lo raggiunge esattamente, perché con 8 casi non è possibile dividerli in tre gruppi perfettamente uguali. Dopo due pesate rimangono in ogni ramo esattamente tre casi compatibili, e la terza pesata li distingue univocamente.

L'interpretazione in termini di codifica. Ogni sequenza di tre pesate produce una parola ternaria di lunghezza 3. L'insieme di tutte queste sequenze ha cardinalità $3^3 = 27 > 24$. La procedura di pesata associa implicitamente a ciascuno dei 24 casi una parola ternaria distinta: il problema delle 12 palline è un problema di "codifica", in cui si vuole identificare uno tra 24 messaggi possibili usando un alfabeto ternario con il minimo numero di simboli. L'entropia iniziale $\log_3 24 \approx 2.893$ indica che bastano 3 simboli ternari, e la strategia ottimale è quella che rende l'albero decisionale il più bilanciato possibile.

Riassunto della strategia. La strategia ottimale è una procedura iterativa: a ogni passo si considera l'insieme dei casi ancora possibili, si sceglie la pesata che massimizza la riduzione di entropia, e si aggiorna l'insieme in base all'esito osservato. Dal punto di vista geometrico, l'albero decisionale risultante (mostrato in Figura 2.3) è un albero ternario di profondità 3 con nodi il più possibile bilanciati.

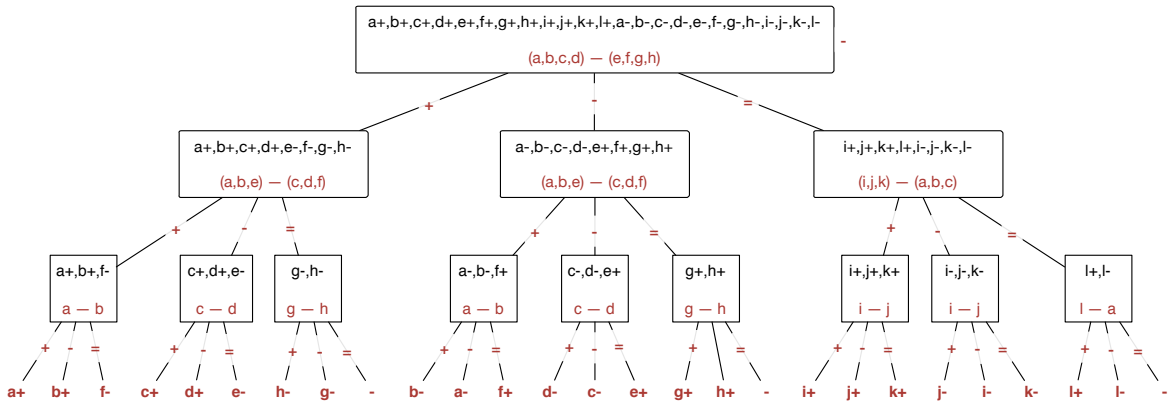

Figura 2.3: Albero di decisione risultante. I casi possibili sono rappresentati da una lettera che indica la pallina di peso diverso e un simbolo + o − a indicare se il suo peso è maggiore o minore delle altre.

L'albero di decisione è stato ottenuto considerando per ogni nodo tutte le possibili partizioni dell'insieme degli esiti possibili e scegliendo quella più informativa. Un metodo di questo tipo viene detto greedy (goloso) perché costruisce la soluzione complessiva scegliendo ad ogni passo l'alternativa localmente più conveniente.

Conclusione. L'entropia fornisce un limite inferiore al numero di pesate necessarie, e la strategia greedy che massimizza la riduzione di entropia a ogni passo produce un albero decisionale ottimale entro quel limite.

Generalizzazione: n palline e k pesate

Supponiamo di avere n palline, tra le quali una e solo una ha peso diverso dalle altre. Per ciascuna pallina esistono due possibili anomalie (più pesante o più leggera), per un totale di $2n$ situazioni distinte.

Il vincolo informativo. Con k pesate su una bilancia a due piatti, il numero massimo di sequenze di esiti distinguibili è 3^k. La condizione necessaria affinché k pesate bastino è:

$$2n \leq 3^k, \qquad \text{ovvero} \qquad n \leq \frac{3^k}{2}. \tag{2.20}$$

Questo è un vincolo informativo fondamentale: se il numero di palline supera questa soglia, nessuna strategia basata su k pesate può risolvere il problema. Non si tratta di un limite tecnico, ma di un limite assoluto imposto dalla quantità di informazione che k operazioni a tre esiti sono in grado di estrarre.

Condizione necessaria vs. sufficiente. La condizione $2n \leq 3^k$ è soltanto **necessaria**, non sufficiente. Disporre di abbastanza "rami" non implica che si riesca sempre a costruire un albero decisionale con la struttura che il problema richiede. Si può dimostrare che il numero minimo di pesate garantito essere sufficiente è:

$$k^* = \lceil \log_3(2n+3) \rceil. \tag{2.21}$$

Inoltre, la strategia greedy che ad ogni pesata sceglie la partizione più informativa è sufficiente per trovare l'albero di decisione di altezza k^*: non è necessario ricorrere a metodi più elaborati.

Per $n = 12$:

$$\log_3(24) \approx 2.892 \qquad \text{e} \qquad \log_3(27) = 3, \tag{2.22}$$

quindi $k = k^* = 3$: il limite informativo e il numero garantito coincidono. La situazione cambia per $n = 13$:

$$\log_3(26) \approx 2.965 \qquad \text{e} \qquad \log_3(29) \approx 3.065, \tag{2.23}$$

quindi $k = 3$ ma $k^* = 4$. Nonostante la condizione informativa $2n \leq 3^k$ sia ancora soddisfatta con $k = 3$ (poiché $26 < 27$), quattro pesate sono comunque necessarie. La ragione non è informativa, ma **strutturale**: anche se il numero di situazioni possibili non supera il numero di sequenze ternarie disponibili, non è possibile costruire un albero di decisione di altezza 3 che le distingua tutte.

Conclusione. Il limite informativo $\lceil \log_3 2n \rceil$ dice quante pesate sono necessarie; il limite strutturale $\lceil \log_3(2n+3) \rceil$ dice quante sono sufficienti. La strategia greedy raggiunge sempre il secondo limite.

Indovina un numero

Per collegare in modo ancora più intuitivo i concetti di entropia e informazione a una strategia di ricerca concreta, consideriamo il seguente problema. Vogliamo indovinare un numero intero x compreso tra 1 e 10^6, potendo porre soltanto domande del tipo *"x è minore di k?"*. Ogni domanda ammette due soli esiti (sì/no) e può quindi fornire al massimo 1 bit di informazione.

Il limite inferiore. Per identificare un valore tra 10^6 possibilità equiprobabili è necessario un numero di bit pari a:

$$\log_2 10^6 \approx 19.931 \text{ bit}. \tag{2.24}$$

Nessuna strategia può risolvere il problema in meno di $\lceil 19.931 \rceil = 20$ domande. Questo è un limite che deriva esclusivamente dalla quantità di informazione, non dalla nostra abilità nel formulare le domande.

La strategia ottimale. La strategia che massimizza l'informazione fornita da ciascuna domanda divide a ogni passo l'insieme dei casi ancora possibili in due parti di dimensione il più possibile uguale, scegliendo k come elemento mediano dell'intervallo corrente. La domanda "x è minore di k?" dimezza sistematicamente lo spazio delle possibilità, fornendo esattamente 1 bit a ogni passo. Il procedimento che ne deriva è il noto algoritmo di "ricerca binaria", che risolve il problema in $\lceil \log_2 10^6 \rceil = 20$ domande, raggiungendo il limite inferiore. Una domanda mal posta (ad esempio "x è minore di 2?") potrebbe dividere i casi in modo molto sbilanciato (1 contro 999999), fornendo quasi zero informazione nel caso più probabile.

Applicazione all'ordinamento. Lo stesso ragionamento si applica al problema di ordinare un insieme di n valori distinti effettuando confronti binari. Ogni confronto fornisce al più 1 bit, e i possibili ordinamenti sono $n!$, quindi il numero minimo di confronti è $\lceil \log_2 n! \rceil$. Si può dimostrare che:

$$\log_2 n! \geq n \log_2 n - 1.443(n-1), \tag{2.25}$$

e quindi che almeno $\lceil n \log_2 n - 1.443(n-1) \rceil$ confronti sono necessari. L'algoritmo mergesort effettua un numero di confronti pari a $n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$, che si avvicina molto al limite informativo:

numero elementi	10	50	100	500	1000	5000	10000	50000	100000
confronti necessari	21	212	522	3763	8525	54227	118452	708349	1516696
confronti sufficienti	25	237	573	3989	8977	56809	123617	734465	1568929
gap %	16	10.5	8.9	5.7	5	4.5	4.2	3.6	3.3

Conclusione. L'entropia fornisce un limite inferiore universale al numero di operazioni necessarie in qualsiasi problema di ricerca o decisione. La ricerca binaria e il mergesort sono esempi di algoritmi che si avvicinano a questo limite, massimizzando l'informazione acquisita a ogni passo.

2.5 Entropia congiunta, condizionata e informazione mutua

L'entropia misurata finora descrive l'incertezza di una sorgente considerata in isolamento. In molte situazioni reali, però, osserviamo grandezze tra loro correlate, oppure disponiamo di informazione aggiuntiva che riduce la nostra incertezza. Per descrivere queste situazioni occorre estendere l'apparato introdotto finora attraverso tre concetti strettamente connessi, che presentiamo come una triade:

1. **Entropia congiunta** $H(X, Y)$: l'incertezza sulla coppia (X, Y) considerata insieme.
2. **Entropia condizionale** $H(X|Y)$: l'incertezza residua su X dopo aver osservato Y.
3. **Informazione mutua** $I(X; Y)$: la riduzione di incertezza su X prodotta dall'osservazione di Y, ovvero la quantità di informazione condivisa tra le due variabili.

Entropia congiunta

L'incertezza relativa alla prossima coppia osservata (X, Y) è misurata dall'entropia congiunta:

$$H(X, Y) = -\sum_x \sum_y p(x, y) \log_2 p(x, y). \tag{2.26}$$

Se X e Y sono indipendenti vale l'additività $H(X,Y) = H(X) + H(Y)$: la conoscenza della coppia richiede esattamente la somma delle informazioni necessarie per ciascuna variabile separata. In generale, $H(X,Y) \geq H(X)$: la coppia non può essere meno incerta di una sola delle sue componenti.

Entropia condizionale

L'entropia condizionale di X dato Y è la media, su tutti i possibili valori di Y, dell'incertezza su X una volta che Y è noto:

$$H(X|Y) = -\sum_x \sum_y p(x,y) \log_2 p(x|y). \tag{2.27}$$

Se X e Y sono indipendenti, allora $p(x|y) = p(x)$ e quindi $H(X|Y) = H(X)$: conoscere Y non cambia la nostra incertezza su X, come ci aspettiamo. La relazione fondamentale tra le quantità introdotte è:

$$H(X,Y) = H(X|Y) + H(Y). \tag{2.28}$$

L'incertezza sulla coppia è pari all'incertezza su Y più l'incertezza residua su X dopo aver conosciuto Y: per descrivere la coppia, prima risolviamo l'incertezza su Y, poi quella su X dato Y.

Informazione mutua

L'incertezza su X prima di osservare Y è $H(X)$; dopo aver osservato Y diventa $H(X|Y)$. La differenza

$$I(X;Y) = H(X) - H(X|Y) \tag{2.29}$$

misura quanta incertezza su X ci aspettiamo di eliminare conoscendo Y, ossia quanta informazione Y porta su X. L'informazione mutua è simmetrica ($I(X;Y) = I(Y;X)$): la quantità di informazione condivisa è una proprietà della "relazione" tra le due variabili, indipendentemente da quale si consideri nota. È sempre non negativa, $I(X;Y) \geq 0$, con uguaglianza solo nel caso di indipendenza: conoscere Y non può mai aumentare l'incertezza su X.

La Figura 2.4 illustra graficamente le relazioni tra le varie quantità. I cerchi rappresentano $H(X)$ e $H(Y)$; la loro intersezione è $I(X;Y)$; le parti non sovrapposte sono le entropie condizionate $H(X|Y)$ e $H(Y|X)$.

Le Figure 2.5 e 2.6 illustrano i casi estremi. Quando X e Y sono molto dipendenti, i cerchi si sovrappongono quasi completamente e l'informazione mutua è grande. Quando sono quasi indipendenti, i cerchi si toccano appena e $I(X;Y)$ è quasi nulla.

Applicazione alla classe di studenti. Indichiamo con X l'identità dello studente che entrerà per primo e con Y il suo sesso. Abbiamo già calcolato $H(X) = 5$ bit, $H(X|Y = M) = \log_2 30 \approx 4.906$ bit e $H(X|Y = F) = 1$ bit. L'entropia condizionale media è:

$$H(X|Y) = \frac{30}{32} \log_2 30 + \frac{2}{32} \cdot 1 \approx 4.660 \text{ bit}. \tag{2.30}$$

L'informazione mutua tra identità e sesso è quindi:

$$I(X;Y) = H(X) - H(X|Y) \approx 5 - 4.660 = 0.340 \text{ bit}. \tag{2.31}$$

Sapere il sesso dello studente fornisce circa 0.34 bit di informazione sulla sua identità: un valore modesto rispetto ai 5 bit totali di incertezza, ma non trascurabile. Questo riflette il fatto che il sesso divide la classe

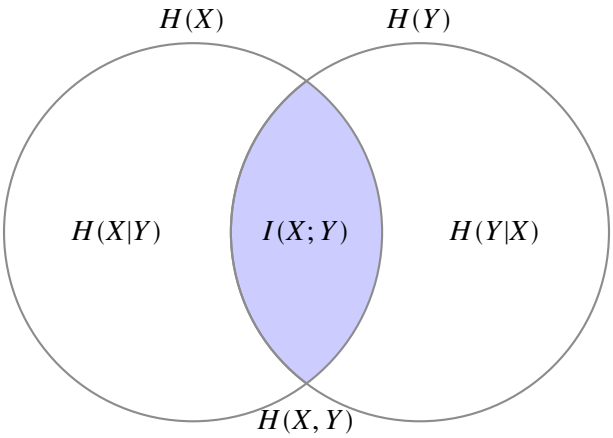

Figura 2.4: Relazione tra entropia, entropia condizionale e informazione mutua. I cerchi rappresentano $H(X)$ e $H(Y)$; la loro intersezione è $I(X;Y)$.

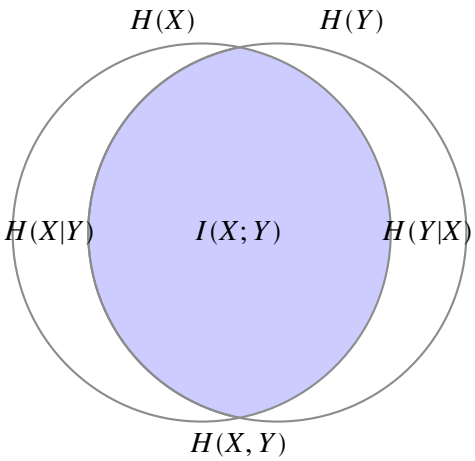

Figura 2.5: X e Y molto dipendenti: i cerchi si sovrappongono in gran parte, e l'informazione mutua è dominante.

in modo molto sbilanciato (30 maschi contro 2 femmine): la conoscenza del sesso riduce la lista dei candidati, ma nella maggior parte dei casi (quando l'entrato è maschio) la riduzione è piccola.
Conclusione. Entropia congiunta, condizionale e informazione mutua formano un sistema coerente per quantificare l'incertezza in presenza di dipendenze statistiche. L'informazione mutua, in particolare, misura quanto due variabili "si parlano": è zero per variabili indipendenti e massima quando una determina completamente l'altra.

Ancora sulle 12 palline

Le definizioni introdotte possono essere rilette in modo molto concreto attraverso il problema delle 12 palline. Indichiamo con X lo stato reale del sistema (quale pallina è anomala e se è più pesante o più leggera) e con Y l'esito della prima pesata. Supponendo i 24 valori di X equiprobabili, l'obiettivo nella progettazione della prima pesata è massimizzare l'informazione mutua $I(X;Y)$.

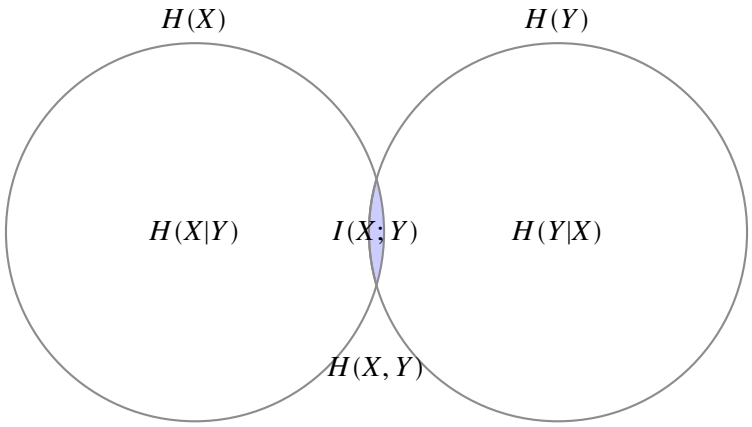

Figura 2.6: X e Y quasi indipendenti: i cerchi si sovrappongono appena e l'informazione mutua è trascurabile.

Poiché $I(X;Y) = H(X) - H(X|Y)$ e $H(X)$ è fissata a $\log_2 24 \approx 4.585$ bit, massimizzare $I(X;Y)$ equivale a minimizzare l'entropia condizionale $H(X|Y)$: vogliamo che, dopo aver osservato l'esito della pesata, rimanga il meno possibile di incertezza su X. Supponendo di porre m palline su ciascun piatto, i valori numerici dell'informazione mutua per $m = 1, \dots, 6$ sono:

m	1	2	3	4	5	6
$H(X\|Y)$	3.769	3.333	3.085	3	3.102	3.585
$I(X;Y)$	0.817	1.251	1.5	1.585	1.483	1

La scelta $m = 4$ massimizza l'informazione mutua: porre 4 palline per piatto nella prima pesata è la scelta più efficiente dal punto di vista informativo, perché distribuisce i 24 casi nel modo più uniforme possibile tra i tre esiti della bilancia. Questa non è un'intuizione vaga, ma una conseguenza diretta del principio che massimizzare l'entropia degli esiti equivale a massimizzare l'informazione mutua tra lo stato reale e il risultato della pesata.

2.6 Altri esempi: ricerca, giochi e linguaggi

I concetti sviluppati nelle sezioni precedenti trovano applicazione in contesti molto diversi tra loro. Gli esempi che seguono mostrano come l'entropia e l'informazione mutua permettano di analizzare con precisione la struttura dell'incertezza e il modo in cui essa si riduce nel corso di un processo osservativo.

Battaglia navale semplificata

Consideriamo una versione semplificata del gioco della battaglia navale. Ciascun giocatore dispone di un solo sottomarino che occupa una singola posizione di una griglia 8×8 (64 posizioni totali). Ogni tentativo può produrre solo due esiti: mancato (m) oppure distrutto (d, e la partita termina).

L'incertezza iniziale e i limiti di ogni colpo. All'inizio il sottomarino può trovarsi in una qualunque delle 64 posizioni con uguale probabilità:

$$H = \log_2 64 = 6 \text{ bit}. \tag{2.32}$$

Questa è la quantità totale di informazione da acquisire per individuare la posizione. Al k-esimo tentativo (con $k-1$ posizioni già escluse):

$$p(X=d) = \frac{1}{65-k}, \qquad p(X=m) = \frac{64-k}{65-k}. \tag{2.33}$$

Se il primo colpo coglie il sottomarino si acquisiscono $\log_2 64 = 6$ bit in un solo colpo. Se va a vuoto si acquisiscono soltanto $\log_2 \frac{64}{63} \approx 0.023$ bit: sapere che il sottomarino non è in "una" casella su 64 è quasi irrilevante.

L'informazione si conserva. Vale l'elegante identità:

$$I_1(m) + I_2(d) = \log_2 \frac{64}{63} + \log_2 63 = \log_2 64 = 6 \text{ bit}. \tag{2.34}$$

Indipendentemente dall'ordine con cui si verificano un "mancato" e un "distrutto", l'informazione totale acquisita è sempre pari all'entropia iniziale. Ogni sequenza di esiti che porta alla vittoria deve raccogliere complessivamente tutti i 6 bit di informazione sulla posizione del sottomarino.

L'informazione cumulativa acquisita dopo k tentativi tutti a vuoto è:

$$\log_2 \frac{64}{63} + \log_2 \frac{63}{62} + \ldots + \log_2 \frac{65-k}{64-k} = \log_2 \frac{64}{64-k}. \tag{2.35}$$

Ad esempio, dopo 32 tentativi a vuoto si ha $\log_2 \frac{64}{32} = 1$ bit: abbiamo dimezzato il numero di possibilità e acquisito esattamente 1 bit di informazione.

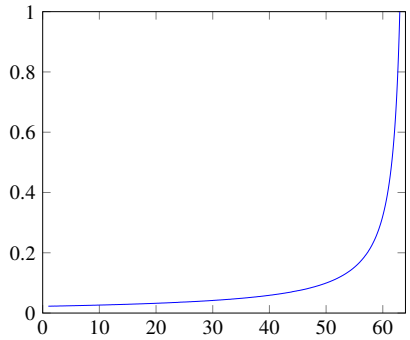

Figura 2.7: Informazione acquisita per ogni tentativo a vuoto, al crescere del numero di tentativi effettuati. Escludere le ultime posizioni è progressivamente più informativo.

Conclusione. Questo esempio illustra un principio generale: l'informazione si conserva durante un processo di ricerca. La somma dell'informazione acquisita lungo qualsiasi sequenza di esiti che porta alla vittoria è sempre pari all'entropia iniziale. I colpi a vuoto contribuiscono poco per volta; il colpo finale concentra tutta l'incertezza residua. La partita è, dal punto di vista informativo, un processo di erosione progressiva dell'entropia che si conclude solo quando tutta l'incertezza è stata eliminata.

Linguaggi, informazione ed entropia

Un ambito naturale in cui applicare i concetti introdotti è quello dei linguaggi. Se consideriamo una sorgente informativa come origine di testi in un determinato linguaggio, possiamo interpretare la produzione di un testo come una sequenza di simboli generati da una sorgente probabilistica. Le nozioni di informazione di Shannon, entropia, entropia condizionale e informazione mutua permettono di quantificare in modo preciso quanto un testo sia prevedibile, ridondante o informativamente ricco.

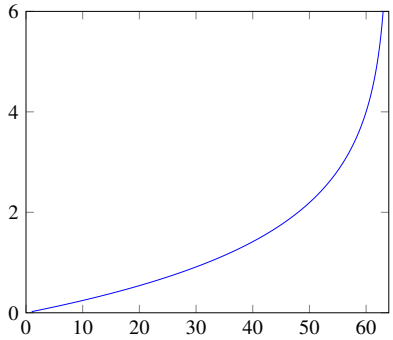

Figura 2.8: Informazione cumulativa acquisita per tentativi a vuoto. La crescita è lenta all'inizio e accelera man mano che le posizioni rimanenti si riducono.

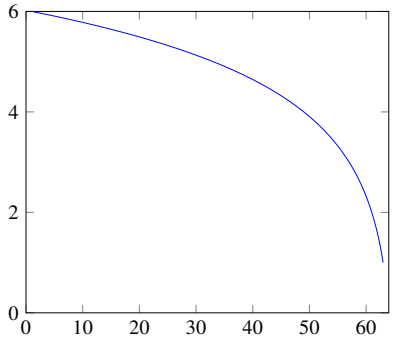

Figura 2.9: Informazione acquisita in caso di successo al k-esimo tentativo: tende a zero man mano che le posizioni rimaste da esplorare diminuiscono.

Per definire formalmente una sorgente informativa dobbiamo specificare il suo alfabeto e la distribuzione di probabilità associata ai diversi simboli. Nel caso dell'italiano, l'alfabeto è l'insieme delle 26 lettere, e la distribuzione si stima empiricamente dalle frequenze relative dei caratteri in un testo sufficientemente lungo. A titolo di esempio, le frequenze stimate dalla traduzione italiana de "Lo Hobbit" di J. R. R. Tolkien sono:

e	0.11522	a	0.11258	o	0.10273	i	0.09923
n	0.07145	r	0.06876	t	0.06025	l	0.06001
s	0.05606	c	0.04273	d	0.03464	u	0.03400
p	0.02771	m	0.02414	v	0.02210	g	0.01926
b	0.01439	h	0.01204	f	0.01189	q	0.00528
z	0.00518	y	0.00018	k	0.00010	w	0.00005
x	0.00000						

Una volta note le probabilità, è immediato associare a ciascun carattere la sua informazione di Shannon:

e	3.116	a	3.151	o	3.283	i	3.333
n	3.807	r	3.862	t	4.053	l	4.059
s	4.157	c	4.549	d	4.851	u	4.878
p	5.173	m	5.372	v	5.500	g	5.699
b	6.119	h	6.376	f	6.394	q	7.564
z	7.593	y	12.424	k	13.355	w	14.258
x	17.781						

Come previsto, i caratteri più frequenti hanno un contenuto informativo minore: osservare una `e` o una `a` è poco sorprendente, mentre incontrare una `x` o una `w` è un evento raro e quindi altamente informativo. L'entropia del linguaggio italiano, nel modello in cui i caratteri sono considerati indipendenti, è:

$$H(X) = 3.997 \text{ bit}. \tag{2.36}$$

La ridondanza del linguaggio. Se confrontiamo questo valore con il numero di bit necessari per rappresentare 26 simboli equiprobabili, $\log_2 26 \approx 4.700$ bit, osserviamo che l'entropia è significativamente inferiore. Questo scarto misura il fatto che l'italiano non usa i simboli in modo equiprobabile, ma presenta una struttura statistica marcata. La ridondanza quantifica questa differenza:

$$1 - \frac{H(X)}{\log_2 N} = 1 - \frac{3.997}{4.700} \approx 15\%. \tag{2.37}$$

La ridondanza non è un difetto del linguaggio: è esattamente quella che rende possibile comprendere un testo anche in presenza di errori o parti mancanti, e che consente agli algoritmi di compressione di funzionare efficacemente. È anche il motivo per cui possiamo leggere senza difficoltà frasi come `abbassa il vo..me di q..o st.reo`.

Un linguaggio artificiale. Per chiarire ulteriormente come l'informazione si distribuisca tra i caratteri, consideriamo un linguaggio artificiale derivato dall'italiano. Costruiamo un dizionario di $2^{18} = 262144$ parole, tutte di lunghezza 5 caratteri, generate casualmente secondo le frequenze dei caratteri italiani. Ogni parola ha un'informazione di Shannon pari a $\log_2 2^{18} = 18$ bit, ma questa informazione è distribuita tra i 5 caratteri in modo non uniforme. Se il primo carattere è una `p` (rara, $I \approx 5.17$ bit), dopo averlo letto sappiamo molto di più su quale parola stiamo leggendo rispetto al caso in cui il primo carattere sia una `i` (comune, $I \approx 3.33$ bit). Caratteri rari riducono il campo delle parole possibili molto più dei caratteri comuni: l'informazione di ogni carattere contribuisce in modo diverso a ridurre l'incertezza sull'intera parola.

2.7 Entropia del linguaggio naturale

Una volta introdotti i concetti di entropia e ridondanza, è naturale applicarli al linguaggio naturale per quantificare quanta struttura statistica esso contenga. Il punto cruciale è che la struttura del linguaggio non si esaurisce nella distribuzione delle singole lettere: ci sono dipendenze fortissime tra le lettere di una stessa parola, tra le parole di una frase, tra le frasi di un paragrafo. Il modello che assume i caratteri indipendenti è una prima approssimazione, ma seriamente insufficiente.

Questa ridondanza ha un ruolo funzionale fondamentale. Se il linguaggio fosse una sequenza di simboli completamente imprevedibili ed equiprobabili, basterebbe perdere una piccola frazione del messaggio per rendere impossibile la ricostruzione. Invece, nella comunicazione quotidiana comprendiamo regolarmente discorsi in ambienti rumorosi, perché i vincoli grammaticali, fonotattici, lessicali e semantici restringono fortemente l'insieme delle continuazioni plausibili di una frase.

L'esperimento di predizione di Shannon

Shannon descrisse un esperimento in cui un testo viene sottoposto a un soggetto umano che deve, iterativamente, predire il prossimo carattere. Si registra quante volte la predizione è corretta. Se spesso il soggetto indovina, significa che l'insieme dei simboli plausibili in quel contesto è piccolo, e quindi l'entropia condizionata del prossimo carattere è bassa. Un testo T viene associato a un testo ridotto T' in cui la posizione i contiene "-" se il carattere è stato predetto correttamente, o il carattere originale altrimenti. Ad esempio:

THE ROOM WAS NOT VERY LIGHT A SMALL OBLONG READING LAMP ON THE DESK
SHED GLOW ON POLISHED WOOD BUT LESS ON THE SHABBY RED CARPET

```
-- ROO—NOT-V--I—SM--OBL--REA——-O—D--SHED -GLO-O-P-L-S--O—BU-L-S-O—SH--RE-C--
```

Su 129 caratteri, 89 (circa il 69%) sono stati predetti correttamente. Gli errori compaiono più spesso all'inizio delle parole o delle sillabe, dove l'insieme delle continuazioni ammissibili è più ampio. La predizione corretta è una forma di compressione implicita: quando un carattere è altamente prevedibile, non è necessario trasmetterlo esplicitamente.

Entropia condizionata in funzione del contesto

Definendo l'entropia condizionata del prossimo carattere dato un contesto di lunghezza N come:

$$H(c_{N+1} \mid c_1 c_2 \dots c_N) = -\sum_{c_1,\dots,c_{N+1}} p(c_1,\dots,c_{N+1}) \log_2 p(c_{N+1} \mid c_1,\dots,c_N), \tag{2.38}$$

al crescere di N questa quantità diminuisce in modo monotono: aggiungere più contesto non può aumentare l'incertezza sul simbolo successivo. La sequenza

$$H(c) \geq H(c_2|c_1) \geq H(c_3|c_1c_2) \geq \dots \geq H(c_{N+1}|c_1c_2\dots c_N) \tag{2.39}$$

è decrescente e ammette un limite, che Shannon definisce come "entropia del linguaggio":

$$H = \lim_{N\to\infty} H(c_{N+1}|c_1c_2\dots c_N). \tag{2.40}$$

L'entropia del linguaggio è il valore a cui si avvicina l'entropia condizionata al crescere del contesto, e cattura l'idea che l'entropia "vera" del linguaggio tenga conto di tutte le dipendenze tra simboli, non solo della frequenza dei singoli caratteri.

Nel caso de "Lo Hobbit" italiano, la Figura 2.10 riporta l'entropia di un carattere condizionata al carattere precedente. La lettera □ (spazio) produce l'entropia condizionata più alta, circa 4.08 bit: dopo lo spazio si inizia una nuova parola e l'insieme delle possibili continuazioni è massimo. Al contrario, dopo una `Q` l'entropia è quasi nulla (circa 0.23 bit): in italiano la `Q` è quasi sempre seguita da `U`. La media pesata di questi valori fornisce l'entropia condizionata media:

$$H(c_2|c_1) = 3.378 \text{ bit}. \tag{2.41}$$

Rispetto all'entropia marginale $H(c) = 3.997$ bit, la differenza è:

$$I(c_2; c_1) = H(c_2) - H(c_2|c_1) = 0.620 \text{ bit}. \tag{2.42}$$

In media, conoscere il carattere precedente elimina circa 0.62 bit di incertezza sul carattere successivo: un segnale significativo di struttura statistica nel linguaggio.

$H(c_2\|c_1 = A)$	2.81874
$H(c_2\|c_1 = B)$	2.4633
$H(c_2\|c_1 = C)$	2.53605
$H(c_2\|c_1 = D)$	2.19469
$H(c_2\|c_1 = E)$	2.49292
$H(c_2\|c_1 = F)$	2.53523
$H(c_2\|c_1 = G)$	2.79755
$H(c_2\|c_1 = H)$	1.53494
$H(c_2\|c_1 = I)$	2.90252
$H(c_2\|c_1 = J)$	0
$H(c_2\|c_1 = K)$	0.75045
$H(c_2\|c_1 = L)$	2.92958
$H(c_2\|c_1 = M)$	2.41964
$H(c_2\|c_1 = N)$	2.95497
$H(c_2\|c_1 = O)$	2.40532
$H(c_2\|c_1 = P)$	2.4004
$H(c_2\|c_1 = Q)$	0.23449
$H(c_2\|c_1 = R)$	2.96679
$H(c_2\|c_1 = S)$	2.8953
$H(c_2\|c_1 = T)$	2.55758
$H(c_2\|c_1 = U)$	3.17885
$H(c_2\|c_1 = V)$	2.03129
$H(c_2\|c_1 = W)$	0.58133
$H(c_2\|c_1 = X)$	0.23011
$H(c_2\|c_1 = Y)$	0.50311
$H(c_2\|c_1 = Z)$	1.98285
$H(c_2\|c_1 = \square)$	4.08388

Figura 2.10: Entropia di un carattere condizionata dal precedente, stimata per l'italiano sulla base de "Lo Hobbit".

Testo casuale e la "scimmia" di Shannon

In un articolo che pose le fondamenta della teoria dell'informazione, Shannon propose un esperimento ideale: immaginare una scimmia che schiaccia tasti a caso su una tastiera, e chiedersi quanto il testo prodotto assomigli a vero inglese man mano che aumentiamo la complessità del modello statistico utilizzato per generarlo. L'esperimento non valuta comprensione o intenzionalità, ma la somiglianza statistica: un testo generato con un modello che conosce solo le frequenze delle singole lettere sembrerà molto diverso dall'inglese rispetto a uno generato con un modello che conosce le frequenze dei bigrammi, dei trigrammi, e così via. Shannon riportò esempi di testi generati casualmente sotto ipotesi progressive di dipendenza, che mostrano in modo sorprendentemente chiaro l'effetto della riduzione dell'entropia condizionata:

Simboli indipendenti, generati secondo $p(c)$:

XFOML RXKHRJFFJUJ ALPWXFWJXYJ FFJEYVJCQSGHYD QPAAMKBZAACIBZLKJQD

Simboli generati secondo $p(c_2|c_1)$:

OCRO HLO RGWR NMIELWIS EU LL NBNESEBYA TH EEI ALHENHTTPA OOBTTVA NAH BRL

Simboli generati secondo $p(c_3|c_1c_2)$:

ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TUCOOWE AT TEASONARE

Simboli generati secondo $p(c_4|c_1c_2c_3)$:

IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID PONDENOME OF DEMONSTURES OF THE REPTAGIN

Parole indipendenti, generate secondo $p(w)$:

REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME

Parole generate secondo $p(w_2|w_1)$:

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE

Già il passaggio da simboli completamente casuali a simboli con le frequenze dell'inglese produce frammenti che ricordano sillabe plausibili. Con contesti più lunghi emergono strutture via via più simili a parole e segmenti sintattici. In termini informativi, la distribuzione condizionata del prossimo simbolo si concentra su un insieme via via più ristretto e l'entropia condizionata diminuisce, rendendo il testo più prevedibile e più simile al linguaggio naturale.

Conclusione. La ridondanza del linguaggio naturale è misurabile e quantitativamente significativa. Il linguaggio italiano usa circa il 15% di "capacità" informativa in meno rispetto a un codice ottimale con lo stesso alfabeto, e le dipendenze contestuali riducono ulteriormente l'incertezza sul prossimo carattere. È questa struttura statistica che rende il linguaggio robusto agli errori, comprimibile, e che pone un limite preciso a quanta informazione può contenere un testo di una data lunghezza.

Capitolo 3

Compressione e correzione di errori

3.1 Codici su simboli, canali e teoremi fondamentali di Shannon

Una delle idee guida della teoria della codifica è che l'informazione non è "tutta uguale": se alcuni simboli compaiono spesso e altri raramente, allora ha senso sfruttare questa asimmetria per rappresentare il messaggio nel modo più compatto possibile. In termini molto concreti, se un simbolo è frequente possiamo permetterci di usare una parola di codice corta, mentre per un simbolo raro possiamo usare una parola più lunga. Questa scelta non è soltanto un trucco ingegneristico: riflette un principio generale di economia delle risorse — bit, memoria, banda — ed è alla base sia della compressione dei dati sia, più in generale, di come i sistemi digitali trattano l'informazione.

Per rendere rigorosa questa intuizione è utile distinguere fin dall'inizio due problemi, che a prima vista sembrano diversi ma che Shannon ha mostrato essere governati da quantità comuni. Il primo problema è quello della "compressione di sorgente": una sorgente produce simboli e vogliamo descriverli con il minor numero medio di bit, senza errori, realizzando cioè una compressione senza perdita. Il secondo problema riguarda invece la "trasmissione su canale": i bit codificati devono attraversare un mezzo fisico (cavo, aria, memoria) che può introdurre rumore, e quindi l'obiettivo non è soltanto comprimere, ma anche proteggere l'informazione in modo da renderla recuperabile all'uscita. Come vedremo, i due problemi sono per certi versi opposti: comprimere significa eliminare ridondanza, mentre proteggere significa introdurla.

I teoremi fondamentali di Shannon sono sorprendenti per due ragioni complementari. Da un lato stabiliscono dei limiti "impossibili da superare": sotto una certa soglia non si può comprimere senza perdere informazione, e sopra una certa soglia non si può comunicare in modo affidabile su un canale rumoroso. Dall'altro lato, questi limiti non sono soltanto barriere astratte: Shannon mostra che sono in un certo senso "raggiungibili", almeno asintoticamente, con codifiche costruite su blocchi di simboli sufficientemente lunghi. Questa è la caratteristica più affascinante della sua teoria: non si limita a dire "fin qui non si può andare", ma afferma anche "fin qui si può arrivare".

Sorgente, codifica e decodificabilità

Consideriamo una sorgente di informazione X, caratterizzata da un alfabeto finito di simboli

$$S = \{s_1, s_2, \dots, s_m\}, \tag{3.1}$$

e da una distribuzione di probabilità $p_i = p(s_i)$ con $i = 1, \dots, m$. La sorgente produce sequenze di simboli in S, e p_i rappresenta la frequenza media con cui il simbolo s_i compare. Per descrivere questi

simboli in modo digitale scegliamo tipicamente l'alfabeto binario $\{0, 1\}$, naturale nei sistemi elettronici e sufficientemente generale da rappresentare qualunque informazione.

Una codifica (di sorgente) associa a ciascun simbolo s_i una parola binaria $c(s_i)$. Indichiamo con l_i la lunghezza della parola assegnata, cioè il numero di bit di $c(s_i)$. L'idea "codice corto per simbolo frequente" diventa allora una scelta sulle lunghezze l_i: a probabilità grandi p_i dovrebbero corrispondere lunghezze piccole l_i, e viceversa. In pratica, si tratta di allocare lo spazio di codice in modo proporzionale alla frequenza con cui i simboli compaiono: i simboli che "vediamo di più" meritano una descrizione più breve, proprio come in certi linguaggi naturali le parole più usate tendono ad essere le più corte.

Questa idea, però, ha senso solo se la trasformazione in bit è realmente "usabile": data una stringa di bit ottenuta concatenando parole di codice, dobbiamo poter ricostruire senza ambiguità la sequenza originale dei simboli. Questa proprietà è detta "decodificabilità univoca", e dal punto di vista concettuale è ciò che separa una codifica informativa da una semplice "etichettatura": se la decodifica non è univoca, il destinatario non può sapere quale messaggio fosse stato trasmesso, e quindi non stiamo davvero conservando informazione.

Un esempio mostra subito il rischio. Se $S = \{a, b, c, d\}$ e scegliamo $a = 0$, $b = 01$, $c = 11$, $d = 00$, allora la stringa 0011 può essere interpretata come dc (cioè 00 11) oppure come aac (cioè 0 0 11). L'ambiguità non è un dettaglio tecnico: significa che due sequenze diverse di simboli collassano nella stessa sequenza binaria, e quindi l'informazione non è preservata. In un sistema di comunicazione reale, questo equivarrebbe a inviare messaggi che il destinatario non può sempre leggere correttamente, il che rende il codice inutilizzabile.

Per evitare ambiguità, una classe cruciale è quella dei "codici prefisso", nei quali nessuna parola assegnata a un simbolo è prefisso della parola assegnata a un simbolo diverso. Un "prefisso" di una stringa è semplicemente il suo inizio: per esempio, 01 è un prefisso di 011 e di 01100. Il motivo dell'importanza dei codici prefisso è operativo: con un tale codice la decodifica avviene "al volo", da sinistra a destra, perché non si rischia mai che una parola già riconosciuta sia l'inizio di una parola più lunga. In altre parole, la proprietà di prefisso trasforma la decodifica in un processo deterministico e sequenziale, senza necessità di tornare indietro a rivedere bit già letti.

Informazione di Shannon ed entropia come limite alla compressione

Il problema della compressione di sorgente consiste nel rendere piccola la lunghezza media del messaggio codificato. Per collegare questa nozione a una quantità intrinseca della sorgente, applichiamo a questo contesto le definizioni fondamentali di Shannon, per cui ognuno dei simboli $s_1, \dots, s_m$ ha associata la relativa informazione

$$I(s_i) = -\log_2 p_i. \tag{3.2}$$

e la sorgente X ha associata la relativa entropia

$$H(X) = -\sum_{i=1}^{n} p_i \log_2 p_i. \tag{3.3}$$

Da un punto di vista operativo, l'entropia è l'informazione media per simbolo prodotta dalla sorgente: è la quantità media di incertezza che dobbiamo eliminare per descrivere correttamente l'uscita della sorgente. Se la sorgente è molto prevedibile, $H(X)$ è piccola e ci aspettiamo di poter comprimere molto il messaggio; se è imprevedibile, $H(X)$ è grande e la compressione senza perdita diventa difficile. Si può dimostrare che l'entropia raggiunge il suo valore massimo quando tutti i simboli sono equiprobabili, e si annulla quando la sorgente è deterministica, cioè produce sempre lo stesso simbolo. Questo riflette perfettamente

l'intuizione: una sorgente che produce sempre la stessa lettera non contiene alcuna informazione, perché il messaggio è noto in anticipo.

Osserviamo che se un messaggio ha lunghezza n e indichiamo con n_i il numero di occorrenze di s_i, allora n_i/n è la frequenza empirica del simbolo s_i, che al crescere di n approssima sempre meglio p_i. In questo caso l'entropia può essere scritta come

$$H(X) = -\sum_{i=1}^{m} \frac{n_i}{n} \log_2 \frac{n_i}{n}, \tag{3.4}$$

che rende evidente il suo carattere di media empirica su messaggi lunghi.

Una volta fissata la codifica c, la "lunghezza media" del codice è

$$L(c, X) = \sum_{i=1}^{m} p_i l_i. \tag{3.5}$$

Questa è la misura pratica del costo: il numero medio di bit usati per rappresentare un simbolo. Se un messaggio contiene n simboli, ci aspettiamo una lunghezza complessiva dell'ordine di $n \cdot L(c, X)$. Tra tutti i codici ammissibili, il "codice ottimo" sarà quello che minimizza $L(c, X)$. La domanda centrale diventa allora: quanto piccolo può diventare $L(c, X)$ e qual è il limite inferiore non aggirabile?

Sulla codifica di sorgente

Il "teorema della codifica di una sorgente" formalizza l'idea che l'entropia sia la soglia di comprimibilità. Esso afferma, in sostanza, due cose: da un lato che nessun codice può fare meglio di $H(X)$, dall'altro che questo limite può essere avvicinato quanto si vuole.

Da un lato, per ogni codifica univocamente decodificabile vale la disuguaglianza

$$L(c, X) \geq H(X). \tag{3.6}$$

Il significato intuitivo di questo risultato è che non possiamo usare, in media, meno bit dell'informazione media che la sorgente produce. Se provassimo a comprimere stabilmente sotto $H(X)$, dovremmo inevitabilmente far "collidere" rappresentazioni diverse: cioè, prima o poi, due messaggi distinti finirebbero per condividere la stessa descrizione, rendendo impossibile la ricostruzione senza perdita. La disuguaglianza $L \geq H$ è quindi l'espressione quantitativa del fatto che la compressione senza perdita non può eliminare informazione, ma solo ridondanza statistica.

Dall'altro lato, il teorema afferma anche che questo limite è asintoticamente raggiungibile, nel senso che possiamo costruire codifiche in cui la lunghezza media del codice è arbitrariamente vicina (ma sempre superiore) all'entropia. Più precisamente, per ogni valore $\varepsilon > 0$ arbitrariamente piccolo esistono codifiche, tipicamente costruite codificando "blocchi" di simboli successivi, tali che

$$L(c, X) \leq H(X) + \varepsilon. \tag{3.7}$$

L'idea alla base è che le lunghezze delle parole di codice devono essere intere (non si possono assegnare "2.3 bit" a un simbolo), quindi su singoli simboli c'è un inevitabile arrotondamento. Codificando blocchi lunghi, questo arrotondamento si "spalma" su molti simboli, e l'overhead medio per simbolo può diventare arbitrariamente piccolo. In formula, se indichiamo con $L(c, X^k)$ la lunghezza media per simbolo della migliore codifica per blocchi di lunghezza k, allora

$$\lim_{k \to \infty} L(c, X^k) = H(X). \tag{3.8}$$

Questo chiarisce un punto concettuale importante: l'entropia non è soltanto un limite astratto, ma il valore verso cui tendono le migliori compressioni quando sfruttiamo pienamente la struttura statistica della sorgente. In altre parole, $H(X)$ è il "prezzo minimo" medio per simbolo per rappresentare senza perdita ciò che la sorgente produce.

Un contesto in cui questi concetti diventano immediatamente concreti è quello della "compressione di sorgenti". L'obiettivo non è decidere tra ipotesi diverse tramite misure o domande, ma rappresentare una sequenza di simboli nel modo più efficiente possibile, cioè con il minor numero medio di bit, senza perdere informazione sul contenuto originario. In una compressione senza perdita, ciò che conta è che la decodifica ricostruisca esattamente la sequenza iniziale: si pensi, ad esempio, alla compressione di un testo o di un programma, dove anche un solo carattere errato può rendere il file inutilizzabile.

Supponiamo che una sorgente produca simboli appartenenti a un alfabeto finito

$$\mathscr{S} = \{s_1, s_2, \dots, s_m\}, \tag{3.9}$$

emessi in modo indipendente ma con probabilità non necessariamente uguali. Indichiamo con X la variabile che descrive il simbolo emesso. Questa variabile ha una distribuzione $p(s_i) = p_i$ associata.

Se tutti i simboli fossero equiprobabili, sarebbe naturale usare una codifica a lunghezza fissa di $\lceil \log_2 n \rceil$ bit per simbolo: non ci sarebbe alcun motivo per trattare un simbolo diversamente dagli altri. Quando però le probabilità differiscono, questa strategia diventa inefficiente perché usa lo stesso numero di bit per eventi frequenti e rari. È qui che entra in gioco la codifica a lunghezza variabile: assegnare parole corte ai simboli che compaiono spesso e parole più lunghe a quelli che compaiono raramente. La definizione $I(s_i) = -\log_2 p_i$ suggerisce, in modo quasi inevitabile, questo principio: se un simbolo è poco sorprendente, "merita" una descrizione corta; se è molto sorprendente, "richiede" una descrizione più lunga. Si tratta, in fondo, dello stesso principio che governa i codici Morse, dove le lettere più frequenti come la "e" vengono rappresentate da sequenze brevissime.

Consideriamo una sorgente che emette quattro simboli $\{a, b, c, d\}$ con probabilità

$$p(a) = 0.5, \qquad p(b) = 0.25, \qquad p(c) = 0.125, \qquad p(d) = 0.125. \tag{3.10}$$

Si noti che queste probabilità sono scelte come potenze di $1/2$: questa scelta semplifica i calcoli, ma ha anche un significato più profondo che emergerà a breve. Calcoliamo l'entropia:

$$\begin{aligned} H(X) &= -\left(0.5\log_2 0.5 + 0.25\log_2 0.25 + 0.125\log_2 0.125 + 0.125\log_2 0.125\right) \\ &= -(0.5\cdot(-1) + 0.25\cdot(-2) + 0.125\cdot(-3) + 0.125\cdot(-3)) \\ &= 0.5 + 0.5 + 0.375 + 0.375 \\ &= 1.75 \text{ bit}. \end{aligned} \tag{3.11}$$

Questo valore ha un significato operativo molto chiaro: in media, ogni simbolo porta 1.75 bit di informazione, e dunque una compressione senza perdita non potrà, nel lungo periodo, scendere sotto quel costo medio.

Se usassimo una codifica a lunghezza fissa, dovremmo usare 2 bit per simbolo (perché $2^2 = 4$), con uno spreco medio di $2 - 1.75 = 0.25$ bit per simbolo rispetto al limite entropico. Questo spreco può sembrare piccolo, ma su un file da un milione di simboli corrisponde a 250 000 bit sprecati — quasi 30 kilobyte in più del necessario.

Un possibile codice binario a lunghezza variabile è

$$a \mapsto 0, \qquad b \mapsto 10, \qquad c \mapsto 110, \qquad d \mapsto 111. \tag{3.12}$$

Questo codice è prefisso, quindi decodificabile in modo immediato. Le lunghezze sono $\ell(a) = 1$, $\ell(b) = 2$, $\ell(c) = 3$, $\ell(d) = 3$ e la lunghezza media vale

$$\begin{aligned} L(c, X) &= 0.5 \cdot 1 + 0.25 \cdot 2 + 0.125 \cdot 3 + 0.125 \cdot 3 \\ &= 0.5 + 0.5 + 0.375 + 0.375 \\ &= 1.75 \text{ bit.} \end{aligned} \tag{3.13}$$

Qui $L(c, X) = H(X)$ esattamente. Non è un caso fortunato: le probabilità sono potenze inverse di 2, quindi le lunghezze ideali $-\log_2 p_i$ risultano intere, e un codice prefisso può realizzarle perfettamente.

Quando ciò non accade, in generale non si riesce a raggiungere esattamente $H(X)$ con un codice prefisso su singoli simboli. Tuttavia esistono procedure costruttive che producono codici ottimi nel senso della lunghezza media: i codici di Huffman, che vedremo nel dettaglio più avanti, garantiscono

$$H(X) \leq L < H(X) + 1. \tag{3.14}$$

In termini pratici, questo significa che anche nel caso peggiore l'inefficienza rispetto al limite teorico è inferiore a 1 bit per simbolo.

Codifica come trasformazione e lettura in termini di informazione mutua

È utile osservare che una codifica senza perdita può essere vista come una trasformazione: al simbolo casuale X associamo una rappresentazione binaria Y (la parola di codice). Se la codifica è priva di perdita, allora X è determinabile univocamente da Y, e quindi la conoscenza di Y azzera completamente l'incertezza su X:

$$H(X \mid Y) = 0. \tag{3.15}$$

Ne consegue che l'informazione mutua tra simbolo e rappresentazione è

$$I(X; Y) = H(X) - H(X \mid Y) = H(X). \tag{3.16}$$

Questa identità mette in luce un fatto concettuale importante: comprimere senza perdita significa cambiare rappresentazione senza distruggere informazione, cercando però di ridurre il "costo medio" della rappresentazione stessa. In altre parole, l'operazione di compressione è una trasformazione che mira a eliminare ridondanza, non contenuto informativo. È come riformulare una frase eliminando le ripetizioni e le parole superflue: il significato rimane, ma le parole usate sono meno.

Canali e rumore: quando la trasmissione distrugge informazione

Fin qui abbiamo discusso la codifica come se il messaggio codificato arrivasse al destinatario senza alterazioni. In pratica, però, la comunicazione avviene attraverso un mezzo fisico che può introdurre rumore. Per modellare questa situazione introduciamo il concetto di canale.

Un canale è un meccanismo probabilistico che, dato un simbolo in ingresso X, produce un simbolo in uscita Y secondo una distribuzione condizionata $p(y \mid x)$. In un canale "rumoroso", Y non coincide necessariamente con X: il canale può commettere errori, e quindi l'uscita conserva solo una parte dell'informazione contenuta nell'ingresso. Questa "parte" è misurata dall'informazione mutua $I(X; Y)$, che rappresenta quanta informazione su X è effettivamente accessibile osservando Y. L'entropia condizionata $H(X \mid Y)$, invece, quantifica quanta incertezza rimane su X anche dopo aver osservato Y: se il canale è perfetto, $H(X \mid Y) = 0$; se il rumore è così forte da rendere l'uscita completamente indipendente dall'ingresso, $H(X \mid Y) = H(X)$.

Esempio: canale binario rumoroso (BSC)

Un contesto particolarmente semplice, ma estremamente istruttivo, per rendere concreto il significato di entropia e informazione mutua è quello di un canale binario rumoroso, noto in letteratura come "Binary Symmetric Channel" (BSC). Il modello è volutamente minimale — forse il più semplice modello di canale che abbia senso considerare — ma è proprio questa semplicità a renderlo trasparente: con poche quantità ben scelte, permette di vedere con chiarezza cosa significhi, operativamente, "perdere informazione" a causa del rumore.

Immaginiamo che Anna voglia trasmettere a Bruno un singolo bit, cioè un valore che può essere soltanto 0 oppure 1. Il canale fisico attraverso cui avviene la trasmissione non è perfetto: con una certa probabilità p, il bit subisce un'inversione e arriva a destinazione con il valore sbagliato; con la probabilità complementare $1-p$, invece, viene ricevuto correttamente. La parola "simmetrico" nel nome del modello si riferisce al fatto che questa probabilità di errore p è la stessa indipendentemente da quale valore sia stato inviato: il canale non "preferisce" invertire gli zeri piuttosto che gli uni, né viceversa. Non c'è alcuna asimmetria o bias sistematico verso uno dei due simboli.

Per concentrarci sull'effetto del canale, supponiamo che Anna invii 0 e 1 con la stessa frequenza: in media, metà dei bit trasmessi sono zero e metà sono uno. Questo è il regime in cui l'incertezza su ciò che viene trasmesso è massima — si tratta del caso più difficile, in un certo senso, perché Bruno non può fare affidamento su alcuna informazione a priori. In queste condizioni, prima ancora di ricevere qualunque segnale, Bruno si trova di fronte a un'incertezza di esattamente 1 bit su ciò che Anna ha inviato.

Ora, cosa cambia quando Bruno osserva il bit ricevuto Y? Se il canale fosse perfetto ($p = 0$), la risposta sarebbe: cambia tutto. Leggere $Y = 0$ garantisce che Anna ha inviato 0, e leggere $Y = 1$ garantisce che ha inviato 1. L'incertezza si azzera completamente. All'estremo opposto, se $p = 1/2$, il canale è completamente caotico: il bit ricevuto è indipendente da quello inviato, come se venisse estratto a sorte a prescindere dall'ingresso. In questo caso, osservare Y non dà a Bruno alcuna informazione utile su X: l'incertezza rimane invariata, ancora pari a 1 bit.

Tra questi due estremi si colloca il caso generale: un valore di p compreso tra 0 e $1/2$ produce un canale "parzialmente rumoroso", in cui osservare l'uscita riduce l'incertezza ma non la elimina del tutto. L'incertezza residua che rimane su X dopo aver osservato Y è misurata dall'entropia condizionata $H(X \mid Y)$, che vale

$$H(X \mid Y) = H_2(p) = -\left[(1-p)\log_2(1-p) + p\log_2 p\right]. \tag{3.17}$$

Questa quantità è zero quando il canale è perfetto, 1 quando è completamente casuale, e cresce in modo regolare al crescere di p: più il canale è rumoroso, meno l'osservazione dell'uscita riduce l'incertezza sull'ingresso. Si noti che $H_2(p)$ è simmetrica attorno a $p = 1/2$: un canale che inverte i bit con probabilità $p = 0.9$ è altrettanto "informativo" di uno con $p = 0.1$, perché nel primo caso Bruno può semplicemente ribaltare sistematicamente ciò che riceve e ottenere lo stesso risultato.

La quantità di informazione che effettivamente "sopravvive" al rumore, cioè che transita dall'ingresso all'uscita in modo recuperabile, è l'informazione mutua $I(X;Y) = H(X) - H(X \mid Y)$. Intuitivamente, è ciò che resta dell'incertezza originale una volta sottratta la parte che il canale non riesce a trasmettere. Quando il canale è perfetto, tutta l'informazione passa e $I(X;Y) = 1$ bit; quando il canale è completamente casuale, nulla passa e $I(X;Y) = 0$. Per valori intermedi di p, l'informazione mutua decresce in modo continuo al crescere del rumore, descrivendo con precisione quanto il canale sia in grado di fare da "ponte" tra mittente e destinatario.

Sulla codifica di canale

L'esempio del BSC introduce in modo naturale una domanda più ambiziosa: dato un canale rumoroso, qual è la massima quantità di informazione che possiamo sperare di trasmettergli in modo affidabile? La risposta a questa domanda è uno dei risultati più profondi e sorprendenti dell'intera teoria dell'informazione.

Il primo passo è riconoscere che l'informazione mutua $I(X;Y)$ — la quantità di informazione che "sopravvive" al rumore — non è una proprietà fissa del canale, ma dipende anche da come decidiamo di usarlo. In particolare, dipende dalla distribuzione con cui scegliamo i simboli da trasmettere. Su certi canali, alcune distribuzioni in ingresso permettono di trasferire più informazione di altre: scegliere bene come codificare i messaggi non è un dettaglio, è parte integrante del problema. Shannon formalizzò questa osservazione definendo la capacità di canale C come il massimo valore di informazione mutua che si riesce a ottenere ottimizzando la distribuzione dell'ingresso. Per il BSC, la scelta uniforme — inviare 0 e 1 con la stessa frequenza — è già ottimale, e la capacità risulta $C = 1 - H_2(p)$, dove $H_2(p)$ è l'entropia binaria del parametro di errore.

Il significato di C è intuitivo: rappresenta il numero massimo di bit di informazione utile che il canale può trasportare per ogni uso, nell'ipotesi di poter scegliere liberamente come codificare i messaggi. Quando $p = 0$, il canale è perfetto e $C = 1$: ogni uso del canale trasporta un bit intero. Quando $p = 1/2$, il rumore è così intenso che l'uscita è completamente indipendente dall'ingresso, e $C = 0$: il canale non riesce a trasportare alcuna informazione utilizzabile, indipendentemente da come si tenti di usarlo.

Il "secondo teorema fondamentale di Shannon", noto come teorema di codifica di canale, stabilisce due fatti complementari di straordinaria portata.

Il primo è un limite invalicabile. Se si tenta di trasmettere informazione a un ritmo R superiore alla capacità del canale, nessuna strategia di codifica — per quanto sofisticata, per quanto lunga sia la sequenza trasmessa — può impedire che la probabilità di errore rimanga significativa. Non è una questione di ingegnosità o di potenza di calcolo: è un limite fondamentale, intrinseco alla struttura del canale. Il canale semplicemente non ha abbastanza "spazio informativo" per sostenere quel ritmo.

Il secondo fatto è costruttivo, e in un certo senso sorprendente. Se si trasmette a un ritmo R strettamente inferiore alla capacità, allora esistono codici — schemi di codifica basati su blocchi sufficientemente lunghi — capaci di rendere la probabilità di errore arbitrariamente piccola. Non solo piccola: piccola quanto si vuole, al costo di aumentare la lunghezza dei blocchi trasmessi. In altre parole, un canale rumoroso non condanna a una comunicazione inaffidabile: impone soltanto che si "paghi" una quota di ridondanza, rinunciando a trasmettere più di C bit utili per uso.

Esempio 3.1.1

Vale la pena illustrare questa idea con un esempio elementare, anche se lontano dall'ottimalità. Supponiamo di usare il BSC con $p = 0.1$: ogni bit ha il 10% di probabilità di essere invertito. Se trasmettessimo ogni bit così com'è, uno su dieci arriverebbe sbagliato — spesso inaccettabile. Una strategia semplicissima per ridurre gli errori è la "ripetizione": invece di trasmettere ogni bit una volta sola, lo trasmettiamo tre volte, e il ricevitore adotta una regola a maggioranza (se riceve almeno due 1, decide che è stato inviato 1; altrimenti decide 0).
Con questo schema, un errore si verifica solo se almeno due dei tre bit trasmessi vengono invertiti, evento che ha probabilità circa $3p^2(1-p) \approx 0.027$ per $p = 0.1$: l'errore è sceso dal 10% al 2.7%. Potremmo ripetere ogni bit cinque volte, sette volte, e così via, riducendo l'errore ulteriormente a ogni passo. Il prezzo da pagare è il ritmo di trasmissione: con ripetizione tripla si trasmette solo $1/3$ di bit utile per uso di canale, con ripetizione quintupla solo $1/5$, e così via. Il teorema di Shannon

ci dice che non siamo costretti a scendere così in basso: per $p = 0.1$, la capacità del BSC è $C = 1 - H_2(0.1) \approx 0.53$ bit per uso, ben al di sopra di $1/3$. Esistono dunque codici molto più efficienti della semplice ripetizione, capaci di avvicinarsi a C mantenendo errori trascurabili — anche se costruirli esplicitamente è un problema di ingegneria tutt'altro che banale, che ha occupato i teorici delle comunicazioni per decenni.

I due teoremi fondamentali di Shannon delineano insieme un quadro di grande coerenza. Il primo teorema riguarda la sorgente: l'entropia $H(X)$ è la soglia minima sotto la quale non si può comprimere un messaggio senza perdere informazione, ma a cui ci si può avvicinare quanto si vuole con una codifica adeguata. Il secondo teorema riguarda il canale: la capacità C è la soglia massima oltre la quale non si può comunicare in modo affidabile, ma al di sotto della quale gli errori possono essere resi trascurabili.

Tra questi due estremi si colloca gran parte dell'ingegneria moderna delle comunicazioni. La compressione dei dati — dai formati audio e video alle trasmissioni satellitari — mira a eliminare la ridondanza della sorgente, avvicinandosi al limite di Shannon per la codifica di sorgente. La codifica di canale — dai codici a correzione d'errore nei dischi rigidi ai codici turbo nelle reti mobili — mira invece a introdurre la ridondanza giusta per combattere il rumore, avvicinandosi al limite di Shannon per la codifica di canale. Comprimere e proteggere sono, in questo senso, due facce della stessa medaglia.

3.2 Codici binari

Proviamo ora a definire in modo più preciso cosa intendiamo per codice binario e quali proprietà sono richieste affinché un codice sia realmente utilizzabile. L'obiettivo non è soltanto assegnare a ogni simbolo una stringa di bit, ma farlo in modo tale che la codifica sia interpretabile senza ambiguità e, possibilmente, anche semplice da decodificare e vicina ai limiti teorici della compressione.

Dato l'insieme dei simboli $S = \{s_1, \dots, s_m\}$, un codice binario per S è una funzione iniettiva $c : S \mapsto \{0,1\}^+$, dove $\{0,1\}^+$ è l'insieme di tutte le sequenze di 0 e 1 con almeno un simbolo. Si noti che due simboli diversi non possono essere codificati allo stesso modo, per cui non può essere che ci siano s_i e s_j con $c(s_i) = c(s_j)$. Matematicamente, questo vuol dire che il codice deve corrispondere a una funzione iniettiva.

Dato che in pratica quello che si codificherà sono sequenze di simboli, estendiamo il discorso a questo caso mediante la definizione di codice esteso, che data una sequenza di simboli $x_1 x_2 \dots x_n$ fornisce la sequenza delle loro codifiche $c(x_1)c(x_2)\dots c(x_n)$.

Questa definizione riflette esattamente ciò che accade in pratica: un testo codificato è un flusso continuo di bit, senza alcun marcatore che indichi dove finisce un simbolo e dove inizia il successivo. Proprio per questo diventa centrale la domanda: come fa il decodificatore a sapere "dove tagliare" la sequenza di bit per ricostruire i simboli originali? La risposta dipende interamente dalla struttura del codice.

Esempio 3.2.1

Per rendere concreto il concetto, consideriamo una sorgente X con alfabeto $S = \{a, b, c, d\}$ e probabilità $\{1/2, 1/4, 1/8, 1/8\}$. Definiamo il codice:

s_i	a	b	c	d
$c(s_i)$	1000	0100	0010	0001
l_i	4	4	4	4

In questo caso tutte le codifiche hanno la stessa lunghezza: si tratta quindi di un codice a lunghezza fissa. Il pregio di una codifica a lunghezza fissa è evidente: la decodifica è immediata perché basta leggere blocchi di 4 bit alla volta. Tuttavia l'esempio mette in luce anche un limite: il codice non sfrutta in alcun modo il fatto che a è molto più probabile di c e d. Usando 4 bit per tutti i simboli, stiamo "pagando" lo stesso prezzo per eventi che hanno probabilità molto diverse. Se l'obiettivo è comprimere, dobbiamo permettere lunghezze diverse.

Codici binari e decodificabilità

Prima di addentrarci nei risultati teorici sulla compressione, vale la pena fermarsi su una domanda apparentemente semplice: cosa significa, concretamente, codificare un messaggio in binario? L'idea di base è intuitiva — assegnare a ogni simbolo una sequenza di bit — ma i requisiti che un codice deve soddisfare per essere davvero utile sono più sottili di quanto sembri.

Dato un alfabeto di simboli $S = \{s_1, \dots, s_m\}$, un codice binario è semplicemente una regola che associa a ogni simbolo una stringa di zeri e uni. Il requisito minimo è che simboli diversi ricevano codifiche diverse: se a e b vengono entrambi codificati come `01`, non c'è modo di capire, a partire dai bit, quale dei due fosse il simbolo originale. In termini tecnici, il codice deve essere una funzione iniettiva. Un codice che non rispetta questa proprietà non è utilizzabile nemmeno in linea di principio.

Un codice non iniettivo: l'ambiguità immediata

Per capire perché l'iniettività è indispensabile, consideriamo un alfabeto $\{a, b, c\}$ e la seguente assegnazione:

s_i	a	b	c
$c(s_i)$	`0`	`10`	`0`

Qui a e c ricevono la stessa codifica `0`. Il problema è immediato: ricevendo il bit `0`, il decodificatore non sa se il simbolo trasmesso fosse a oppure c. L'ambiguità non riguarda la sequenza — riguarda già il singolo simbolo. Un codice di questo tipo è semplicemente inutilizzabile: non esiste nessuna strategia di decodifica che possa funzionare, perché l'informazione è irrimediabilmente persa già al momento della codifica.

Il problema del "dove tagliare"

Supposto che il codice sia iniettivo, rimane un secondo problema, più sottile. In pratica, quello che si codifica non è mai un simbolo isolato, ma una sequenza di simboli — una parola, un testo, un flusso di dati. La codifica di una sequenza si ottiene semplicemente concatenando le codifiche dei singoli simboli, uno dopo l'altro, senza alcun separatore. È questa concatenazione a rendere le cose interessanti, e a volte complicate.

Immaginiamo che il decodificatore riceva il flusso di bit `0110100011`. Come fa a sapere dove finisce la codifica del primo simbolo e dove inizia quella del secondo? In un testo scritto, le parole sono separate da spazi; in un flusso binario, non esiste nulla di analogo. Il decodificatore deve riuscire a "tagliare" la sequenza nei punti giusti basandosi esclusivamente sulla struttura del codice.

Il modo più semplice per risolvere questo problema è usare un codice a "lunghezza fissa": se ogni simbolo è sempre codificato con lo stesso numero di bit, il decodificatore sa esattamente quanti bit leggere alla volta. Consideriamo per esempio una sorgente con quattro simboli $\{a, b, c, d\}$ e il codice seguente:

s_i	a	b	c	d
$c(s_i)$	1000	0100	0010	0001
l_i	4	4	4	4

Con questo codice, decodificare è immediato: si leggono quattro bit, si consulta la tabella, si passa ai quattro bit successivi. Non c'è alcuna ambiguità possibile.

Il prezzo da pagare, però, è l'inefficienza. Supponiamo che i simboli abbiano probabilità molto diverse: a compare nel 50% dei casi, b nel 25%, c e d nel 12.5% ciascuno. Usando sempre 4 bit per simbolo, trattiamo a e d allo stesso modo, anche se a è quattro volte più frequente. Se l'obiettivo è comprimere, questo è uno spreco evidente: avrebbe senso usare codifiche più corte per i simboli che compaiono spesso e più lunghe per quelli rari.

Codici a lunghezza variabile: iniettivi ma non univocamente decodificabili

Per sfruttare le differenze di probabilità, si usano codici a "lunghezza variabile": simboli frequenti ricevono codifiche corte, simboli rari ricevono codifiche lunghe. Questo è esattamente il principio alla base della codifica di Huffman e, più in generale, di tutti i codici orientati alla compressione. Ma la flessibilità ha un costo: il problema del "dove tagliare" si fa più delicato.

La proprietà fondamentale che un codice a lunghezza variabile deve possedere è la "decodificabilità univoca": non devono esistere due sequenze di simboli diverse che producano la stessa sequenza di bit. Si noti che questa proprietà è più forte della semplice iniettività sui singoli simboli. È possibile costruire un codice in cui ogni simbolo ha una codifica diversa, ma la concatenazione produce ambiguità. Un esempio chiarisce il punto. Consideriamo il codice:

s_i	a	b	c	d
$c(s_i)$	0	01	10	1

Questo codice è iniettivo: tutti e quattro i simboli hanno codifiche distinte. Eppure, consideriamo la sequenza di bit `01`. Può essere decodificata come ab (prima `0`, poi `1`), oppure come b (direttamente `01`), oppure ancora come da (`1` seguito da `0`, leggendo da destra). Tre decodifiche diverse, tutte internamente coerenti con il codice. Questo codice è iniettivo, ma non è univocamente decodificabile: la stessa sequenza di bit corrisponde a messaggi diversi, e non c'è modo di scegliere quello giusto senza informazioni aggiuntive.

Decodificabilità univoca ma con lookahead

Anche quando un codice è univocamente decodificabile, la decodifica può risultare scomoda. Consideriamo il seguente codice:

s_i	a	b	c	d
$c(s_i)$	10	01	101	000

Questo codice è univocamente decodificabile: si può dimostrare che ogni sequenza di bit ammette al più una decodifica valida. Tuttavia, la decodifica non è immediata. Ricevendo il flusso `101001`, il decodificatore legge i primi due bit `10` e potrebbe interpretarli come a; ma potrebbe anche aspettare e leggere `101`, interpretandolo come c. Solo guardando il bit successivo è possibile risolvere l'ambiguità: se il terzo bit è `1`, i primi tre bit formano c; se è `0`, i primi due formano a e si riparte. Questo fenomeno si chiama "lookahead": il decodificatore è costretto a "sbirciare avanti" nel flusso per capire cosa ha già ricevuto. In applicazioni dove i dati arrivano in tempo reale — un flusso audio, una trasmissione di rete — il lookahead è scomodo, introduce latenza e può rendere la decodifica fragile in caso di errori.

Codici prefisso: decodifica immediata

Esiste una classe di codici che evita completamente il problema del lookahead: i codici prefisso. Un codice è detto prefisso se nessuna parola di codice è il prefisso iniziale di un'altra. In altre parole, non esiste una coppia di simboli s_i e s_j tali che la codifica di s_i coincida con l'inizio della codifica di s_j.

Un esempio di codice "non" prefisso, ma univocamente decodificabile, è il seguente:

s_i	a	b	c	d
$c(s_i)$	`1`	`10`	`100`	`1000`

Qui `1` è prefisso di `10`, che è prefisso di `100`, e così via: ogni parola di codice inizia con tutte le parole più corte. Ciononostante, si può dimostrare che il codice è univocamente decodificabile, perché la decodifica può procedere da destra a sinistra oppure attendendo il primo `0` come marcatore di fine simbolo. Il costo, però, è un lookahead potenzialmente illimitato: per sapere se il bit iniziale `1` è da solo oppure fa parte di una parola più lunga, bisogna leggere tutti i bit successivi fino al prossimo `0`. In pratica, un codice del genere è quasi impossibile da usare in modo efficiente.

Per capire perché i codici prefisso eliminano questo problema, si pensi a cosa fa il decodificatore mentre legge il flusso bit per bit. Con un codice prefisso, nel momento in cui riconosce una parola valida, sa con certezza che il simbolo corrispondente è terminato: nessun bit aggiuntivo potrebbe mai trasformare quella parola nella codifica di un altro simbolo. Non c'è niente da aspettare, niente da "sbirciare": la decodifica è davvero immediata.

Un esempio di codice prefisso ben costruito, per lo stesso alfabeto di quattro simboli con probabilità $\{1/2, 1/4, 1/8, 1/8\}$, è:

s_i	a	b	c	d
$c(s_i)$	`0`	`10`	`110`	`111`
l_i	1	2	3	3

Nessuna di queste parole è prefisso di un'altra. Ricevendo il flusso `10110111`, il decodificatore legge `1`, si accorge che non è una parola valida, legge `10` e riconosce b; poi legge `1`, poi `11`, poi `110` e riconosce c; infine legge `111` e riconosce d. In ogni passaggio, non è stato necessario guardare oltre il bit corrente per completare la decodifica. Si noti inoltre come questo codice assegni la codifica più corta al simbolo più frequente (a con probabilità $1/2$) e le codifiche più lunghe ai simboli rari: è un codice pensato per comprimere, non soltanto per evitare ambiguità.

È importante notare che la proprietà di prefisso è "sufficiente" ma non "necessaria" per la decodificabilità univoca. Esistono codici univocamente decodificabili che non sono prefisso, come l'esempio con `1`, `10`, `100`, `1000` discusso sopra, ma la loro decodifica richiede lookahead o algoritmi più elaborati. Per questa ragione, nella pratica si preferisce quasi sempre lavorare con codici prefisso: offrono le stesse garanzie teoriche dei codici univocamente decodificabili, con il vantaggio aggiunto di una decodifica semplice e immediata. Vedremo che questa scelta non comporta alcuna perdita dal punto di vista della compressione: per ogni codice univocamente decodificabile esiste un codice prefisso con le stesse lunghezze di codifica, e quindi con la stessa efficienza.

Visualizzazione ad albero

Un modo molto utile per visualizzare i codici prefisso è attraverso un albero binario. Ogni nodo dell'albero rappresenta una stringa binaria: muoversi lungo un ramo corrisponde ad aggiungere un bit, tipicamente 0 su un ramo e 1 sull'altro. In questa rappresentazione, un codice prefisso corrisponde a scegliere

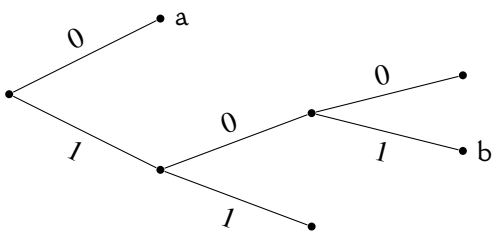

Figura 3.1: Albero binario per codice prefisso $a = 0, b = 101$

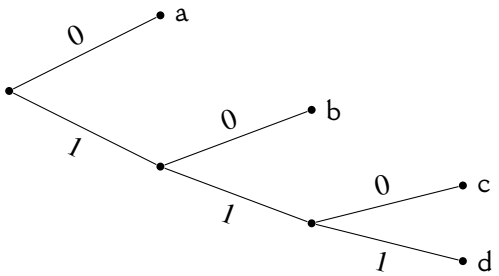

Figura 3.2: Albero binario per codice prefisso $a = 0, b = 10, c = 110, d = 111$

alcune foglie e associare a esse i simboli; la codifica di un simbolo è il cammino dalla radice alla foglia corrispondente. In Figura 3.1 è mostrato l'albero per il codice $a = 0, b = 101$.

Analogamente, il codice $a = 0, b = 10, c = 110, d = 111$ è prefisso, e il suo albero associato è mostrato in Figura 3.2.

La rappresentazione ad albero non è solo un ausilio visivo: fornisce anche un criterio molto semplice per verificare la proprietà di prefisso. Un codice è prefisso se e solo se nessuna parola di codice corrisponde a un nodo che sia antenato di un altro nodo selezionato. Detto altrimenti, i nodi selezionati devono essere "foglie" rispetto all'insieme delle parole scelte, così che nessuna codifica termini in un punto da cui potrebbe proseguire per formare un'altra codifica. Guardando Figura 3.2, si vede immediatamente che nessun nodo etichettato (a, b, c, d) è sul percorso che porta a un altro nodo etichettato.

L'idea di "codice prefisso" non è confinata al caso binario. Ad esempio, qualunque strategia di soluzione del problema delle 12 palline equivale alla definizione di un codice prefisso ternario: a ogni possibile soluzione viene associata una sequenza di lunghezza tre, corrispondente agli esiti delle pesate che la individuano come soluzione. Considerando l'alfabeto $\{D, S, E\}$ associato ai tre esiti di una pesata, la soluzione descritta in precedenza definisce il codice in Figura 3.3.

Questo collegamento rende evidente un principio generale: ogni sequenza di decisioni con un numero finito di esiti può essere vista come un cammino in un albero, e la richiesta di riconoscere la soluzione senza ambiguità equivale a richiedere una struttura di tipo prefisso.

Efficienza e lunghezza attesa

La terza esigenza per un buon codice riguarda la compressione: un codice può essere perfettamente decodificabile e semplice, ma inefficiente in termini di lunghezza media. Per quantificare questo aspetto abbiamo introdotto la lunghezza attesa, intesa come

$$L(c, X) = \sum_{i=1}^{n} p(s_i) l(s_i) = \sum_{i=1}^{n} p_i l_i. \tag{3.18}$$

1^+	DDD	1^-	SSE
2^+	DDS	2^-	SDE
3^+	DSD	3^-	SED
4^+	DSS	4^-	SEE
5^+	SDD	5^-	DSE
6^+	SDS	6^-	DDE
7^+	SSD	7^-	DED
8^+	SSS	8^-	DEE
9^+	EDD	9^-	ESD
10^+	EDS	10^-	ESS
11^+	EDE	11^-	ESE
12^+	EED	12^-	EEE

Figura 3.3: Codice definito da una possibile strategia di soluzione per il problema delle 12 palline

Questa è la quantità naturale da minimizzare quando si parla di compressione senza perdita: misura quanti bit, in media, vengono spesi per rappresentare ciascun simbolo.

Riprendiamo la sorgente con probabilità $\{1/2, 1/4, 1/8, 1/8\}$ e consideriamo il codice prefisso seguente, riportando anche l'informazione $h_i = -\log_2 p_i$ associata a ciascun simbolo:

s_i	$c(s_i)$	p_i	h_i	l_i
a	0	$\frac{1}{2}$	1.0	1
b	10	$\frac{1}{4}$	2.0	2
c	110	$\frac{1}{8}$	3.0	3
d	111	$\frac{1}{8}$	3.0	3

In questo caso le lunghezze l_i coincidono esattamente con i contenuti informativi h_i: il codice realizza perfettamente il principio "lunghezza proporzionale alla sorpresa". L'entropia vale $H(X) = 1.75$ bit e la lunghezza attesa $L(c, X) = 1.75$, quindi il codice è ottimo. Questo è possibile perché le probabilità sono tutte potenze di $1/2$: quando così non è, le lunghezze ideali non sono intere e si introduce un piccolo errore di arrotondamento.

È anche istruttivo osservare che un codice non prefisso può raggiungere la stessa lunghezza attesa $L(c, X) = H(X)$, pur sacrificando la decodifica immediata. Il codice seguente, ad esempio, non è prefisso (la codifica di a è prefisso di quelle di b e c), ma è decodificabile in modo univoco:

s_i	$c(s_i)$	p_i	h_i	l_i
a	0	$\frac{1}{2}$	1.0	1
b	01	$\frac{1}{4}$	2.0	2
c	011	$\frac{1}{8}$	3.0	3
d	111	$\frac{1}{8}$	3.0	3

Questo esempio chiarisce un punto concettuale: la proprietà di prefisso è sufficiente per la decodificabilità univoca e per una decodifica semplice, ma non è necessaria per la sola decodificabilità univoca. Tuttavia, proprio perché vogliamo codici non solo corretti ma anche pratici, è naturale privilegiare i codici prefisso.

Costruire codici efficienti con la codifica di Shannon–Fano

Stabilito che i codici prefisso sono la struttura giusta su cui lavorare, rimane aperta la domanda più concreta: dato un insieme di simboli con le rispettive probabilità, come si costruisce effettivamente un codice prefisso che sia vicino al limite entropico? Esistono diversi metodi, ma uno storicamente e concettualmente importante è la codifica di Shannon–Fano.

L'idea di fondo è molto diretta, quasi ovvia una volta enunciata: i simboli più probabili dovrebbero ricevere codifiche più corte, e quelli più rari codifiche più lunghe. Quanto più corte? La risposta naturale è che la lunghezza della codifica di un simbolo dovrebbe essere proporzionale al suo contenuto informativo, cioè a $-\log_2 p_i$: un simbolo con probabilità $1/2$ porta 1 bit di informazione e dovrebbe essere codificato con 1 bit; un simbolo con probabilità $1/4$ porta 2 bit e dovrebbe usarne 2; e così via. Il problema è che $-\log_2 p_i$ non è in generale un numero intero. Shannon–Fano risolve questo inconveniente nel modo più semplice possibile: arrotonda verso l'alto all'intero più vicino. Il simbolo s_i riceve dunque una codifica di lunghezza $l_i = \lceil \log_2(1/p_i) \rceil$ bit.

Come si costruisce il codice

Una volta fissate le lunghezze, occorre costruire concretamente le parole di codice. Un modo elegante per farlo è attraverso una "divisione ricorsiva" dell'insieme dei simboli, che produce direttamente un codice prefisso.

La procedura è la seguente. Si ordinano i simboli per probabilità decrescente. Si dividono in due gruppi la cui probabilità totale sia il più possibile bilanciata, cioè il più vicina possibile a $1/2$ ciascuno. Al primo gruppo si assegna il prefisso `0`, al secondo il prefisso `1`. Poi si ripete la stessa divisione all'interno di ciascun gruppo, aggiungendo ogni volta un bit al prefisso, fino a quando ogni gruppo contiene un solo simbolo.

L'effetto di questa costruzione è visibile sull'albero binario che rappresenta il codice: i simboli più probabili tendono a trovarsi vicino alla radice (parole corte), mentre quelli meno probabili finiscono in profondità (parole lunghe). Il legame tra probabilità e profondità è reso esplicito e sistematico.

Un esempio

Riprendiamo la sorgente con quattro simboli e probabilità $p(a) = 1/2$, $p(b) = 1/4$, $p(c) = p(d) = 1/8$. I contenuti informativi sono $-\log_2(1/2) = 1$, $-\log_2(1/4) = 2$, $-\log_2(1/8) = 3$, $-\log_2(1/8) = 3$, tutti già interi: non è necessario alcun arrotondamento, e le lunghezze obiettivo sono $l_a = 1$, $l_b = 2$, $l_c = l_d = 3$.

Applicando la divisione ricorsiva: il gruppo $\{a\}$ ha probabilità totale $1/2$, il gruppo $\{b, c, d\}$ ha probabilità totale $1/2$. La divisione è perfettamente bilanciata. Si assegna $a \mapsto$ `0` e il prefisso `1` al gruppo restante. All'interno di $\{b, c, d\}$, il gruppo $\{b\}$ ha probabilità $1/4$ e il gruppo $\{c, d\}$ ha probabilità $1/4$: ancora bilanciato. Si ottiene $b \mapsto$ `10` e il prefisso `11` per $\{c, d\}$. Infine, c e d vengono separati: $c \mapsto$ `110`, $d \mapsto$ `111`.

s_i	a	b	c	d
p_i	$1/2$	$1/4$	$1/8$	$1/8$
l_i	1	2	3	3
$c(s_i)$	`0`	`10`	`110`	`111`

La lunghezza media di questo codice è $L = 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{8} + 3 \cdot \frac{1}{8} = 1.75$ bit per simbolo, che coincide esattamente con l'entropia della sorgente $H(X) = 1.75$ bit. In questo esempio, Shannon–Fano produce un codice ottimo.

Un esempio meno fortunato

Non sempre le probabilità sono potenze di $1/2$, e in quei casi l'arrotondamento introduce un piccolo costo. Consideriamo una sorgente con tre simboli e probabilità $p(a) = 0.5$, $p(b) = 0.3$, $p(c) = 0.2$. I contenuti informativi sono circa 1, 1.74, 2.32 bit rispettivamente, e le lunghezze arrotondate sono $l_a = 1$, $l_b = 2$, $l_c = 3$.

Applicando Shannon–Fano: $\{a\}$ contro $\{b, c\}$ divide le probabilità in 0.5 e 0.5, bilanciato. All'interno di $\{b, c\}$, b e c vengono separati. Il codice risultante è $a \mapsto$ `0`, $b \mapsto$ `10`, $c \mapsto$ `11`.

s_i	a	b	c
p_i	0.5	0.3	0.2
l_i	1	2	2
$c(s_i)$	`0`	`10`	`11`

La lunghezza media è $L = 1 \cdot 0.5 + 2 \cdot 0.3 + 2 \cdot 0.2 = 1.5$ bit per simbolo. L'entropia della sorgente è $H(X) = -0.5 \log_2 0.5 - 0.3 \log_2 0.3 - 0.2 \log_2 0.2 \approx 1.485$ bit. Il codice è quindi a soli 0.015 bit dall'ottimo: l'arrotondamento ha introdotto uno spreco minimo.

In generale, si può dimostrare che la lunghezza media di un codice Shannon–Fano non si discosta mai di più di 1 bit dall'entropia. Questo significa che Shannon–Fano costruisce sempre un codice "ragionevole": nel caso peggiore, si paga un bit in più per simbolo rispetto al minimo teorico. Il limite di 1 bit deriva proprio dall'arrotondamento delle lunghezze: se le probabilità non sono esatte potenze di $1/2$, ogni simbolo potrebbe richiedere al massimo un bit aggiuntivo rispetto al suo contenuto informativo ideale. Shannon–Fano è quindi un primo esempio concreto di come i limiti di Shannon non siano solo affermazioni astratte, ma possano guidare la costruzione di codici reali ed efficienti.

La disuguaglianza di Kraft–McMillan

Quando si progetta un codice a lunghezza variabile, una domanda pratica si pone quasi subito: come si fa a sapere se un certo insieme di lunghezze è compatibile con un codice decodificabile? Devo effettivamente costruire il codice per verificarlo, oppure esiste un criterio più diretto?

La risposta è fornita dalla disuguaglianza di Kraft–McMillan, uno dei risultati più utili della teoria dei codici. Il teorema afferma che un insieme di lunghezze $l_1, l_2, \ldots, l_n$ è compatibile con un codice univocamente decodificabile se e solo se la somma $\sum_i 2^{-l_i}$ non supera 1. Non è necessario costruire il codice: basta calcolare questa somma e verificare se è al massimo uguale a 1.

Il criterio è sorprendentemente semplice. Per esempio, le lunghezze $1, 2, 3, 3$ soddisfano la disuguaglianza: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 0.5 + 0.25 + 0.125 + 0.125 = 1$. Esiste dunque un codice decodificabile con quelle lunghezze — e abbiamo già visto che si tratta del codice $a \mapsto$ `0`, $b \mapsto$ `10`, $c \mapsto$ `110`, $d \mapsto$ `111`. Al contrario, le lunghezze $1, 1, 2, 2$ danno $2^{-1} + 2^{-1} + 2^{-2} + 2^{-2} = 1.5 > 1$: nessun codice decodificabile può avere queste lunghezze, indipendentemente da come si scelgano le parole di codice.

Il teorema ha anche un secondo aspetto, altrettanto importante: ogni insieme di lunghezze che soddisfa la disuguaglianza non solo è compatibile con "qualche" codice decodificabile, ma è compatibile con un codice "prefisso". Questo significa che, nella progettazione di un codice, non c'è mai motivo di accontentarsi di un codice univocamente decodificabile ma non prefisso: si può sempre trovare un codice prefisso con le stesse lunghezze, e quindi con la stessa lunghezza media, ma con il vantaggio della decodifica immediata.

Visualizzazione tramite albero binario

La disuguaglianza di Kraft–McMillan ha un'interpretazione geometrica particolarmente intuitiva. Si immagini un albero binario completo, in cui ogni nodo al livello k occupa una frazione 2^{-k} dello spazio totale disponibile. Ogni parola di codice corrisponde a un nodo dell'albero; scegliere un nodo come parola di codice significa "consumare" la frazione di spazio che gli compete, e anche tutta quella dei suoi discendenti, che non potranno più essere usati senza violare la proprietà di prefisso.

La disuguaglianza di Kraft afferma semplicemente che lo spazio totale consumato non può superare 1. Se è esattamente uguale a 1, l'albero è completamente "coperto" dalle parole di codice e non c'è spazio sprecato: il codice è massimalmente efficiente nella struttura dell'albero. Se è strettamente minore di 1, alcune foglie rimangono inutilizzate, il che suggerisce che potrebbe esserci margine per accorciare qualche parola di codice.

In Figura 3.4 è mostrato l'albero per il codice $a \mapsto \texttt{0}$, $b \mapsto \texttt{10}$, $c \mapsto \texttt{110}$, $d \mapsto \texttt{111}$, che soddisfa Kraft con uguaglianza: i nodi selezionati coprono esattamente tutto lo spazio disponibile, senza sovrapposizioni né lacune.

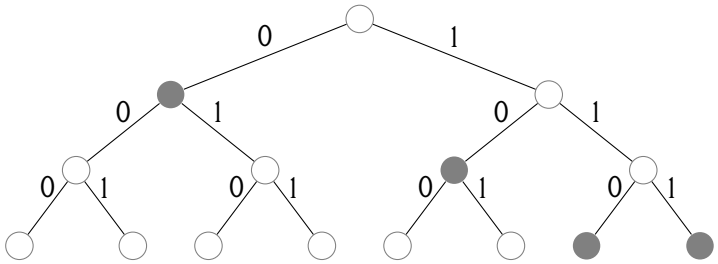

Figura 3.4: Nodi associati al codice $a = 0, b = 10, c = 110, d = 111$

In Figura 3.5 è mostrato invece l'albero per il codice $a \mapsto \texttt{0}$, $b \mapsto \texttt{01}$, $c \mapsto \texttt{011}$, $d \mapsto \texttt{111}$, che è univocamente decodificabile ma non prefisso: guardando l'albero, il nodo corrispondente ad a è antenato del nodo corrispondente a b, violando la condizione di prefisso. Entrambi i codici soddisfano la disuguaglianza di Kraft con uguaglianza — hanno le stesse lunghezze — ma solo il primo offre la comodità della decodifica immediata.

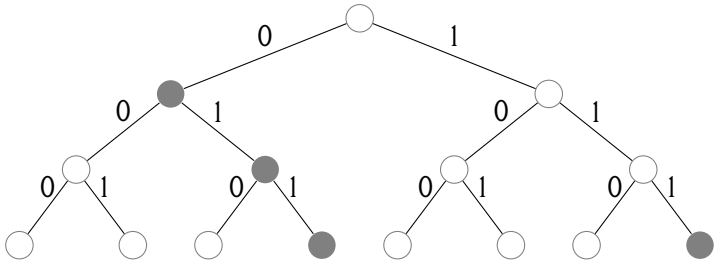

Figura 3.5: Nodi associati al codice $a = 0, b = 01, c = 011, d = 111$

3.3 Codici di Huffman

Nel capitolo precedente abbiamo incontrato la codifica di Shannon–Fano, che costruisce un codice prefisso assegnando a ogni simbolo una lunghezza proporzionale al suo contenuto informativo. Abbiamo

visto che questo metodo garantisce una lunghezza media entro 1 bit dall'entropia, il che è già un risultato notevole. Tuttavia, Shannon–Fano ha un difetto che vale la pena mettere a fuoco prima di procedere: non è ottimo.

Il limite di Shannon–Fano

Shannon–Fano costruisce il codice attraverso una divisione ricorsiva dall'alto verso il basso: a ogni passo si separa l'insieme dei simboli in due gruppi di probabilità il più possibile bilanciata, si assegna `0` a un gruppo e `1` all'altro, e si ripete. L'idea è sensata, ma nasconde un problema strutturale: ogni divisione viene effettuata guardando solo le probabilità "totali" dei due gruppi, senza considerare come queste probabilità si distribuiranno internamente nei passi successivi. In altre parole, Shannon–Fano ottimizza localmente, passo per passo, senza garanzia che le scelte locali si combinino in un codice globalmente ottimo.

Per capire dove questo può portare fuori strada, consideriamo una sorgente con cinque simboli e probabilità:

s_i	a	b	c	d	e
p_i	0.35	0.25	0.20	0.15	0.05

Applichiamo Shannon–Fano. Ordiniamo i simboli per probabilità decrescente e cerchiamo la divisione più bilanciata. La scelta migliore è $\{a, b\}$ con probabilità totale 0.60 contro $\{c, d, e\}$ con probabilità totale 0.40; oppure $\{a\}$ con 0.35 contro $\{b, c, d, e\}$ con 0.65. La prima divisione è più bilanciata, e Shannon–Fano la sceglie: il gruppo $\{a, b\}$ riceve il prefisso `0`, il gruppo $\{c, d, e\}$ riceve il prefisso `1`.

All'interno di $\{a, b\}$, l'unica divisione possibile è $\{a\}$ contro $\{b\}$: si ottiene $a \mapsto$ `00`, $b \mapsto$ `01`. All'interno di $\{c, d, e\}$, la divisione più bilanciata è $\{c\}$ con 0.20 contro $\{d, e\}$ con 0.20: si ottiene $c \mapsto$ `10`, e poi $d \mapsto$ `110`, $e \mapsto$ `111`.

Il codice di Shannon–Fano risultante è:

s_i	a	b	c	d	e
$c(s_i)$	`00`	`01`	`10`	`110`	`111`
l_i	2	2	2	3	3

La lunghezza media è:

$$L_{\mathrm{SF}} = 2 \cdot 0.35 + 2 \cdot 0.25 + 2 \cdot 0.20 + 3 \cdot 0.15 + 3 \cdot 0.05 = 2.20 \text{ bit per simbolo.}$$

Bottom-up batte top-down

L'intuizione alla base dei codici di Huffman è diversa da quella di Shannon–Fano: invece di partire dall'alto e dividere, si parte dal basso e si "accorpa". L'idea è semplice: i simboli meno probabili sono quelli che riceveranno le codifiche più lunghe. Se due simboli sono i meno probabili in assoluto, ha senso trattarli come una coppia e costruire il codice attorno a questa coppia, che nel linguaggio degli alberi significa assegnar loro due foglie sorelle, figlie dello stesso nodo.

Questo cambio di prospettiva, dal top-down di Shannon–Fano al bottom-up di Huffman, non è solo estetico. È ciò che permette all'algoritmo di Huffman di garantire l'ottimalità in ogni caso, non solo in quelli fortunati.

Come funziona l'algoritmo

La procedura è la seguente. Si parte con tutti i simboli disposti in una lista, ciascuno con il proprio peso (la propria probabilità). A ogni passo si selezionano i "due" simboli (o gruppi di simboli) con peso minore, li si fondono in un nuovo gruppo il cui peso è la somma dei due, e si assegna `0` ai simboli del primo gruppo e `1` ai simboli del secondo. Si ripete fino a quando rimane un solo gruppo contenente tutti i simboli. Le parole di codice si ottengono leggendo, per ogni simbolo, la sequenza di bit accumulata durante le fusioni, nell'ordine inverso rispetto a quello in cui è stata costruita.

Geometricamente, questa procedura costruisce un albero binario dal basso verso l'alto: si parte dalle foglie (i simboli) e si aggiungono nodi interni fino alla radice. I simboli più rari finiscono nelle foglie più profonde, lontane dalla radice, e ottengono codifiche più lunghe. I simboli più frequenti finiscono vicino alla radice e ottengono codifiche brevi. Il risultato è un albero perfettamente calibrato sulla distribuzione della sorgente.

La decodifica è altrettanto semplice: si percorre l'albero dalla radice, leggendo un bit alla volta e scendendo verso sinistra o destra a seconda che il bit sia `0` o `1`. Quando si raggiunge una foglia, il simbolo corrispondente è stato identificato; si riparte dalla radice per il simbolo successivo. Non è necessario alcun delimitatore, né alcun lookahead: la proprietà di prefisso garantisce che la decodifica sia sempre immediata e non ambigua.

Esempio 3.3.1

Consideriamo la sorgente X con cinque simboli $\{a, b, c, d, e\}$ e probabilità $\{0.25, 0.25, 0.20, 0.15, 0.15\}$. Costruiamo il codice di Huffman passo per passo.

Passo 1. I due simboli con peso minore sono d e e, entrambi con probabilità 0.15. Li fondiamo in un gruppo $\{d, e\}$ di peso 0.30, assegnando `0` a d e `1` a e.

$\{a\}$	$\{b\}$	$\{c\}$	$\{d$=`0`, e=`1`$\}$
0.25	0.25	0.20	0.30

Passo 2. I due pesi minori sono ora 0.20 (c) e 0.25 (a o b, ne scegliamo uno, diciamo b). Li fondiamo in $\{b, c\}$ di peso 0.45, assegnando `0` a b e `1` a c.

$\{a\}$	$\{b$=`0`, c=`1`$\}$	$\{d$=`0`, e=`1`$\}$
0.25	0.45	0.30

Passo 3. I due pesi minori sono 0.25 (a) e 0.30 ($\{d, e\}$). Li fondiamo: `0` va ad a, `1` va al gruppo $\{d, e\}$, aggiornando le sequenze di d ed e che diventano `10` e `11`.

$\{a$=`0`, d=`10`, e=`11`$\}$	$\{b$=`0`, c=`1`$\}$
0.55	0.45

Passo 4. Rimangono due soli gruppi. Li fondiamo: `0` al primo gruppo, `1` al secondo. Le sequenze finali, lette al contrario, danno le parole di codice.

a	b	c	d	e
`00`	`10`	`11`	`010`	`011`

La lunghezza media è $L = 2 \cdot 0.25 + 2 \cdot 0.25 + 2 \cdot 0.20 + 3 \cdot 0.15 + 3 \cdot 0.15 = 2.30$ bit per simbolo, contro un'entropia di circa 2.29 bit. Il codice è praticamente al limite teorico.
L'albero corrispondente è mostrato in Figura **??**. Si osservi come i simboli più frequenti — a, b, c — si trovino tutti a profondità 2, mentre d ed e, i meno probabili, siano a profondità 3.

Ancora sul confronto tra Shannon-Fano e Huffman

Applichiamo ora Huffman alla stessa situazione considerata sopra per mostrare i limiti di Shannon-Fano e verifichiamo come Huffman in effetti si comporti meglio. L'algoritmo parte dai simboli meno probabili e li accorpa progressivamente.

Passo 1. I due pesi minori sono $e = 0.05$ e $d = 0.15$. Li fondiamo in $\{d, e\}$ con peso 0.20.

Passo 2. I due pesi minori sono ora $\{d, e\} = 0.20$ e $c = 0.20$. Li fondiamo in $\{c, d, e\}$ con peso 0.40.

Passo 3. I due pesi minori sono $b = 0.25$ e $\{c, d, e\} = 0.40$. Li fondiamo in $\{b, c, d, e\}$ con peso 0.65.

Passo 4. Rimangono $a = 0.35$ e $\{b, c, d, e\} = 0.65$. Fusione finale.

Il codice di Huffman risultante è:

s_i	a	b	c	d	e
$c(s_i)$	`0`	`10`	`110`	`1110`	`1111`
l_i	1	2	3	4	4

La lunghezza media è:

$$L_{\mathrm{H}} = 1 \cdot 0.35 + 2 \cdot 0.25 + 3 \cdot 0.20 + 4 \cdot 0.15 + 4 \cdot 0.05 = 2.10 \text{ bit per simbolo.}$$

L'entropia della sorgente vale $H(X) \approx 2.09$ bit: il codice di Huffman è praticamente al limite teorico, con uno scarto inferiore a 0.01 bit. Shannon–Fano si ferma a 2.20 bit, con uno scarto di 0.11 bit — più di dieci volte peggio.

Cosa è andato storto con Shannon–Fano? Il problema si trova già al primo passo. Dividendo $\{a, b\}$ da $\{c, d, e\}$, Shannon–Fano ha bilanciato bene le probabilità totali (0.60 contro 0.40), ma ha forzato a — il simbolo più frequente, con probabilità 0.35 — a condividere un prefisso con b, ottenendo per entrambi una codifica di 2 bit. Huffman, invece, riconosce che a è abbastanza probabile da meritare una codifica di 1 solo bit, e costruisce l'albero di conseguenza. Il guadagno si accumula: ogni volta che la sorgente emette a — il 35% delle volte — Huffman usa 1 bit mentre Shannon–Fano ne usa 2. Su un testo lungo, la differenza diventa apprezzabile.

Questo esempio illustra il difetto strutturale di Shannon–Fano: la ricerca di un bilanciamento al primo livello dell'albero non garantisce che i simboli più frequenti ottengano le codifiche più corte possibili. Huffman, costruendo l'albero dal basso verso l'alto e accorpando sempre i due simboli meno probabili, evita questo problema per costruzione: ogni scelta è guidata non da un criterio locale di bilanciamento, ma dalla struttura globale delle probabilità. È questa differenza — bottom-up contro top-down, ottimizzazione globale contro locale — che rende Huffman ottimo e Shannon–Fano soltanto buono.

Huffman sull'italiano

Un esempio più vivido di Huffman in azione è la costruzione del codice per la lingua italiana. A partire dalle frequenze dei caratteri (stimate su un corpus e riportate in Figura 3.6), l'algoritmo produce un codice prefisso che assegna parole più corte alle lettere più frequenti (come `e`, `a`, `o`, `i`) e parole molto più lunghe alle lettere rare (come `w`, `x`, `k`).

Il codice risultante è riportato in Figura 3.7

Come si può vedere, la lettera `e`, la più frequente con circa l'11.5% delle occorrenze, riceve il codice `000`, appena 3 bit. All'estremo opposto, `x`, che compare con frequenza trascurabile, riceve una parola di 12 bit. La distanza tra questi due estremi (da 3 a 12 bit) riflette direttamente la distanza tra le probabilità dei simboli corrispondenti, su scala logaritmica.

e	0.11522	a	0.11258	o	0.10273	i	0.09923	n	0.07145
r	0.06876	t	0.06025	l	0.06001	s	0.05606	c	0.04273
d	0.03464	u	0.03400	p	0.02771	m	0.02414	v	0.02210
g	0.01926	b	0.01439	h	0.01204	f	0.01189	q	0.00528
z	0.00518	y	0.00018	k	0.00010	w	0.00005	x	0.00000

Figura 3.6: Frequenze di caratteri derivate per la lingua italiana

a	001	b	0111010	c	1010	d	01000
e	000	f	0111110	g	10111	h	0111011
i	100	k	01111111110	l	1110	m	01110
n	0100	o	0110	p	011100	q	01111110
r	1100	s	1111	t	1101	u	01011
v	10110	w	011111111110	x	011111111111	y	0111111110
z	011111110						

Figura 3.7: Codice di Huffman derivato per la lingua italiana

Questo non è soltanto un esercizio accademico. Un file di testo italiano codificato con ASCII standard usa 8 bit per carattere, indipendentemente dalla sua frequenza. Il codice di Huffman calibrato sull'italiano riduce la lunghezza media a circa 4 bit per carattere, con un risparmio di quasi il 50% rispetto alla codifica a lunghezza fissa. È il principio alla base di molti formati di compressione reali.

Huffman contro Shannon–Fano: quale scegliere?

A questo punto è naturale chiedersi: se Huffman è ottimo e Shannon–Fano non lo è, perché mai si userebbe Shannon–Fano? La risposta è che in molti casi pratici la differenza è trascurabile (spesso meno di un decimo di bit per simbolo) e Shannon–Fano è più semplice da implementare e da spiegare. Dal punto di vista teorico, tuttavia, Huffman è superiore: è l'unico dei due a garantire che non esista nessun codice prefisso con lunghezza media inferiore. Questa garanzia di ottimalità ha un valore intrinseco, indipendentemente dall'entità del miglioramento pratico.

Vale anche la pena sottolineare che i due metodi, pur partendo da intuizioni diverse, producono spesso lo stesso codice, o almeno codici con la stessa lunghezza media, su distribuzioni regolari. È nei casi limite, con distribuzioni molto asimmetriche o con probabilità che non sono potenze di $1/2$, che Huffman mostra il suo vantaggio in modo più netto.

3.4 Codici aritmetici

I codici di Huffman sono ottimi nel loro regime: nessun codice prefisso può fare meglio, e su distribuzioni regolari si avvicinano molto al limite entropico. Eppure, come abbiamo visto, presentano un limite strutturale difficile da aggirare: ogni simbolo deve ricevere una codifica di almeno 1 bit, anche quando il suo contenuto informativo è molto inferiore. Per sorgenti con entropia bassa — un simbolo dominante, sequenze altamente prevedibili, o testi in lingue naturali con forti dipendenze contestuali — questo vincolo può diventare costoso.

C'è un secondo limite, forse più sottile. Huffman tratta ogni simbolo in modo indipendente: la lunghezza della codifica di un simbolo dipende dalla sua probabilità marginale, non da quella condizionata al contesto. Ma in una lingua naturale, la probabilità di una lettera dipende fortemente da quelle che la

precedono. In italiano, dopo una `q` la lettera successiva è quasi certamente una `u`: questa informazione contestuale ha un valore enorme ai fini della compressione, e Huffman — nella sua forma base — non la sfrutta.

I codici aritmetici nascono esattamente per superare questi limiti. L'idea di fondo è radicalmente diversa da quella di Huffman: invece di assegnare un codice separato a ogni simbolo, si codifica l'intera sequenza come un unico oggetto. Il risultato è un metodo che può avvicinarsi al limite entropico con precisione arbitraria, sfruttare dipendenze contestuali di qualsiasi ordine, e aggirare del tutto il vincolo del numero intero di bit per simbolo.

L'idea di fondo: sequenze come intervalli

Il cuore della codifica aritmetica è un'idea geometrica di grande eleganza, che vale la pena capire bene prima di entrare nei dettagli algoritmici.

Partiamo da un'osservazione elementare sulle stringhe binarie. Una stringa di m bit identifica univocamente un intervallo di ampiezza 2^{-m} all'interno di $[0, 1)$: basta interpretare la stringa come un numero binario in $[0, 1)$ e considerare l'intervallo che parte da quel punto e si estende per 2^{-m}. Per esempio, la stringa `01` corrisponde al numero binario $0.01 = 0.25$ e identifica l'intervallo $[0.25, 0.5)$, di ampiezza $2^{-2} = 0.25$. La stringa `011` corrisponde a $0.011 = 0.375$ e identifica $[0.375, 0.5)$, di ampiezza $2^{-3} = 0.125$. La stringa `0100` corrisponde a $0.0100 = 0.25$ e identifica $[0.25, 0.3125)$, di ampiezza $2^{-4} = 0.0625$.

Questo sistema ha una proprietà strutturale fondamentale: una stringa è "prefisso" di un'altra se e solo se il suo intervallo "contiene" quello dell'altra. Per convincersene, basta osservare che aggiungere un bit a una stringa equivale a dimezzare il suo intervallo, selezionando la metà inferiore (se il bit è `0`) o quella superiore (se è `1`). L'intervallo della stringa più lunga è quindi sempre contenuto in quello della stringa più corta. Nel nostro esempio, `01` è prefisso di `011`, e infatti $[0.375, 0.5) \subset [0.25, 0.5)$; non è invece prefisso di `10`, e corrispondentemente $[0.5, 0.75) \not\subset [0.25, 0.5)$. La struttura geometrica degli intervalli rispecchia perfettamente la struttura algebrica dei prefissi.

C'è un'altra relazione importante: stringhe più lunghe corrispondono a intervalli più piccoli, e per identificare un intervallo piccolo servono più bit. Più precisamente, per identificare un intervallo di ampiezza w servono circa $-\log_2 w$ bit. Se $w = 2^{-m}$, servono esattamente m bit; se w non è una potenza di 2, servono $\lceil -\log_2 w \rceil$ bit, cioè l'arrotondamento verso l'alto.

Dal punto di vista della compressione

Questi fatti elementari suggeriscono immediatamente una strategia di compressione. Supponiamo di voler codificare una sequenza di simboli $c_1 c_2 \cdots c_n$. Invece di assegnare un codice separato a ogni simbolo, associamo all'intera sequenza un intervallo in $[0, 1)$ la cui ampiezza sia proporzionale alla probabilità della sequenza: più la sequenza è probabile, più grande è l'intervallo. Poi cerchiamo la stringa binaria più corta il cui intervallo è "contenuto" nell'intervallo della sequenza: quella stringa è la nostra codifica.

Perché questo è efficiente? Perché l'ampiezza dell'intervallo è uguale alla probabilità $p(c_1 \cdots c_n)$, e quindi la lunghezza della stringa più corta contenuta in quell'intervallo è circa $-\log_2 p(c_1 \cdots c_n)$, che è esattamente il contenuto informativo della sequenza. La codifica aritmetica usa dunque, per ogni sequenza, un numero di bit proporzionale al suo contenuto informativo — non a quello dei singoli simboli, ma dell'intera sequenza come unità.

Questo è il salto concettuale rispetto a Huffman. Huffman assegna a ogni simbolo un numero intero di bit, e l'errore di arrotondamento si accumula simbolo per simbolo. La codifica aritmetica tratta la sequenza come un tutto, e l'errore di arrotondamento è al massimo qualche bit per l'intera sequenza,

indipendentemente dalla sua lunghezza. Su sequenze lunghe, questo si traduce in una lunghezza media per simbolo arbitrariamente vicina all'entropia.

Esempio 3.4.1

Per fissare le idee, consideriamo una sorgente binaria con $p(\texttt{0}) = 0.8$ e $p(\texttt{1}) = 0.2$. L'entropia è $H(X) \approx 0.72$ bit per simbolo: ogni simbolo porta meno di 1 bit di informazione, e un codice ideale dovrebbe usare meno di 1 bit per simbolo. Huffman, tuttavia, è costretto a usare almeno 1 bit per simbolo, perché ogni simbolo deve ricevere una codifica non vuota. Il gap tra Huffman e il limite teorico è in questo caso quasi il 40%.
Con la codifica aritmetica, invece, l'intera sequenza di n simboli viene codificata con circa $n \cdot H(X)$ bit. Su una sequenza di 10 simboli, questo significa circa 7.2 bit, contro i 10 bit di Huffman simbolo per simbolo. Il vantaggio cresce proporzionalmente alla lunghezza della sequenza.
Per vedere concretamente come funziona la costruzione dell'intervallo, consideriamo la sequenza corta `010`. La suddivisione iniziale di $[0, 1)$ assegna $[0, 0.8)$ al simbolo `0` e $[0.8, 1)$ al simbolo `1`. Il primo simbolo `0` seleziona l'intervallo $[0, 0.8)$. All'interno di questo intervallo, il secondo simbolo `1` seleziona la porzione proporzionale a $p(\texttt{1}) = 0.2$, cioè l'intervallo $[0.64, 0.8)$, di ampiezza 0.16. Il terzo simbolo `0` seleziona la porzione proporzionale a $p(\texttt{0}) = 0.8$ all'interno di $[0.64, 0.8)$, cioè l'intervallo $[0.64, 0.768)$, di ampiezza 0.128.
L'ampiezza finale $0.128 = 0.8^2 \cdot 0.2$ è esattamente la probabilità della sequenza `010` sotto l'ipotesi di simboli indipendenti: $p(\texttt{0})^2 \cdot p(\texttt{1}) = 0.64 \cdot 0.2 = 0.128$. Per identificare un intervallo di ampiezza 0.128, servono circa $-\log_2 0.128 \approx 2.97$ bit. Huffman userebbe 3 bit per 3 simboli (1 bit per simbolo), ma la codifica aritmetica potrebbe codificare la stessa sequenza con 3 bit oppure con un'opportuna scelta riuscire a fare meglio su sequenze più lunghe, dove il risparmio si accumula.

In conclusione: l'idea delle sequenze come intervalli non è solo una metafora geometrica, ma il meccanismo preciso attraverso cui la codifica aritmetica aggira i limiti di Huffman e raggiunge il limite entropico con precisione arbitraria.

Come si costruisce l'intervallo

Il processo di costruzione dell'intervallo avviene passo per passo, un simbolo alla volta. All'inizio, l'intervallo corrente è l'intero $[0, 1)$. A ogni passo, l'intervallo corrente viene suddiviso in sotto-intervalli proporzionali alle probabilità dei simboli dell'alfabeto, e si seleziona il sotto-intervallo corrispondente al simbolo letto. L'intervallo si restringe a ogni passo, e la sua ampiezza finale è esattamente la probabilità dell'intera sequenza.

Per rendere concreto il meccanismo, consideriamo una sorgente con alfabeto $\{\texttt{a}, \texttt{b}, \square\}$ e probabilità iniziali $p(\texttt{a}) = 0.425$, $p(\texttt{b}) = 0.425$, $p(\square) = 0.15$. La suddivisione iniziale di $[0, 1)$ assegna $[0, 0.425)$ ad `a`, $[0.425, 0.85)$ a `b`, e $[0.85, 1)$ a $\square$.

Quando leggiamo il primo simbolo, selezioniamo il sotto-intervallo corrispondente. Se il primo simbolo è `b`, l'intervallo corrente diventa $[0.425, 0.85)$. Questo intervallo viene poi suddiviso internamente nella stessa proporzione, ma usando le probabilità "condizionate" al fatto che il primo simbolo era `b`: per esempio, potrebbe essere $p(\texttt{a}|\texttt{b}) = 0.28$, $p(\texttt{b}|\texttt{b}) = 0.57$, $p(\square|\texttt{b}) = 0.15$. Il secondo simbolo seleziona uno di questi sotto-intervalli, e così via.

Questo è il punto in cui i codici aritmetici superano strutturalmente Huffman: le probabilità condizionate possono variare a ogni passo in funzione del contesto, incorporando automaticamente qualsiasi dipendenza statistica tra i simboli. Il modello probabilistico può essere tanto semplice quanto sofisticato quanto si vuole — dalle semplici frequenze marginali a modelli linguistici contestuali di qualsiasi ordine.

Un esempio di codifica: la sequenza bbba□

Seguiamo la codifica della sequenza bbba□ passo per passo, usando la sorgente con $p(\mathtt{a}) = 0.425$, $p(\mathtt{b}) = 0.425$, $p(\square) = 0.15$.

Primo simbolo: b

L'intervallo corrente è $[0, 1)$. Il simbolo b seleziona il sotto-intervallo $[0.425, 0.85)$. Questo intervallo attraversa il punto 0.5, che è il confine tra i due valori del primo bit della codifica (0 per la metà inferiore, 1 per quella superiore): il primo bit non è ancora determinato e non può essere emesso.

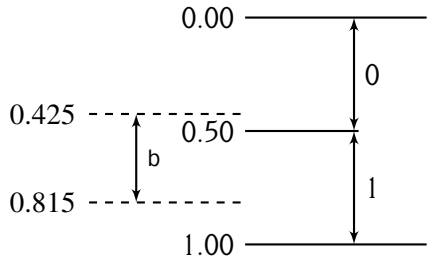

Figura 3.8: Codifica di bbba□. Primo carattere.

Secondo simbolo: b

Con le probabilità condizionate $p(\mathtt{a}|\mathtt{b}) = 0.28$, $p(\mathtt{b}|\mathtt{b}) = 0.57$, $p(\square|\mathtt{b}) = 0.15$, l'intervallo $[0.425, 0.85)$ viene suddiviso internamente e il secondo b seleziona il sotto-intervallo $[0.544, 0.786)$. Questo intervallo è interamente contenuto in $[0.5, 1)$: il primo bit della codifica è dunque 1 e può essere emesso immediatamente.

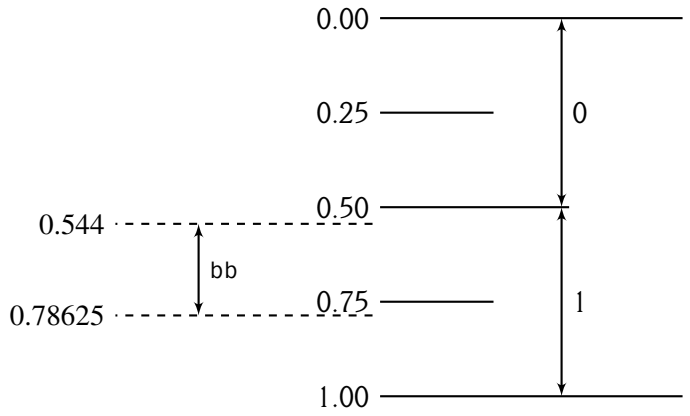

Figura 3.9: Codifica di bbba□. Primi due caratteri.

Terzo simbolo: `b`

Con $p(\texttt{a}|\texttt{bb}) = 0.21$, $p(\texttt{b}|\texttt{bb}) = 0.64$, $p(\square|\texttt{bb}) = 0.15$, l'intervallo si restringe ulteriormente a $[0.5949, 0.7499)$. Questo è contenuto in $[0.5, 0.75)$, che corrisponde al prefisso binario `10`: il secondo bit è `0` e viene emesso. La codifica inizia con `10`.

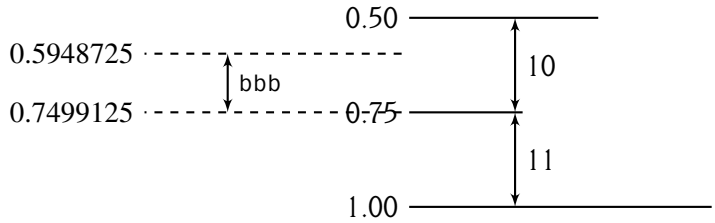

Figura 3.10: Codifica di `bbba`□. Primi tre caratteri.

Quarto simbolo: `a`

Con $p(\texttt{a}|\texttt{bbb}) = 0.17$, l'intervallo diventa $[0.5949, 0.6212)$. Questo è contenuto in $[0.5, 0.625)$ — prefisso `100` — e anche in $[0.5625, 0.625)$ — prefisso `1001`. I bit `0` e `1` vengono emessi: la codifica avanza a `1001`.

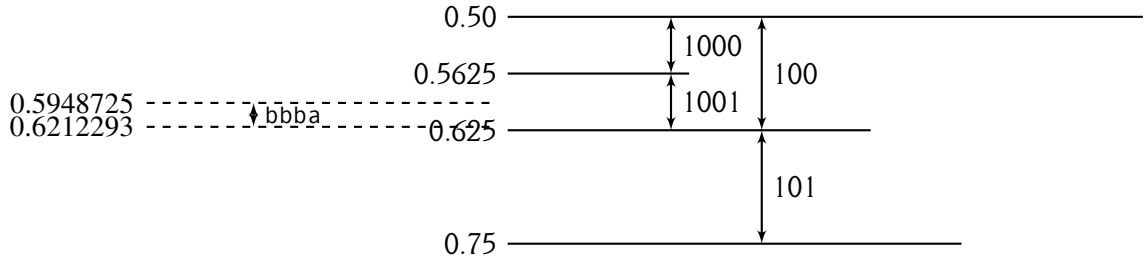

Figura 3.11: Codifica di `bbba`□. Primi quattro caratteri.

Quinto simbolo: □

Con $p(\square|\texttt{bbba}) = 0.15$, l'intervallo finale è $[0.6173, 0.6212)$. Non ci sono altri simboli da leggere: ora bisogna trovare la stringa binaria più corta il cui intervallo è interamente contenuto in $[0.6173, 0.6212)$.

Le figure seguenti mostrano la progressiva suddivisione diadica dell'intervallo $[0.5, 1)$, stringendo sempre di più fino a trovare un intervallo completamente contenuto nell'intervallo obiettivo.

L'intervallo $[0.6191, 0.6211)$ corrispondente alla stringa binaria `100111101` è interamente contenuto nell'intervallo obiettivo $[0.6173, 0.6212)$: la codifica della sequenza `bbba`□ è dunque `100111101`, in soli 9 bit.

Vale la pena confrontare questo risultato con quello che avrebbe prodotto Huffman. Con le probabilità marginali della sorgente, un codice di Huffman assegnerebbe a `a` e `b` codifiche da 2 bit e a □ una codifica da 2 o 3 bit. La sequenza `bbba`□ — cinque simboli — richiederebbe quindi almeno 10 bit, probabilmente 11. La codifica aritmetica usa 9 bit, sfruttando sia le probabilità condizionate sia il fatto che la sequenza, pur non essendo la più probabile in assoluto, è sufficientemente regolare da comprimersi bene.

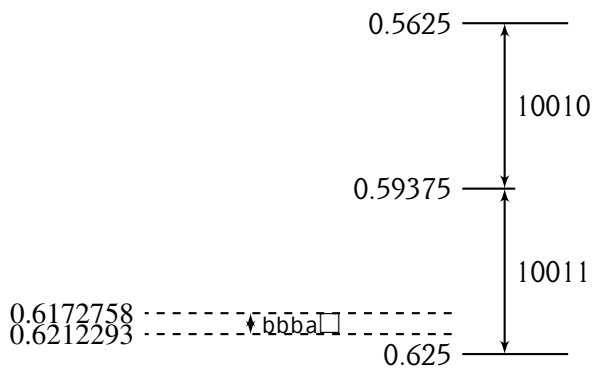

Figura 3.12: Codifica di bbba□. Ricerca della codifica finale.

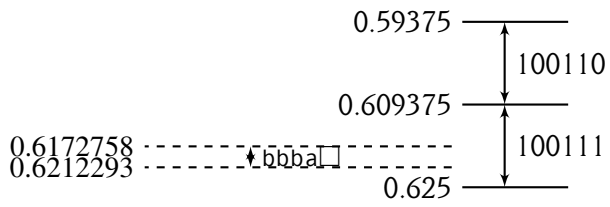

Figura 3.13: Codifica di bbba□. Ricerca della codifica finale.

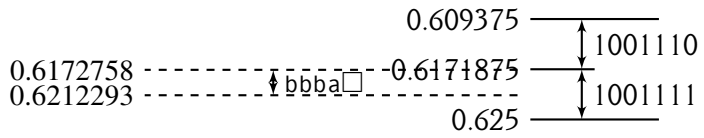

Figura 3.14: Codifica di bbba□. Ricerca della codifica finale.

Come avviene la decodifica

La decodifica è l'operazione inversa, ed è altrettanto elegante. Bruno riceve la stringa binaria `100111101`, che identifica un punto nell'intervallo $[0, 1)$. Usando lo stesso modello probabilistico di Anna, suddivide $[0, 1)$ proporzionalmente alle probabilità dei simboli e identifica in quale sotto-intervallo cade il punto ricevuto: questo gli dice qual è il primo simbolo. Aggiorna poi l'intervallo e il modello contestuale, esattamente come ha fatto Anna, e ripete il processo per il secondo simbolo, il terzo, e così via.

Il punto cruciale è che Anna e Bruno devono usare "esattamente" lo stesso modello probabilistico: anche una piccola discrepanza nelle probabilità condizionate porterebbe a suddivisioni diverse dell'intervallo e quindi a una decodifica errata. Il modello condiviso è, in questo senso, la "chiave" della comunicazione.

Un dettaglio importante riguarda la terminazione: come fa Bruno a sapere quando la sequenza è finita? La soluzione più comune è includere nell'alfabeto un simbolo speciale di fine messaggio — nell'esempio, □ svolge questo ruolo — la cui comparsa segnala a Bruno che può fermarsi.

Il modello probabilistico: statico e adattivo

Fin qui abbiamo ragionato come se il modello probabilistico fosse noto in anticipo e fisso. In pratica, ci sono due approcci principali.

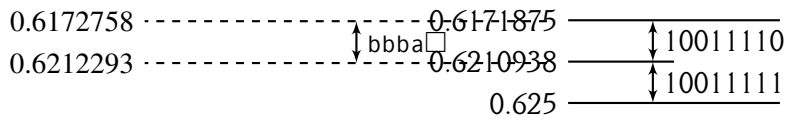

Figura 3.15: Codifica di bbba□. Ricerca della codifica finale.

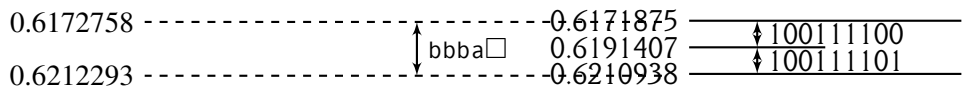

Figura 3.16: Codifica di bbba□. Identificazione della codifica finale.

Il primo è il "modello statico": Anna e Bruno concordano a priori sulle probabilità dei simboli — per esempio, stimate su un grande corpus di testi italiani — e le usano invariate durante tutta la codifica. È semplice e funziona bene quando la sorgente è omogenea e ben caratterizzata.

Il secondo è il "modello adattivo": le probabilità vengono stimate e aggiornate dinamicamente, simbolo per simbolo, a partire dalla sequenza osservata fino a quel momento. All'inizio, in assenza di informazioni, si usano probabilità uniformi o stime iniziali ragionevoli; man mano che i simboli arrivano, il modello si aggiorna contando le frequenze empiriche. Poiché sia Anna sia Bruno applicano la stessa procedura di aggiornamento in modo deterministico — partendo dallo stesso stato iniziale e aggiornando nello stesso ordine — il modello è sempre perfettamente sincronizzato tra i due, senza bisogno di trasmettere esplicitamente le probabilità.

Un dettaglio tecnico importante nell'approccio adattivo riguarda i simboli mai visti prima. Se un simbolo non è ancora apparso nella sequenza, la sua frequenza empirica è zero, e la codifica aritmetica gli assegnerebbe probabilità zero, rendendo impossibile la sua comparsa futura. Per evitare questo problema si usa uno "smoothing": ogni simbolo dell'alfabeto riceve un conteggio iniziale positivo, anche prima di essere osservato. Questo garantisce che tutte le probabilità rimangano strettamente positive in ogni momento, e che il modello sia coerente fin dalla prima osservazione. Man mano che la sequenza cresce, le stime empiriche prendono il sopravvento sui conteggi iniziali, e il modello converge alle frequenze reali della sorgente.

Perché i codici aritmetici sono superiori a Huffman

Ricapitolando, i codici aritmetici offrono tre vantaggi strutturali rispetto a Huffman.

Il primo è la "precisione". Mentre Huffman è costretto ad arrotondare le lunghezze delle codifiche all'intero più vicino, la codifica aritmetica può usare, in media, un numero di bit per simbolo arbitrariamente vicino all'entropia — anche quando questa è molto inferiore a 1 bit per simbolo. Non c'è nessun "+1" residuo: il limite teorico di Shannon è raggiungibile con precisione arbitraria.

Il secondo è la "gestione del contesto". Il modello probabilistico usato dalla codifica aritmetica può essere aggiornato a ogni passo in funzione di tutta la storia precedente della sequenza. Questo permette di sfruttare dipendenze contestuali di qualsiasi ordine, qualcosa che Huffman nella sua forma base non fa.

Il terzo è la "separazione tra modello e codifica". In un codice aritmetico, il modello probabilistico e il meccanismo di codifica sono componenti distinte e intercambiabili: migliorare la compressione significa migliorare il modello, senza dover ridisegnare il codice. Questa modularità è alla base dei compressori

moderni più potenti, che combinano la codifica aritmetica con modelli statistici sofisticati — fino ai modelli linguistici neurali — per avvicinarsi quanto possibile al limite entropico della sorgente.

3.5 Codifica di Lempel–Ziv

I metodi di compressione che abbiamo incontrato finora condividono tutti un presupposto implicito: conoscere in anticipo le probabilità dei simboli della sorgente. Huffman richiede una tabella di frequenze; la codifica aritmetica ha bisogno di un modello probabilistico condiviso tra mittente e destinatario. In molte situazioni reali, però, questo presupposto è difficile da soddisfare. Come si comprime un file di cui non si conosce la struttura? Come si costruisce un compressore che funzioni bene su testi in qualsiasi lingua, su file binari, su sequenze genomiche, senza richiedere alcuna conoscenza preliminare della sorgente?

La risposta, proposta da Abraham Lempel e Jacob Ziv alla fine degli anni Settanta, è sorprendentemente semplice: invece di usare un modello probabilistico fornito dall'esterno, si costruisce un modello direttamente dalla sequenza che si sta comprimendo, mentre la si legge. Il risultato è un algoritmo "universale" che non richiede alcuna conoscenza a priori della sorgente, ma che su sequenze sufficientemente lunghe si comporta in media come un metodo ottimo.

L'idea di fondo: costruire un dizionario al volo

L'intuizione alla base di Lempel–Ziv è molto vicina a qualcosa che facciamo naturalmente quando leggiamo. Immaginiamo di leggere un testo per la prima volta: ogni volta che incontriamo una parola nuova, la aggiungiamo mentalmente al nostro vocabolario; quando incontriamo una parola già vista, non la "impariamo" di nuovo, ma semplicemente la riconosciamo e la colleghiamo a ciò che già sappiamo. Più si legge, più il vocabolario cresce, e più diventa facile riconoscere le parole senza doverle imparare da capo.

Lempel–Ziv funziona in modo analogo con le sottosequenze. Man mano che si scorre la sequenza di input, si individuano nuove "frasi" (sottosequenze di simboli) e le si inserisce in un dizionario che cresce progressivamente. Ogni frase è la più corta sottosequenza che non era ancora presente nel dizionario: si legge carattere per carattere finché si ottiene qualcosa di nuovo, e a quel punto si aggiunge la nuova frase e si riparte.

La compressione nasce da un'osservazione semplice: una sottosequenza che è già nel dizionario può essere trasmessa come un semplice riferimento al dizionario stesso (un numero intero), invece di ripetere tutti i suoi caratteri. Se il riferimento occupa meno bit della sottosequenza originale, si è guadagnato spazio. E man mano che il dizionario cresce, i riferimenti diventano sempre più potenti: una frase lunga può essere trasmessa con pochissimi bit.

Come viene costruito il codice: un esempio

Per capire concretamente come funziona l'algoritmo, seguiamo passo per passo la codifica di una sequenza su un alfabeto di due caratteri $\{a, b\}$:

$$aababbbabaababbbabbabb. \tag{3.19}$$

L'algoritmo legge la sequenza da sinistra a destra, cercando a ogni passo la sottosequenza più corta che non sia ancora nel dizionario.

All'inizio il dizionario è vuoto. Il primo carattere letto è a: non è ancora nel dizionario, quindi costituisce la prima frase. Si aggiunge a al dizionario con identificatore 1 e si riparte.

Il secondo carattere è a: è già nel dizionario. Si legge il carattere successivo: b. La sottosequenza ab non è ancora nel dizionario, quindi costituisce la seconda frase. Si aggiunge ab con identificatore 2 e si riparte.

Si prosegue in questo modo: ogni volta che la sottosequenza corrente è già nel dizionario, si allunga di un carattere; appena si ottiene qualcosa di nuovo, si chiude la frase e si ricomincia. Procedendo sull'intera sequenza, si ottiene la suddivisione:

$$a \mid ab \mid abb \mid b \mid aba \mid abab \mid bb \mid abba \mid bb. \tag{3.20}$$

Il dizionario risultante è:

Frase	a	ab	abb	b	aba	abab	bb	abba
Identificatore	1	2	3	4	5	6	7	8

Si noti che l'ultima sottosequenza bb coincide con la frase 7, già presente nel dizionario: non genera una nuova voce, ma viene trasmessa direttamente come riferimento.

La struttura delle frasi

C'è una proprietà strutturale elegante che vale la pena mettere in evidenza. Ogni frase di k caratteri che entra nel dizionario ha necessariamente come prefisso una frase di $k-1$ caratteri già presente: la nuova frase è stata generata leggendo un carattere in più rispetto a qualcosa già noto. Questo significa che ogni frase può essere descritta in modo compatto come una coppia formata dall'identificatore del suo prefisso e dal carattere finale (con lo 0 a indicare il prefisso vuoto):

Frase	a	ab	abb	b	aba	abab	bb	abba	bb
Codifica	0a	1b	2b	0b	2a	5b	4b	3a	7

La prima frase a ha prefisso vuoto (0) e carattere finale a. La seconda frase ab è l'estensione della frase 1 (a) con il carattere b, quindi si codifica come 1b. La quinta frase aba è l'estensione della frase 2 (ab) con il carattere a, quindi si codifica come 2a. E così via. L'ultima occorrenza di bb, già presente come frase 7, viene trasmessa semplicemente come il riferimento 7, senza carattere aggiuntivo.

Dalla codifica simbolica alla codifica binaria

Per ottenere una stringa di bit, occorre rappresentare sia i caratteri sia gli identificatori in binario. Con a= 0 e b= 1, e con gli identificatori rappresentati in binario con una lunghezza che cresce al crescere del dizionario, si ottiene:

Frase	a	ab	abb	b	aba	abab	bb	abba	bb
Bit	00	11	101	001	0100	1011	1001	0110	0111

La codifica complessiva della sequenza è dunque:

$$\texttt{0011101001010010111001011001 11}. \tag{3.21}$$

La decodifica

La decodifica è speculare alla codifica e non richiede alcuna informazione aggiuntiva. Il decodificatore conosce la stessa regola di costruzione del dizionario, e quindi ricostruisce esattamente lo stesso dizionario leggendo il flusso di bit in entrata: ogni frase decodificata viene aggiunta al dizionario nello stesso ordine in cui l'ha fatto il codificatore. Non è necessario trasmettere il dizionario separatamente: esso emerge in modo deterministico dal flusso compresso stesso. Questa è una delle proprietà più eleganti di Lempel–Ziv, e lo distingue nettamente da Huffman, dove l'albero di codifica deve essere trasmesso insieme al messaggio.

L'efficienza di Lempel–Ziv

Nel nostro esempio, la sequenza originale di 21 caratteri (21 bit in codifica diretta) è diventata una codifica di 30 bit. Lempel–Ziv ha espanso il dato, invece di comprimerlo. Questo non è un errore né un caso sfortunato: è il comportamento atteso su sequenze corte.

Per capire perché, occorre ricordare come nasce la compressione nell'algoritmo. All'inizio, il dizionario è vuoto e quasi ogni frase è nuova: le prime porzioni della sequenza vengono codificate senza alcun risparmio, anzi con un piccolo overhead dovuto alla struttura delle coppie (identificatore, carattere). La compressione compare solo quando il dizionario è abbastanza ricco da permettere riferimenti frequenti a frasi già viste. Su una sequenza di 21 caratteri, il dizionario non fa in tempo a costruirsi.

Questo è il "costo di avviamento" di Lempel–Ziv: su dati corti, il metodo non comprime, e può addirittura espandere. Ma su sequenze lunghe e strutturate il comportamento cambia radicalmente. Si pensi a un testo in italiano di qualche migliaio di caratteri: le parole si ripetono, le sequenze di lettere seguono schemi ricorrenti, e il dizionario accumula rapidamente frasi riutilizzabili. A quel punto, ogni riferimento a una frase lunga può occupare molto meno spazio della frase stessa, e la compressione diventa significativa.

All'estremo opposto, su sequenze molto casuali, in cui ogni sottosequenza è genuinamente nuova e imprevedibile, il dizionario cresce senza accumulare frasi riutilizzabili e la compressione rimane scarsa. Ma questo è il comportamento corretto: una sequenza casuale non ha ridondanza da sfruttare, e nessun metodo di compressione può fare di meglio.

Il risultato teorico fondamentale

Il risultato più importante su Lempel–Ziv riguarda il suo comportamento "in media", su sorgenti che seguono regole statistiche stabili nel tempo (le cosiddette sorgenti "stazionarie ergodiche", che includono quasi tutti i modelli usati in pratica: processi i.i.d., catene di Markov, ecc.).

L'idea chiave è che una sequenza sufficientemente lunga prodotta da una tale sorgente "contiene già in sé" tutta la sua struttura statistica. Lempel–Ziv la impara implicitamente durante la lettura, costruendo un dizionario sempre più accurato man mano che procede.

Il risultato formale è che la lunghezza della codifica tende, su sequenze di n simboli, a $n \cdot H$ bit, dove H è l'entropia per simbolo della sorgente: il minimo teorico assoluto. In altre parole, Lempel–Ziv è "universale": pur non conoscendo nulla della sorgente in anticipo, raggiunge asintoticamente le stesse prestazioni di Huffman o della codifica aritmetica con modello perfetto e noto a priori, senza richiedere alcuna conoscenza preliminare.

Lempel–Ziv nella pratica

La famiglia di algoritmi nata dalle idee di Lempel e Ziv è alla base di molti dei formati di compressione più diffusi. Il formato ZIP, il comando `gzip` su sistemi Unix, il formato PNG per le immagini e molti altri usano varianti di Lempel–Ziv, spesso combinate con ulteriori passaggi di ottimizzazione. La ragione del loro successo è esattamente quella descritta sopra: funzionano bene su una grande varietà di dati senza richiedere alcuna conoscenza preliminare della sorgente, sono efficienti da implementare, e su dati reali si avvicinano notevolmente al limite teorico della compressione.

Rispetto a Huffman e alla codifica aritmetica, Lempel–Ziv occupa una posizione diversa nello spazio dei compromessi pratici. Huffman è semplice e ottimo sui singoli simboli, ma richiede di conoscere le frequenze e di trasmettere l'albero. La codifica aritmetica è potentissima e flessibile, ma richiede un modello probabilistico esplicito e può essere complessa da implementare in modo numericamente stabile. Lempel–Ziv non richiede nulla di tutto questo: basta la sequenza da comprimere, e l'algoritmo fa il resto. Il prezzo è il costo di avviamento su sequenze corte e una convergenza più lenta al limite entropico rispetto alla codifica aritmetica con un buon modello. Ma su dati reali di dimensioni ragionevoli, questo prezzo è quasi sempre accettabile.

3.6 Errori e loro correzione

Quanto discusso fino a questo punto ha fatto invariabilmente riferimento a un'ipotesi molto comoda: il canale di comunicazione tra Anna e Bruno è "perfetto", nel senso che Bruno riceve esattamente ciò che Anna ha inviato. In questo scenario, l'obiettivo principale della codifica è ridurre per quanto possibile la lunghezza del messaggio trasmesso, cioè effettuare "compressione" sfruttando la struttura statistica della sorgente.

Se però rimuoviamo l'ipotesi di canale perfetto, cambia completamente la natura del problema. Diventa possibile che i simboli inviati lungo il canale arrivino "modificati" a destinazione. A questo punto non basta più chiedersi quanto sia corta la rappresentazione: bisogna anche domandarsi quanto sia "affidabile" la trasmissione. In termini operativi, si pone l'esigenza di individuare la presenza di errori nel messaggio giunto a Bruno e, se possibile, di ricostruire il messaggio originale di Anna.

Vale la pena notare che con il termine "canale" non intendiamo necessariamente un mezzo di comunicazione nel senso stretto. Anche un processo di duplicazione o copia di informazione può essere modellato nello stesso modo: ogni volta che una stringa di simboli viene riprodotta da un supporto a un altro, possono verificarsi errori. In questo senso, anche la duplicazione del DNA o la trascrizione in RNA possono essere viste come forme di comunicazione su un canale rumoroso: c'è un messaggio in ingresso, un meccanismo fisico che lo trasforma, e un messaggio in uscita, con la possibilità di introdurre mutazioni.

Canali rumorosi e probabilità di errore

Un canale che non garantisce la trasmissione senza errori dei messaggi viene detto rumoroso. La sua descrizione non è più deterministica, ma "probabilistica": le caratteristiche del canale vengono specificate in termini di probabilità di errore. Per generalità, supponiamo che il canale trasmetta simboli appartenenti a un alfabeto $S = \{s_1, s_2, \dots, s_n\}$. Le proprietà del rumore sono rappresentate tramite una matrice $n \times n$, in cui l'elemento p_{ij} indica la probabilità che un simbolo s_i trasmesso venga ricevuto come s_j.

Nel caso binario $S = \{0, 1\}$, la matrice assume la forma

$$\begin{pmatrix} p_{00} & p_{01} \\ p_{10} & p_{11} \end{pmatrix}, \tag{3.22}$$

dove p_{01} è la probabilità che uno 0 venga ricevuto come 1 e p_{10} è la probabilità che un 1 venga ricevuto come 0. Le probabilità su ciascuna riga sommano a 1. Un caso particolarmente importante è quello in cui i due errori hanno la stessa probabilità, cioè il canale è simmetrico con parametro f:

$$\begin{pmatrix} 1-f & f \\ f & 1-f \end{pmatrix}. \tag{3.23}$$

Il parametro f è la probabilità che un singolo bit venga invertito durante la trasmissione. Un canale "poco rumoroso" ha f piccolo; un canale "molto rumoroso" ha f vicino a 0.5. Se $f = 0.5$, il bit in uscita è completamente indipendente da quello in ingresso: il canale distrugge interamente l'informazione.

La chiave concettuale per affrontare il rumore è introdurre un adeguato livello di ridondanza. La ridondanza è, apparentemente, un costo: significa trasmettere più bit di quelli "necessari" a rappresentare l'informazione. Tuttavia, in presenza di rumore, essa diventa una risorsa: fornisce vincoli aggiuntivi che permettono di inferire quali bit siano stati verosimilmente alterati e di ricostruire il messaggio originale, purché il numero di errori non sia eccessivo. In questo senso, la codifica di canale fa l'opposto della compressione: invece di eliminare ridondanza, la introduce in modo controllato e strutturato.

Codici a ripetizione

Per capire il meccanismo di base della correzione degli errori, partiamo dal metodo più semplice e intuitivo: ripetere ogni bit del messaggio un numero fissato di volte. Questo metodo è detto codice a ripetizione. È inefficiente rispetto ai codici moderni, ma è perfetto per chiarire come la ridondanza possa trasformarsi in capacità di correzione.

L'idea di fondo: ripetizione e voto di maggioranza

L'idea è disarmante nella sua semplicità. Se devo trasmettere un bit e il canale può invertirlo, invece di mandare il bit una sola volta lo mando tre volte. Il destinatario guarda i tre bit ricevuti e sceglie il valore che compare almeno due volte: è il classico "voto di maggioranza". Se il canale inverte al massimo uno dei tre bit, la maggioranza è ancora corretta e l'errore viene corretto. Se invece ne inverte due o tre, la decodifica fallisce.

Supponiamo che Anna debba trasmettere il messaggio $s = 0010110$ su un canale simmetrico con $f = 0.1$. Con ripetizione 3, ogni bit viene trasmesso tre volte e la sequenza effettivamente trasmessa diventa $t = 000000111000111111000$. Il canale rumoroso può essere modellato introducendo una "sequenza di errore" e: l'i-esimo bit di e vale 1 se e solo se il corrispondente bit di t viene invertito. La sequenza ricevuta è

$$r = t \oplus e, \tag{3.24}$$

dove $\oplus$ indica la somma modulo 2 (XOR), cioè l'addizione bit per bit senza riporto.

Perché la maggioranza è giustificata: la regola MAP

La regola di maggioranza non è solo intuitiva: è la scelta ottimale dal punto di vista statistico. Bruno dovrebbe scegliere, per ogni bit s_i, il valore che massimizza la probabilità a posteriori $p(s_i \mid r_{i1}r_{i2}r_{i3})$. Sotto le ipotesi naturali di un canale simmetrico con $f < 0.5$, errori indipendenti e distribuzione a priori uniforme su $\{0, 1\}$, il rapporto tra le probabilità a posteriori si scrive come:

$$\frac{p(r_{i1}r_{i2}r_{i3} \mid s_i = 0)}{p(r_{i1}r_{i2}r_{i3} \mid s_i = 1)} = \prod_{j=1}^{3} \frac{p(r_{ij} \mid s_i = 0)}{p(r_{ij} \mid s_i = 1)}. \tag{3.25}$$

Poiché in un canale simmetrico $p(r_{ij}|s_i) = (1-f)$ se $r_{ij} = s_i$ e f altrimenti, ogni fattore è $\gamma = (1-f)/f > 1$ se il bit ricevuto favorisce $s_i = 0$ e $\gamma^{-1} < 1$ in caso contrario. Il prodotto supera 1 precisamente quando almeno due dei tre bit ricevuti sono 0: questo è esattamente il criterio di maggioranza. La tabella seguente elenca tutti i casi possibili:

$r_{i1}r_{i2}r_{i3}$	$\frac{p(r_{i1}r_{i2}r_{i3}\|s_i=0)}{p(r_{i1}r_{i2}r_{i3}\|s_i=1)}$	s_i
000	$\gamma^3 > 1$	0
001	$\gamma > 1$	0
010	$\gamma > 1$	0
011	$\gamma^{-1} < 1$	1
100	$\gamma > 1$	0
101	$\gamma^{-1} < 1$	1
110	$\gamma^{-1} < 1$	1
111	$\gamma^{-3} < 1$	1

Bruno sceglie 0 quando i tre bit contengono almeno due zeri, e 1 quando contengono almeno due uni.

Quando la correzione fallisce e quanto spesso accade

Consideriamo un esempio concreto:

s	0	0	1	0	1	1	0
t	000	000	111	000	111	111	000
e	000	001	000	000	101	000	000
r	000	001	111	000	010	111	000
s'	0	0	1	0	0	1	0

Il quinto bit viene decodificato erroneamente perché si sono verificati due errori nella stessa tripla. Il codice a ripetizione 3 corregge correttamente quando, in ciascun terzetto, gli errori sono al più uno; fallisce quando sono almeno due.

La probabilità che un terzetto venga decodificato erroneamente è:

$$\binom{3}{2} f^2(1-f) + f^3 = 3f^2(1-f) + f^3. \tag{3.26}$$

Con $f = 0.1$ questa probabilità vale $3 \times 0.01 \times 0.9 + 0.001 = 0.028$, sensibilmente inferiore a 0.1. Il miglioramento è reale, ma ha un costo: la lunghezza del messaggio è triplicata.

Se vogliamo ottenere una probabilità di errore per bit molto piccola, come 10^{-15} (requisito plausibile per copiare senza errori un disco da un terabyte), il numero di ripetizioni necessarie con il solo codice a ripetizione sale a 68: la ridondanza esplode e rende il metodo impraticabile. Questo non significa che la ridondanza sia una cattiva idea, ma che va introdotta in modo più intelligente, usando codici strutturati che correggano molti più errori a parità di ridondanza.

Codici di Hamming

Nella discussione sui codici a ripetizione abbiamo visto un'idea chiara ma poco efficiente. Per ottenere un compromesso più ragionevole tra affidabilità e ridondanza, conviene cambiare prospettiva: invece di

proteggere ogni singolo bit ripetendolo, si proteggono "blocchi di bit" introducendo ridondanza in modo strutturato.

L'idea di fondo: bit di parità e localizzazione degli errori

L'intuizione fondamentale dei codici di Hamming è questa: invece di ripetere ogni bit, si aggiungono alcuni bit speciali — detti "bit di parità" — che tengono traccia di somme modulo 2 di sottoinsiemi dei bit informativi. Se un singolo bit viene alterato, questi controlli di parità non saranno più tutti soddisfatti: il pattern di parità violate (la "sindrome") individua univocamente quale bit è stato alterato. Conoscendo quale bit è sbagliato, basta invertirlo per correggere l'errore.

La somma modulo 2 di un insieme di bit vale 0 se il numero di 1 è pari, e 1 se è dispari. Un bit di parità viene scelto esattamente in modo da rendere pari la somma modulo 2 dell'insieme che controlla: è come aggiungere un testimone che dice "in questo gruppo c'è un numero pari di 1". Se un bit del gruppo viene alterato, il testimone rivela il guasto.

Un codice a blocchi lineare prende una sequenza s di k bit (il messaggio) e la trasforma in una sequenza t di $n > k$ bit (la parola trasmessa), aggiungendo $n - k$ bit di ridondanza. Un codice (n, k) di Hamming usa parole di lunghezza n per codificare blocchi di k bit informativi, con soli $r = n - k$ bit di parità.

Un primo esempio: un codice $(3, 2)$ per la segnalazione di errore

Per introdurre gradualmente l'idea, partiamo da un esempio elementare. Dati due bit $s_1 s_2$, aggiungiamo un terzo bit

$$t_3 = s_1 \oplus s_2. \tag{3.27}$$

Il terzo bit vale 0 se s_1 e s_2 sono uguali, 1 altrimenti. La tabella di codifica è:

s	00	01	10	11
t	000	011	101	110

Introduciamo ora la distanza di Hamming $\delta(x, y)$ tra due sequenze binarie x e y: è il numero di posizioni in cui esse differiscono, cioè il numero di bit che bisogna cambiare per trasformare x in y. In questo codice, due qualunque parole di codice differiscono sempre per esattamente due bit: la distanza minima è $d_{\min} = 2$.

Se si verifica un singolo errore, la parola ricevuta è a distanza 1 dalla parola trasmessa. Poiché tutte le parole valide sono tra loro a distanza almeno 2, una parola a distanza 1 da una parola valida non coincide con un'altra parola valida: l'errore è "rilevabile". Tuttavia, con $d_{\min} = 2$ non si può "correggere" l'errore: se riceviamo 001, essa è a distanza 1 sia da 000 sia da 011, quindi non c'è una scelta univoca su quale parola fosse stata trasmessa. Il codice $(3, 2)$ è quindi un "codice a segnalazione di errore": rileva errori singoli, ma non li corregge.

La regola generale è la seguente. Un codice con distanza minima $d_{\min}$ può rilevare fino a $d_{\min} - 1$ errori e correggere fino a $\lfloor (d_{\min} - 1)/2 \rfloor$ errori. Nel caso $(3, 2)$, $d_{\min} = 2$ implica rilevazione di 1 errore e correzione di 0 errori. Per correggere almeno un errore occorre $d_{\min} \geq 3$, che è il requisito dei codici di Hamming veri e propri.

Il codice di Hamming $(7,4)$: struttura e bit di parità

Il codice di Hamming $(7,4)$ codifica 4 bit informativi in 7 bit aggiungendo 3 bit di parità. Come illustrato in Figura 3.17, i 7 bit sono collocati nelle regioni determinate da tre cerchi che si intersecano. Ciascun cerchio corrisponde a una condizione di parità.

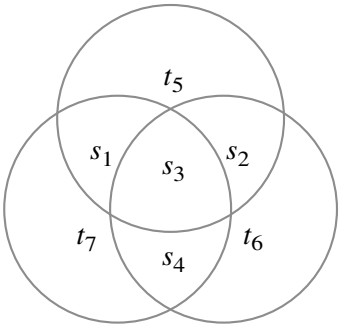

Figura 3.17: Rappresentazione grafica del codice di Hamming $(7,4)$

I bit di parità t_5, t_6, t_7 sono definiti in modo che la somma modulo 2 di tutti i bit in ciascun cerchio sia zero:

$$t_5 = s_1 \oplus s_2 \oplus s_3, \qquad t_6 = s_2 \oplus s_3 \oplus s_4, \qquad t_7 = s_1 \oplus s_3 \oplus s_4. \tag{3.28}$$

Queste formule non sono arbitrarie: sono scelte in modo che, quando si verifica un errore singolo, il pattern di parità violate identifichi "quale" bit è stato alterato. Come vedremo, questo meccanismo funziona perché ogni bit occupa una combinazione unica di cerchi.

La parola trasmessa è $s_1 s_2 s_3 s_4 t_5 t_6 t_7$. La tabella seguente riporta le codifiche per tutti i possibili blocchi di 4 bit:

0000	0001	0010	0011	0100	0101	0110	0111
0000000	0001011	0010111	0011100	0100110	0101101	0110001	0111010
1000	1001	1010	1011	1100	1101	1110	1111
1000101	1001110	1010010	1011001	1100011	1101000	1110100	1111111

Si può verificare che due qualunque codifiche differiscono per almeno 3 bit: la distanza minima è $d_{\min} = 3$. Questo è sufficiente per correggere un errore singolo.

Decodifica per minima distanza e correzione tramite sindrome

La regola concettualmente più semplice per decodificare è la "decodifica a minima distanza": dato un vettore ricevuto r, scegliamo la parola di codice t che minimizza $\delta(r,t)$. Nel codice di Hamming $(7,4)$, questa regola coincide con la correzione di un singolo errore.

In pratica, la correzione avviene tramite la sindrome. Quando si riceve una parola, si ricalcolano le tre parità e si osserva quali condizioni risultano violate. Il risultato è una tripletta di bit (uno per cerchio) che indica se la parità è corretta. Se un singolo bit è stato invertito, appartiene a un certo sottoinsieme di cerchi, e solo le parità di quei cerchi risulteranno violate. Il pattern di parità violate è quindi una firma univoca della posizione dell'errore.

Nel codice $(7,4)$, con 3 bit di sindrome si possono distinguere $2^3 = 8$ casi: la sindrome nulla (nessuna parità violata, nessun errore) e le 7 sindromi non nulle, ciascuna corrispondente a uno dei 7 bit in errore. Conoscere quale bit è in errore permette di correggerlo semplicemente invertendolo.

Ad esempio, se Bruno riceve $r = 1001101$, la parola di codice a distanza minima è $t = 1000101$: l'errore è nel quarto bit e la decodifica restituisce 1000.